TRƯỜNG ĐẠI HỌC QUỐC TẾ HỒNG BÀNG
BỘ MÔN CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI: "THIẾT KẾ HỆ HỖ TRỢ QUẢN TRỊ CƠ SỞ TRI THỨC DẠNG COKB"

GVHD: Gs. Đỗ Văn Nhơn & Ths. Mai Trung Thành
SVTH: Lê Châu Trần Phát - 2211110068
Lớp: TH22DH-CN2

TP. Hồ Chí Minh, 2026

MỤC LỤC

1 Giới thiệu và Đặt vấn đề **10**
1.1 Giới thiệu đề tài và Đặt vấn đề . . . 10
1.1.1 Lý do chọn đề tài (Motivation) . . . 10
1.1.2 Mục tiêu nghiên cứu (Research Objectives) . . . 10
1.1.3 Đối tượng và Phạm vi nghiên cứu . . . 10
1.1.4 Ý nghĩa khoa học và thực tiễn . . . 10

2 Cơ sở Lý thuyết COKB **11**
2.1 Mô hình Đối tượng Tính toán COKB . . . 11
2.1.1 Thành phần Hệ thống tri thức COKB . . . 11
2.1.2 Mô hình Đối tượng Tính toán (Computational Object) . . . 11
2.1.3 Phân cấp Khái niệm (Concept Levels) . . . 11
2.2 Cơ chế Suy luận và Giải quyết vấn đề . . . 12
2.2.1 Các Quy tắc Suy luận (Inference Rules) . . . 12
2.2.2 Thuật toán Tìm bao đóng (F-Closure Algorithm) . . . 12
2.2.3 Chiến lược Giải bài toán: Heuristics . . . 12

3 Phân tích và Thiết kế Hệ thống **13**
3.1 Khảo sát Hiện trạng & Mục tiêu (Survey & Objectives) . . . 13
3.1.1 Khảo sát Hiện trạng (Current State Survey) . . . 13
3.1.2 Mục tiêu Nghiên cứu (Research Objectives) . . . 14
3.2 Tác nhân & Vai trò Hệ thống (Actors & Roles) . . . 14
3.2.1 Phân loại Người dùng . . . 14
3.2.2 Mô hình Phân quyền (RBAC) . . . 14
3.3 Sơ đồ Hoạt động Tổng quát (Visual Overview) . . . 14
3.3.1 Sơ đồ Use Case Tổng quát . . . 15
3.3.2 Sơ đồ Sequence Hệ thống . . . 15
3.4 Đặc tả Yêu cầu Hệ thống Master (Master Requirements) . . . 16
3.4.1 Yêu cầu Chức năng (Functional Requirements) . . . 16
3.4.2 Yêu cầu Chức năng theo Tầng Kiến trúc (4 Tiers) . . . 16
3.4.3 Bản đồ Chức năng KBQL (Detailed Registry) . . . 17
3.4.4 Yêu cầu Phi chức năng (Non-functional) . . . 18

4 Kiến trúc Hệ thống và các Tầng xử lý **19**
4.1 Tổng quan Kiến trúc (System Overview) . . . 19
4.1.1 Mô hình Kiến trúc 4 Tầng . . . 19
4.1.2 Luồng Yêu cầu Tổng quát . . . 20
4.1.3 Bảo mật & Chẩn đoán Hệ thống . . . 21
4.1.4 Công nghệ Sử dụng (Technology Stack) . . . 21
4.2 Tầng Ứng dụng (Application Layer) . . . 22
4.2.1 KBMS Studio (IDE Chuyên nghiệp) . . . 22
4.2.2 KBMS CLI (Command Line Interface) . . . 22
4.3 Tầng Mạng (Network Layer) . . . 22
4.3.1 Giao thức Nhị phân (Custom Binary Protocol) . . . 22
4.3.2 Quản lý Kết nối (Connection Manager) . . . 22
4.4 Tầng Máy chủ (Server Layer) . . . 23
4.4.1 Knowledge Manager (Orchestrator) . . . 23
4.4.2 KBQL Parser & Compiler . . . 23
4.4.3 Reasoning Engine (Bộ máy Suy diễn) . . . 23

4.4.4 Dịch vụ Bảo mật & Xác thực (Security & Auth) . . . 23
4.4.5 Hệ thống Nhật ký & Chẩn đoán (Native Logging) . . . 23
4.5 Tầng Lưu trữ (Storage Layer) . . . 24
4.5.1 Ánh xạ Thành phần Hệ thống . . . 24
4.5.2 Quản lý Trang (Physical Paging) . . . 24
4.5.3 Cấu trúc Chỉ mục (B+ Tree Indexing) . . . 24
4.5.4 Điều phối Truy cập Đồng thời (Latching & Pinning) . . . 25
4.5.5 Bảo mật Dữ liệu (Encryption) . . . 25
4.6 Đối soát Yêu cầu Kỹ thuật (Requirements Spec) . . . 25
4.6.1 Yêu cầu Chức năng (Functional Requirements) . . . 25
4.6.2 Yêu cầu Phi chức năng (Non-functional Requirements) . . . 25

5 Định nghĩa Mô hình và Khái niệm 27
5.1 Định nghĩa Mô hình tri thức dạng COKB . . . 27
5.1.1 Thành phần C (Concepts - Tập các khái niệm) . . . 28
5.1.2 Thành phần H (Hierarchy - Phân cấp tri thức) . . . 28
5.1.3 Thành phần R (Relations - Quan hệ ngữ nghĩa) . . . 29
5.1.4 Thành phần Rules & Logic (Bộ máy suy diễn) . . . 31
5.1.5 Thành phần Ops & Funcs (Bộ máy thực thi) . . . 31
5.1.6 Thực thể Đối tượng (ObjectInstance) . . . 32
5.1.7 Ví dụ minh họa thực tế cho mô hình COKB . . . 32

6 Ngôn ngữ Truy vấn Tri thức KBQL 34
6.1 Giới thiệu về Ngôn ngữ Truy vấn KBQL . . . 34
6.1.1 Triết lý Thiết kế . . . 34
6.1.2 Các Thành phần Chính của KBQL . . . 34
6.1.3 Hỗ trợ Chỉnh sửa (ALTER Support) . . . 34
6.1.4 Kiểu Dữ liệu Hỗ trợ (Data Types) . . . 35
6.1.5 Khái niệm Cốt lõi . . . 35
6.2 Định nghĩa Tri thức (KDL) . . . 35
6.2.1 Quản lý Cơ sở Tri thức (Knowledge Base) . . . 35
6.2.2 Định nghĩa Khái niệm (Concept) . . . 36
6.2.3 Quản lý Phân cấp (Hierarchy) . . . 37
6.2.4 Định nghĩa Quan hệ (Relation) . . . 37
6.2.5 Định nghĩa Luật (Rule) . . . 37
6.2.6 Toán tử & Hàm Tùy biến (Operator & Function) . . . 37
6.2.7 Công thức tự động (Computation) . . . 37
6.2.8 Chỉ mục (Index) . . . 38
6.2.9 Sự kiện tự động (Trigger) . . . 38
6.3 Thao tác tri thức (KML) . . . 38
6.3.1 Chèn Sự kiện (INSERT) . . . 38
6.3.2 Cập nhật Sự kiện (UPDATE) . . . 38
6.3.3 Xóa Sự kiện (DELETE) . . . 38
6.3.4 Chuyển đổi Dữ liệu (IMPORT / EXPORT) . . . 39
6.3.5 Đặc điểm của KML trong KBMS . . . 39
6.4 Truy vấn và Suy diễn (KQL) . . . 39
6.4.1 Truy vấn Dữ liệu (SELECT) . . . 39
6.4.2 Giải quyết Tri thức (SOLVE) . . . 40
6.4.3 Kiểm tra Hệ thống (SHOW) . . . 40
6.4.4 Phân tích & Mô tả (DESCRIBE / EXPLAIN) . . . 40
6.4.5 Truy vấn Metadata (Hệ thống) . . . 40
6.5 Kiểm soát truy cập (KCL) . . . 41
6.5.1 Quản lý Người dùng (USER Management) . . . 41
6.5.2 Quản trị Quyền hạn (GRANT / REVOKE) . . . 41
6.5.3 Các thực thể Quản trị đặc biệt (System Admin) . . . 41
6.6 TCL: Ngôn ngữ Kiểm soát Giao dịch (Transaction Control Language) . . . 41
6.6.1 Một Giao dịch là gì? (Transaction) . . . 41
6.6.2 Các lệnh Giao dịch . . . 42
6.6.3 Các lệnh Giao dịch . . . 42
6.6.4 Vai trò của Giao dịch trong KBMS . . . 42
6.7 Bảo trì Hệ thống (Maintenance) . . . 42

6.7.1 Lệnh Bảo trì (MAINTENANCE) . . . 42
6.7.2 Các hành động cụ thể . . . 42
6.7.3 Tần suất thực hiện bản khuyến nghị . . . 43
6.8 Biểu thức và Hàm số (Expressions) . . . 43
6.8.1 Toán tử Cơ bản (Operators) . . . 43
6.8.2 Hàm Tích hợp (Built-in Functions) . . . 43
6.8.3 Cách sử dụng trong CALC() . . . 44
6.8.4 Tương tác với Biến (Variables) . . . 44
6.8.5 Lưu ý về Hiệu năng . . . 44
6.9 Các trường hợp Chuyên gia (Expert Cases) . . . 44
6.9.1 Giải ngược Phương trình (Inverse Problem Solving) . . . 44
6.9.2 Suy diễn Chùm (Chained Inference) . . . 44
6.9.3 Phân cấp Kế thừa Đa tầng (Recursive Property Propagation) . . . 44
6.9.4 Ràng buộc nội bộ phức tạp (Internal Constraints) . . . 45
6.9.5 Giải hệ phương trình 2 ẩn (Newton-Raphson 2D) . . . 45
6.9.6 Kết hợp Trigger và Rule (The Chain Reaction) . . . 45
6.10 Xác thực Ngôn ngữ (Language Validation) . . . 46
6.10.1 Kiểm thử Cú pháp (Parsing Test) . . . 46
6.10.2 Bằng chứng Thực nghiệm (Evidence) . . . 46
6.10.3 Nhật ký Lexer (Lexing Proof) . . . 46

7 Cơ chế Lưu trữ và Quản lý Dữ liệu nhị phân 47
7.1 Tham chiếu Thuật ngữ Kỹ thuật (Technical Glossary) . . . 47
7.1.1 Đơn vị lưu trữ và Cấu trúc dữ liệu . . . 47
7.1.2 Quản lý Bộ nhớ và I/O . . . 47
7.1.3 Độ bền và An toàn dữ liệu . . . 47
7.1.4 Cơ chế chuyển đổi dữ liệu . . . 48
7.2 Tổng quan về Kiến trúc Lưu trữ (Storage Architecture) . . . 48
7.2.1 Hệ thống Phân tầng I/O (Multi-layered I/O) . . . 48
7.2.2 Sơ đồ Kiến trúc Thành phần (Component Architecture) . . . 48
7.2.3 Quản lý Bộ đệm (Buffer Pool Manager) . . . 48
7.2.4 Hạn mức Kỹ thuật của Kiến trúc V3 . . . 48
7.2.5 Lộ trình Triển khai Kỹ thuật . . . 49
7.3 Giải phẫu Trang vật lý (Page Anatomy) . . . 49
7.3.1 Cấu trúc nội tại của Trang (Internal Structure) . . . 49
7.3.2 Đặc tả Header (24-Byte Layout) . . . 49
7.3.3 Quản lý bản ghi biến thiên (Slotted Page Logic) . . . 50
7.3.4 Lợi ích của kiến trúc 16KB . . . 50
7.4 Chỉ mục Cây B+ Tree (Indexing Structure) . . . 50
7.4.1 Cấu trúc Đa tầng (Tiered Structure) . . . 50
7.4.2 Các Thuật toán Vận hành Cốt lõi . . . 51
7.4.3 Clustered Index trong KBMS . . . 52
7.5 Ánh xạ Mô hình sang Dữ liệu nhị phân (Model Serialization) . . . 52
7.5.1 Chiến lược Nhất quán và Linh hoạt . . . 52
7.5.2 Đặc tả Kỹ thuật của ‘ObjectInstance‘ Tuple . . . 52
7.5.3 Tuần tự hóa Siêu dữ liệu (Metadata Serialization) . . . 53
7.5.4 Cơ chế Phục hồi Kiểu dữ liệu (Hydration & Casting) . . . 53
7.6 Độ bền & Nhật ký ghi trước (Durability & WAL) . . . 53
7.6.1 Giao thức Write-Ahead Logging (WAL) . . . 53
7.6.2 Số tuần tự nhật ký (LSN - Log Sequence Number) . . . 54
7.6.3 Quy trình Phục hồi sau sự cố (Recovery) . . . 54
7.6.4 Cơ chế Checkpoint . . . 54
7.7 Quản lý Vật lý & Mã hóa (Physical Disk Manager) . . . 54
7.7.1 Ánh xạ Trang vật lý (Physical Block Mapping) . . . 55
7.7.2 Mã hóa dữ liệu tĩnh (AES-256 Encryption) . . . 55
7.7.3 Quản lý tệp tin và Cấp phát (Allocation) . . . 55
7.8 Thực tế hóa: Từ Mô hình đối tượng sang Chuỗi nhi phân . . . 55
7.8.1 Đối tượng đầu vào (C# Layer) . . . 55
7.8.2 Tuần tự hóa thành Tuple (Serialization) . . . 56
7.8.3 Bản đồ vị trí trong Trang (Slotted Page Mapping) . . . 56
7.8.4 Biểu diễn dưới dạng mã Hex (Hex Dump Trace) . . . 56

8 Bộ máy Suy diễn và Thuật toán Bao đóng **57**
8.0.1 Kiến trúc Điều phối Reasoning . . . 57
8.0.2 Nguyên lý Đóng kín Tri thức (Knowledge Closure) . . . 58
8.0.3 Biểu diễn Tri thức (Knowledge Representation) . . . 58
8.0.4 Quy trình Suy diễn (Reasoning Lifecycle) . . . 59
8.0.5 Chiến lược Suy diễn: Forward vs. Backward Chaining . . . 59
8.0.6 Giải trình và Truy vết (Traceability) . . . 60
8.0.7 Kế hoạch Triển khai Mã nguồn Engine (Implementation Strategy) . . . 60
8.0.8 Các Thuật toán Suy diễn Cốt lõi (Forward Chaining - FClosure) . . . 60
8.0.9 Giải phương trình 1D (Brent's Method - RC5) . . . 62
8.0.10 Giải hệ phương trình 2D (Newton-Raphson - RC5) . . . 62
8.0.11 Phân tích Độ phức tạp (Complexity Analysis) . . . 62
8.0.12 Nhật ký Suy diễn (Reasoning Trace Example) . . . 62
8.1 Xác thực Suy diễn (Reasoning Validation) . . . 63
8.1.1 Kiểm thử Thuật toán F-Closure . . . 63
8.1.2 Kiểm thử Logic Toán học (Solver Testing) . . . 63

9 Giao thức Mạng và Truyền tải Dữ liệu **64**
9.1 Giao thức Mạng (Network Protocol) . . . 64
9.2 Giao thức Truyền dẫn Nhị phân (Binary Protocol) . . . 64
9.2.1 Cấu trúc Gói tin (Packet Structure) . . . 64
9.2.2 Các loại Thông điệp (Message Types) . . . 64
9.2.3 Thuật toán Xử lý Luồng tin (Streaming Algorithm) . . . 65
9.2.4 Bảo mật và Kết nối . . . 65
9.2.5 Kiến trúc Xử lý Chịu tải (Concurrency Threading) . . . 65
9.2.6 Vòng đời của một Truy vấn (Query Lifecycle) . . . 66
9.2.7 Luồng xử lý Packet (Packet Processing Flow) . . . 66
9.2.8 Phân tích Từ vựng (Lexer) . . . 68
9.2.9 Phân tích Cú pháp (Parser & AST) . . . 68
9.2.10 Thực thi và Cấp phát (Executor & Router) . . . 68
9.2.11 Lập kế hoạch Thực thi (Query Execution Plan) . . . 68
9.2.12 Ví dụ luồng xử lý thực tế . . . 68
9.2.13 Kế hoạch Triển khai Mã Nguồn Backend (Implementation Strategy) . . . 69
9.3 Xác thực Giao thức (Protocol Validation) . . . 69
9.3.1 Kiểm thử Gói tin Nhị phân (Binary Packet Testing) . . . 69
9.3.2 Kiểm thử Tải Kết nối (Concurrency Proof) . . . 69

10 Hệ thống Máy chủ và Quản trị Tài nguyên **71**
10.1 Điều phối Mạng & Luồng Hoạt động . . . 71
10.1.1 Mô hình Xử lý Luồng (Async Threading Model) . . . 71
10.1.2 Quản lý Phiên (Session Management) . . . 72
10.1.3 Vòng đời của một Giao dịch (Lifecycle) . . . 72
10.2 Kiến trúc Bộ não Điều phối (Knowledge Engine) . . . 72
10.2.1 Cơ chế Bộ hạ lệnh (The Dispatcher) . . . 72
10.2.2 Các Công nghệ Liên kết Tri thức (Knowledge Binding) . . . 73
10.2.3 Quản lý Giao dịch (Transaction Control) . . . 73
10.3 Hạ tầng V3 (Infrastructure) . . . 73
10.3.1 Cơ chế Tự nạp Hệ thống (System Bootstrapper) . . . 73
10.3.2 Các Danh mục Chuyên biệt (Catalogs) . . . 74
10.3.3 Hệ thống Cập nhật & Di trú (Updater & Migrator) . . . 74
10.4 Quản trị & Đo lường từ xa (Telemetry) . . . 74
10.4.1 Giám sát Hệ thống (Stats & Sessions) . . . 75
10.4.2 Hệ thống Nhật ký Luồng (Live Log Streaming) . . . 75
10.4.3 Lệnh Quản trị Tối cao (Management Command) . . . 75
10.5 Xác thực Máy chủ (Server Validation) . . . 75
10.5.1 Kiểm thử Khởi động (Startup Proof) . . . 75
10.5.2 Kiểm thử Tích hợp Đa luồng (Multi-threading Proof) . . . 76
10.5.3 Xác thực Bảo mật (Security & RBAC) . . . 76

11 Bộ Phân tích Cú pháp và Trình biên dịch **77**
11.1 Lexing: Phân tích Từ vựng chuyên sâu (Lexer) . . . 77
11.1.1 Cơ chế Quét (Scanning Mechanism) . . . 77
11.1.2 Xử lý các Hằng số (Literals) . . . 77
11.1.3 Quản lý Từ khóa (Keyword Dictionary) . . . 78
11.2 Tầng Parser: Cơ chế Đệ quy Đi xuống (Recursive Descent) . . . 78
11.2.1 Cơ chế Điều hướng (Sentence Dispatching) . . . 78
11.2.2 Kỹ thuật Dự đoán & Tiêu thụ (Look-ahead) . . . 79
11.2.3 Phân tích Khối Lệnh phức tạp (Complex Parsing) . . . 79
11.3 Hệ thống Cây cú pháp Trừu tượng (AST Nodes) . . . 79
11.3.1 Hệ phả KDL (Knowledge Definition Language) . . . 80
11.3.2 Hệ phả KQL & KML (Query & Maintenance) . . . 81
11.3.3 Hệ phả KCL & TCL (Control & Transaction) . . . 82
11.3.4 Đặc điểm chung của mọi Node . . . 83
11.4 Bộ máy Đánh giá Biểu thức (Expression Engine) . . . 83
11.4.1 Phân tích Biểu thức Nhị phân (Binary Expressions) . . . 83
11.4.2 Các Đơn vị Biểu thức Cốt lõi (Core Expression Units) . . . 84
11.4.3 Xử lý Biểu thức Đơn phân (Unary Expressions) . . . 84
11.5 Xác thực Bộ biên dịch (Parser Validation) . . . 84
11.5.1 Thống kê Độ bao phủ Testing . . . 84
11.5.2 Đặc tả Lỗi Cú pháp (Syntax Error Proof) . . . 85

12 Giao diện Dòng lệnh (CLI) **86**
12.1 Tổng quan Giao diện Dòng lệnh (CLI) . . . 86
12.1.1 Các Tính năng Chính . . . 86
12.1.2 Các Lệnh Hệ thống Đặc biệt . . . 86
12.1.3 Trải nghiệm người dùng (UX) . . . 86
12.2 Luồng Xử lý Dữ liệu CLI . . . 87
12.2.1 Luồng Gửi lệnh (Command Execution Flow) . . . 87
12.2.2 Bộ máy Phân tích Phản hồi (‘ResponseParser‘) . . . 87
12.2.3 Chế độ Chạy Script (Batch Source) . . . 87
12.2.4 Đa luồng & Kết nối (Multi-threading) . . . 88
12.3 Các Trường hợp Sử dụng CLI . . . 89
12.3.1 Kịch bản 1: Phân tích Cấu trúc (Structure Analysis) . . . 89
12.3.2 Kịch bản 2: Tự động hóa Nạp Tri thức (Batch Loading) . . . 89
12.3.3 Kịch bản 3: Kiểm tra Kết nối & An ninh (Security Check) . . . 89
12.4 Xác thực CLI (CLI Validation) . . . 90
12.4.1 Kiểm thử Phiên làm việc (Session Logs) . . . 90
12.4.2 Kiểm thử Định dạng Kết quả . . . 90

13 Môi trường Phát triển KBMS Studio **91**
13.1 Tổng quan & Tính năng KBMS Studio là một môi trường phát triển tích hợp (IDE) hiện đại, được xây dựng trên nền tảng **Electron** và **React**, cung cấp trải nghiệm trực quan hóa tri thức thay vì chỉ thao tác dòng lệnh thuần túy. . . . 91
13.1.1 Các Phân hệ Cốt lõi . . . 91
13.1.2 Tiêu chuẩn Giao diện (Aesthetics) . . . 92
13.2 Kiến trúc 4 Tầng & Luồng Xử lý Chi tiết . . . 92
13.2.1 Sơ đồ Luồng xử lý 4 Tầng (4-Tier Flow) . . . 92
13.2.2 Đặc tả các Tầng . . . 92
13.2.3 Luồng Thông báo Thời gian thực (Notification Flow) . . . 93
13.2.4 Quy trình Đăng nhập (Authentication Flow) . . . 93
13.3 Các Trường hợp Sử dụng Studio . . . 93
13.3.1 Kịch bản 1: Thiết kế Bản đồ Tri thức Hình học (KDL Design) . . . 94
13.3.2 Kịch bản 2: Giải toán Tự động (Reasoning & Solving) . . . 94
13.3.3 Kịch bản 3: Quản trị Hệ thống (Advanced Management) . . . 95
13.3.4 Kịch bản 4: Giám sát An ninh & Phiên làm việc (Security & Session Management) . . . 96
13.3.5 Kịch bản 5: Phân tích Nhật ký Chuyên sâu (Deep Log Analysis) . . . 96
13.3.6 Tổng kết Giá trị . . . 97
13.4 Xác thực Studio (Studio Validation) . . . 97
13.4.1 Kiểm thử IntelliSense & Đồ họa . . . 97
13.4.2 Kiểm thử Giám sát & Quản lý (Management Proof) . . . 98

14 Cài đặt, Kiểm thử và Đánh giá Hiệu năng **99**
14.1 Hướng dẫn Cài đặt & Triển khai . . . 99
14.1.1 Yêu cầu Tiên quyết (Prerequisites) . . . 99
14.1.2 Cài đặt trên Windows 10/11 . . . 99
14.2 Cài đặt dependencies . . . 100
14.2.1 Xác thực Cài đặt (Health Check) . . . 100
14.3 Kịch bản Kiểm thử Hệ thống . . . 100
14.3.1 Môi trường Cấu hình . . . 100
14.3.2 Kết quả 12 Kịch bản tiêu chuẩn . . . 101
14.3.3 Tổng hợp Kết quả Kiểm thử Toàn hệ thống . . . 101
14.4 Đánh giá Hiệu năng (Performance Benchmarks) . . . 101
14.4.1 Hiệu năng theo Tầng Kiến trúc . . . 101
14.4.2 Hiệu năng Truy vấn Khối lượng (Scale Testing) . . . 101
14.4.3 Nhật ký Xác thực Hiệu năng . . . 102
14.5 Các Bộ Dữ liệu Kiểm thử (Test Data Sets) . . . 102
14.5.1 Danh sách Datasets Chuẩn . . . 102
14.5.2 Chi tiết Bộ dữ liệu trọng tâm: ”Charlie”(Suy diễn Đệ quy) . . . 102
14.5.3 Nhật ký Logic suy diễn (Log Trace) . . . 102

15 Tài liệu Tham khảo **104**
15.1 Danh mục Tài liệu tham khảo (References) . . . 104
15.1.1 Tài liệu Tiếng Việt . . . 104
15.1.2 Tài liệu Tiếng Anh . . . 104

DANH MỤC HÌNH ẢNH

3.1 Sơ đồ Use Case tổng quát sự tương tác giữa người dùng và các dịch vụ lõi. 15
3.2 Sơ đồ Sequence mô tả luồng sinh mệnh của một yêu cầu trong hệ thống. 15

4.1 Kiến trúc 4 tầng chuẩn hóa của hệ thống KBMS V3. 20
4.2 Luồng xử lý yêu cầu đi xuyên suốt 4 tầng kiến trúc. 21
4.3 Sơ đồ luồng bảo mật và chẩn đoán hệ thống song song. 21
4.4 Bản đồ giải phẫu thành phần tầng lưu trữ vật lý. 24

5.1 Sơ đồ lớp mô tả cấu trúc thực thể của KnowledgeBase trong KBMS V3. 27
5.2 Sơ đồ lớp đặc tả cấu trúc nội tại phức hợp của một Concept. 28
5.3 Sơ đồ minh họa quan hệ Parent-Child thông qua HierarchyType. 29
5.4 Sơ đồ lớp đặc tả Relation với Domain, Range và tri thức nội tại. 30
5.5 Sơ đồ lớp đặc tả cấu trúc Rule và cấu trúc đệ quy Expression. 31
5.6 Sơ đồ lớp đặc tả thành phần Function và Operator. 32
5.7 Sơ đồ lớp đặc tả ObjectInstance và cơ chế lưu trữ giá trị Dictionary. 32

6.1 Kết quả kiểm thử bộ phân tích cú pháp (Parser) với hơn 21,000 kịch bản. 46
6.2 Luồng Token được phân tách bởi Lexer trước khi đưa vào Parser. 46

7.1 Sơ đồ chiến lược luồng I/O đa tầng của KBMS. 48
7.2 Sơ đồ kiến trúc thành phần chi tiết của Storage Engine V3. 48
7.3 Giải phẫu cấu trúc trang nhị phân Slotted-Page của KBMS. 49
7.4 Sơ đồ cấu trúc đa tầng của cây B+ Tree trong việc quản lý thực thể. 51
7.5 Cơ chế tách nút (Split) tự động khi trang dữ liệu bị đầy. 52
7.6 Luồng dữ liệu qua các tầng xử lý từ Model đến Disk. 53
7.7 Cơ chế tái hiện (Redo) dữ liệu từ nhật ký WAL để khôi phục trạng thái nhất quán. 54
7.8 Minh họa vị trí thực tế của đối tượng TamGiac trong bộ nhớ nhị phân. 56

8.1 Sơ đồ điều phối giữa Engine suy diễn và các phân hệ ngoại vi. 57
8.2 So sánh chiến lược Suy diễn tiến (Forward) và Suy diễn lùi (Backward). 58
8.3 1. Sơ đồ Luồng FClosure 61
8.4 Kịch bản kiểm thử thuật toán F-Closure nâng cao. 63
8.5 Kết quả tìm tập đóng và vết suy diễn tương ứng. 63
8.6 Nhật ký log của Engine khi tìm nghiệm phương trình bậc hai 63

9.1 Vòng đời xử lý truy vấn xuyên suốt 4 tầng kiến trúc 66
9.2 Quy trình đóng gói và phân rã gói tin nhị phân 67
9.3 Kịch bản kiểm thử đóng gói và phân rã gói tin nhị phân. 69
9.4 Nhật ký gói tin nhị phân (Little-endian) truyền trên Socket. 69
9.5 Bằng chứng Server xử lý đồng thời nhiều Client mà không nghẽn. 70

10.1 Mô hình xử lý bất đồng bộ (Asynchronous) sử dụng ThreadPool của .NET. 71
10.2 Sơ đồ tương tác giữa Knowledge Manager và các phân hệ cấp thấp. 73
10.3 Luồng khởi động Server và đăng ký các Manager 74
10.4 Nhật ký khởi động máy chủ thành công với đầy đủ các Manager. 75
10.5 Kịch bản kiểm thử tích hợp (End-to-End) thông qua TCP Sockets. 76
10.6 Kết quả thực thi 111 kịch bản kiểm thử tích hợp toàn diện. 76
10.7 Minh chứng mã nguồn kiểm thử bảo mật RBAC. 76
10.8 Kết quả từ chối truy cập (Access Denied) đối với người dùng không có quyền. 76

11.1 Luồng Token được phân tách bởi Lexer trước khi đưa vào Parser 77
11.2 Cấu trúc cây phân tích cú pháp (AST) của KBQL 79
11.3 Tổng quan kiến trúc nút AST của ngôn ngữ KBQL 80
11.4 Sơ đồ chi tiết nút AST cho ngôn ngữ KDL 81
11.5 Sơ đồ chi tiết nút AST cho ngôn ngữ KQL và KML 82
11.6 Sơ đồ chi tiết nút AST cho ngôn ngữ KCL và TCL 83
11.7 Minh chứng mã nguồn kiểm thử độ bao phủ của bộ phân tích (Parser). 84
11.8 Kết quả vượt qua toàn bộ 21,000+ kịch bản kiểm thử cú pháp. 85
11.9 Chứng minh Parser xử lý thành công các biểu thức lồng nhau phức tạp. 85

12.1 Sơ đồ luồng xử lý đầu vào trong giao diện CLI 87
12.2 Kịch bản thực thi hàng loạt qua lệnh SOURCE trong CLI 88
12.3 Chế độ hiển thị dọc (Vertical Mode) trong CLI 89
12.4 Minh chứng mã nguồn kiểm thử giao diện dòng lệnh (CLI). 90
12.5 Giao diện tương tác CLI thực thi câu lệnh truy vấn tri thức. 90
12.6 Hiển thị kết quả truy vấn dưới dạng bảng (Table View) trong CLI. 90

13.1 Luồng xử lý một yêu cầu tri thức xuyên suốt 4 tầng từ Studio UI. 92
13.2 Cơ chế Server Push cho các thông báo hệ thống thời gian thực. 93
13.3 Luồng quy trình thiết kế và kiểm tra tri thức trong Studio. 94
13.4 Quy trình các bước từ gửi yêu cầu đến truy vết suy diễn. 94
13.5 Các bước thao tác nạp dữ liệu lớn và bảo trì B+ Tree. 95
13.6 Quy trình phát hiện và ngắt kết nối các phiên làm việc bất thường. 96
13.7 Luồng phân tích lỗi hệ thống thông qua Log Analyzer. 97
13.8 Minh chứng mã nguồn API giám sát (Dashboard API). 98
13.9 Giao diện soạn thảo tri thức trực quan trong KBMS Studio. 98
13.10 Chứng minh luồng dữ liệu Studio và Electron Main truyền nhận gói tin nhị phân. 98

14.1 Giao diện cài đặt thành công KBMS trên Linux 100
14.2 Tổng kết kết quả 111 kịch bản kiểm thử (100% Passed). 101
14.3 Kết quả đo đạc hiệu năng với cột Duration chi tiết 102
14.4 Vết suy diễn của thuật toán F-Closure trên dữ liệu kiểm thử 103

DANH MỤC BẢNG

3.1 So sánh Tính năng Chi tiết giữa KBMS và các hệ thống khác . . . 13
3.2 Đặc tả Yêu cầu Chức năng Hệ thống Master . . . 16

4.1 Mô tả chức năng 4 tầng kiến trúc KBMS . . . 21
4.2 Đặc tả yêu cầu phi chức năng hệ thống KBMS . . . 25

6.1 Phân loại Từ khóa dành riêng (Reserved Keywords) . . . 34
6.2 Khả năng hỗ trợ lệnh ALTER cho từng đối tượng KBQL . . . 35
6.3 Phân loại Kiểu dữ liệu nguyên thuỷ trong KBQL . . . 35
6.4 Danh mục Hàm tích hợp (Built-in Functions) trong KBQL . . . 43
6.5 Phân bổ kịch bản kiểm thử theo nhóm ngôn ngữ . . . 46

7.1 Thông số định lượng của Storage Engine V3 . . . 49
7.2 Bảng 11 . . . 50
7.3 Bảng 12 . . . 52
7.4 Bảng 13 . . . 55
7.5 Bảng 14 . . . 56

8.1 Bảng ký hiệu thuật toán F-Closure . . . 62
8.2 Tổng hợp kết quả kiểm thử bộ máy suy diễn . . . 63

9.1 Đặc tả cấu trúc gói tin nhị phân KBMS . . . 64

10.1 Tổng hợp kết quả kiểm thử hệ thống máy chủ . . . 76

11.1 Phân loại các nhóm Token trong Lexer . . . 78

12.1 Danh mục lệnh điều khiển CLI . . . 86
12.2 Kiểm thử định dạng kết quả truy vấn trong CLI . . . 90

14.1 Yêu cầu cấu hình hệ thống tối thiểu . . . 100
14.2 Danh sách kịch bản kiểm thử tích hợp toàn diện . . . 101
14.3 Hiệu năng theo Tầng Kiến trúc . . . 101
14.4 Hiệu năng Truy vấn Khối lượng trên tập dữ liệu mô phỏng . . . 101
14.5 Danh mục các Bộ Dữ liệu Kiểm thử chuẩn . . . 102
14.6 Dữ liệu nguồn bộ kiểm thử Charlie . . . 102

CHƯƠNG 1: Giới thiệu và Đặt vấn đề

1.1 Giới thiệu đề tài và Đặt vấn đề

1.1.1 Lý do chọn đề tài (Motivation)

Trong kỷ nguyên chuyển đổi số, nhu cầu về việc quản trị và khai thác tri thức đã trở nên cấp thiết hơn bao giờ hết. Các hệ quản trị cơ sở dữ liệu truyền thống (RDBMS) mặc dù rất mạnh mẽ trong việc lưu trữ và truy xuất dữ liệu có cấu trúc, nhưng vẫn còn nhiều hạn chế trong việc tự động suy diễn ra các tri thức mới từ tập dữ liệu hiện có.

Việc tích hợp trí tuệ nhân tạo, cụ thể là các hệ thống dựa trên tri thức (Knowledge-based systems), vào tầng lưu trữ dữ liệu giúp rút ngắn khoảng cách giữa ”Dữ liệu thô”và ”Tri thức hữu ích”[1]. Đề tài ”Xây dựng Hệ quản trị Cơ sở tri thức (KBMS) dựa trên mô hình Đối tượng tính toán”được lựa chọn nhằm giải quyết bài toán này.

1.1.2 Mục tiêu nghiên cứu (Research Objectives)

Mục tiêu chính của đề tài là xây dựng một hệ thống KBMS hoàn chỉnh, có khả năng:

1. **Biểu diễn tri thức chuyên sâu**: Sử dụng mô hình COKB [1] để định nghĩa các thực thể có khả năng tính toán.
2. **Suy diễn tự động hiệu năng cao**: Áp dụng thuật toán F-Closure để tìm tập đóng tri thức một cách tối ưu.
3. **Lưu trữ bền vững và an toàn**: Phát triển công cụ lưu trữ nhị phân hỗ trợ chỉ mục B+ Tree và nhật ký WAL [5].
4. **Giao diện phát triển trực quan**: Cung cấp Studio IDE giúp người dùng dễ dàng thiết kế và kiểm chứng tri thức.

1.1.3 Đối tượng và Phạm vi nghiên cứu

- **Đối tượng**: Các mô hình biểu diễn tri thức, thuật toán suy diễn tiến (Forward Chaining) và các kỹ thuật quản lý CSDL.
- **Phạm vi**:
  - **Quản lý tri thức**: Thực hiện các thao tác thêm, sửa, xóa và tìm kiếm tri thức chuyên sâu qua ngôn ngữ KBQL.
  - **Suy diễn tri thức**: Xây dựng bộ máy suy duyễn dựa trên tập luật và sự thật hiện có để sinh ra tri thức mới.
  - **Lưu trữ**: Nghiên cứu phương pháp lưu trữ tri thức bền vững dưới cấu trúc mô hình đối tượng tính toán (COKB).

1.1.4 Ý nghĩa khoa học và thực tiễn

Đề tài góp phần chuẩn hóa phương pháp xây dựng hệ thống tri thức có khả năng mở rộng (Scale-out) và tính linh hoạt cao trong việc thay đổi luật mà không cần can thiệp vào mã nguồn ứng dụng. Đây là nền tảng quan trọng cho việc phát triển các hệ chuyên gia và hệ trợ giúp quyết định thông minh.

CHƯƠNG 2: Cơ sở Lý thuyết COKB

2.1 Mô hình Đối tượng Tính toán COKB

Mô hình **COKB (Computational Objects Knowledge Base)** [1] là sự mở rộng của các hệ thống logic truyền thống, tích hợp khả năng tính toán mạnh mẽ vào các cấu trúc đối tượng, phục vụ như là định dạng lưu trữ và biểu diễn bộ não cốt lõi của hệ thống KBMS.

2.1.1 Thành phần Hệ thống tri thức COKB

Một cơ sở tri thức COKB được xác định bởi bộ 6 thành phần [1, 3]:

$$COKB = (C, H, R, Ops, Funcs, Rules) \tag{2.1}$$

Trong đó:

- **C (Concepts)**: Tập hợp các khái niệm hoặc lớp đối tượng tính toán.
- **H (Hierarchy)**: Các quan hệ phân cấp đặc biệt hóa giữa các khái niệm (quan hệ IS-A).
- **R (Relations)**: Tập các quan hệ ngữ nghĩa giữa các lớp đối tượng (ví dụ: song song, vuông góc).
- **Ops (Operators)**: Các toán tử tính toán trên các miền giá trị (Số thực, Vector, Ma trận).
- **Funcs (Functions)**: Các hàm xác định ánh xạ giữa các thuộc tính.
- **Rules (Rules)**: Tập hợp các luật dẫn để suy diễn ra tri thức mới.

2.1.2 Mô hình Đối tượng Tính toán (Computational Object)

Mỗi thực thể (Object) trong hệ thống được biểu diễn bởi bộ ba thành phần [2]:

$$O = (Attrs, Facts, Rules) \tag{2.2}$$

1. **Attrs (Attributes)**: Tập các thuộc tính của đối tượng. Mỗi thuộc tính bản thân nó cũng có thể là một đối tượng tính toán khác (cấu trúc đệ quy) [4].
2. **Facts (Facts)**: Các giá trị hoặc tính chất vốn có của đối tượng đã được xác định.
3. **Rules (Internal Rules)**: Các quy tắc, phương trình nội tại ràng buộc mối quan hệ giữa các Attrs bên trong đối tượng đó.

2.1.3 Phân cấp Khái niệm (Concept Levels)

Trong mô hình COKB, các khái niệm được phân tầng dựa theo độ phức tạp của cấu trúc attributes:

- **Cấp 0 (Base Concepts)**: Các kiểu dữ liệu cơ sở (Số thực - , Điểm - Point).
- **Cấp 1**: Các khái niệm xây dựng trực tiếp từ cấp 0 (Đoạn thẳng, Góc).
- **Cấp n**: Các khái niệm phức tạp hình thành từ các lớp thấp hơn (Tam giác, Tứ giác, Động cơ).

Việc phân cấp này giúp hệ thống quản lý tri thức theo hướng mô-đun hóa và hỗ trợ lan truyền kế thừa tri thức một cách tự động.

2.2 Cơ chế Suy luận và Giải quyết vấn đề

Dựa trên cấu trúc mô hình COKB, quá trình giải quyết bài toán thực chất là quá trình mở rộng tập sự thật dựa trên các quy tắc suy luận cưỡng bức, cho phép hệ thống ”hiểu”và tự động phát sinh nội dung mà không cần người dùng nhập thủ công.

2.2.1 Các Quy tắc Suy luận (Inference Rules)

Hệ thống KBMS vận hành dựa trên 6 loại quy tắc suy luận chính (RC1 - RC6) [2]:

- **RC1 (Vốn có)**: Sinh ra sự kiện từ thuộc tính mặc định của đối tượng (ví dụ: Tam giác cân có 2 góc đáy bằng nhau).
- **RC2 (Mặc nhiên)**: Các phép toán gán, thay thế và bắc cầu cơ bản giữa các biến và hằng số.
- **RC3 (Thay thế quan hệ)**: Sử dụng các quan hệ tính toán đã biết để tìm giá trị biến còn thiếu (Ohm's Law: $U, I \implies R$).
- **RC4 (Luật dẫn)**: Kích hoạt các luật dạng ‘IF <Hypothesis> THEN <Goal>‘.
- **RC5 (Giải hệ phương trình)**: Kết hợp nhiều biến và phương trình để giải đồng thời bằng phương pháp Newton-Raphson hoặc Brent [2].
- **RC6 (Hành vi nội bộ)**: Suy diễn dựa trên các đặc tính cấu thành (PART-OF) [2].

2.2.2 Thuật toán Tìm bao đóng (F-Closure Algorithm)

Đây là thuật toán trọng tâm của KBMS dùng để tìm tập đóng tri thức lớn nhất có thể suy ra được từ tập giả thiết **GT**.

Mô tả thuật thuật giải:

1. Khởi tạo tập ‘KnownFacts = GT‘.
2. Lặp lại cho đến khi không còn sự thật mới (Fixed-point):
   - Quét qua toàn bộ tập luật (Rules) và phương trình (Equations).
   - Nếu điều kiện của luật thỏa mãn trong ‘KnownFacts‘, đẩy kết luận vào tập đóng.
   - Cập nhật ‘KnownFacts‘.
     (a) Kết quả: ‘FClosure(GT)‘ là tập hợp chứa toàn bộ tri thức tường minh và dẫn xuất.

2.2.3 Chiến lược Giải bài toán: Heuristics

Để tăng tốc độ suy diễn trong các không gian tri thức khổng lồ, KBMS sử dụng các quy tắc heuristics dựa trên lớp bài toán [2]:

- **Heuristic 1**: Ưu tiên các luật có phần kết luận chứa biến mục tiêu (**Goal-directed**).
- **Heuristic 2**: Ưu tiên giải quyết các đối tượng có mức độ xác định (Confidence Level) cao nhất trước.
- **Heuristic 3**: Sử dụng đồ thị phụ thuộc biến để giới hạn phạm vi quét luật.

CHƯƠNG 3: Phân tích và Thiết kế Hệ thống

3.1 Khảo sát Hiện trạng & Mục tiêu (Survey & Objectives)

Dự án KBMS (Knowledge Base Management System) được xây dựng nhằm thu hẹp khoảng cách giữa các hệ quản trị CSDL truyền thống và các hệ thống chuyên gia dựa trên tri thức.

3.1.1 Khảo sát Hiện trạng (Current State Survey)

Dựa trên phân tích so sánh với các hệ thống hiện có, chúng tôi nhận thấy các giới hạn sau:

- **CSDL Quan hệ (RDBMS):** Rất mạnh về lưu trữ đĩa (Persistence) và giao dịch (ACID) nhưng hoàn toàn thiếu khả năng suy diễn logic tự động [5]. Việc xử lý tri thức phải thực hiện ở tầng ứng dụng, gây khó khăn cho việc bảo trì luật.
- **Hệ thống Logic (Prolog):** Có khả năng suy diễn cực mạnh nhưng chủ yếu hoạt động trên bộ nhớ RAM, thiếu cơ chế quản lý dữ liệu nhị phân quy mô lớn trên đĩa cứng và khả năng quản lý đa người dùng thực thụ [8].
- **Công cụ Ontologies (Protégé):** Hỗ trợ mô hình hóa tri thức tốt nhưng hiệu năng truy vấn và khả năng tích hợp vào các hệ thống phần mềm thương mại còn hạn chế [7].

Bảng so sánh Tính năng Chi tiết

Đặc tính	CSDL Quan hệ [5]	Ngôn ngữ Logic [8]	Công cụ Tri thức [7]	**KBMS [1]**
Kiểu dữ liệu	Structured Tables	Logic Predicates	OWL / RDF	**Knowledge Concepts**
Suy diễn	Không (Chỉ Join)	Rất mạnh (Backward)	Reasoner Plugin	**Mạnh (Forward)**
Lưu trữ đĩa	B+ Tree, ACID	Chủ yếu RAM	Files (OWL/XML)	**B+ Tree nhị phân, WAL**
Đa người dùng	Rất tốt	Không	Hạn chế	**Tốt (Socket/Auth)**
Giao diện	Workbench / CLI	CLI đơn giản	Desktop phức tạp	**Studio IDE + CLI**

Bảng 3.1: So sánh Tính năng Chi tiết giữa KBMS và các hệ thống khác

Ưu thế vượt trội của KBMS

1. **Sự kết hợp giữa Persistence và Reasoning**: KBMS tích hợp **Inference Engine** trực tiếp trên tầng **Physical Storage**, cho phép suy diễn trên hàng triệu thực thể được lưu trữ bền vững.
2. **Giao diện Phát triển (Studio IDE)**: Cung cấp môi trường trực quan hóa đồ thị tri thức và hỗ trợ IntelliSense, giúp rút ngắn thời gian thiết kế bài toán.
3. **Hiệu năng Truyền tin (Network Layer)**: Sử dụng giao thức nhị phân (Binary Protocol) giúp đạt tốc độ xử lý cao với độ trễ tối thiểu.

Kết luận: Cần có một hệ thống kết hợp được cả **Hiệu năng lưu trữ (Indexing/WAL)** và **Khả năng suy diễn (Inference Engine)**.

3.1.2 Mục tiêu Nghiên cứu (Research Objectives)

Dự án KBMS hướng tới việc xây dựng một hệ quản trị tri thức toàn diện với các mục tiêu cụ thể:

1. **Storage Engine:** Phát triển cấu trúc cây B+ Tree nhị phân và cơ chế ghi nhật ký phục hồi (WAL) để quản lý hàng triệu thực thể tri thức bền vững.
2. **Reasoning Engine:** Xây dựng bộ máy suy diễn tiến (Forward Chaining) dựa trên điểm đóng F-Closure cho phép tự động tính toán dữ liệu bổ sung.
3. **Language Compiler:** Thiết kế ngôn ngữ KBQL (Knowledge Base Query Language) và bộ biên dịch (Parser/Lexer) hỗ trợ KDL (Định nghĩa) và KQL (Truy vấn).
4. **Integrated IDE:** Phát triển môi trường Studio IDE chuyên nghiệp giúp trực quan hóa và thiết kế tri thức dễ dàng.

3.2 Tác nhân & Vai trò Hệ thống (Actors & Roles)

Hệ thống KBMS được thiết kế để phục vụ các nhu cầu khác nhau từ học tập, nghiên cứu đến triển khai hệ thống công nghiệp.

3.2.1 Phân loại Người dùng

a. Học viên / Nghiên cứu sinh (Researchers)

- **Nhu cầu**: Thiết kế các mô hình tri thức lý thuyết (Hình học, Y học, Hóa học).
- **Công cụ**: Sử dụng **KBMS Studio** làm môi trường chính để vẽ đồ thị tri thức và định nghĩa tập luật.
- **Hành động**: CREATE CONCEPT, INSERT FACT, SOLVE bài toán.

b. Quản trị viên (System Administrators)

- **Nhu cầu**: Giám sát sức khỏe hệ thống và duy trì tính toàn vẹn của dữ liệu lớn.
- **Công cụ**: Sử dụng **System Dashboard** trong Studio và **CLI Management commands**.
- **Hành động**: REINDEX, CHECKPOINT, Quản lý Roles (GRANT/REVOKE), Giám sát RAM/Disk.

c. Nhà phát triển (Developers)

- **Nhu cầu**: Tích hợp KBMS làm lõi thông minh cho các ứng dụng Expert System khác.
- **Công cụ**: Sử dụng **KBMS CLI** và tương tác trực tiếp qua **Binary Protocol**.
- **Hành động**: Bulk Insert, Tự động hóa qua Script (.kbql), Xử lý luồng kết quả Streaming.

3.2.2 Mô hình Phân quyền (RBAC)

Hệ thống sử dụng cơ chế **Role-Based Access Control** để bảo mật tri thức:

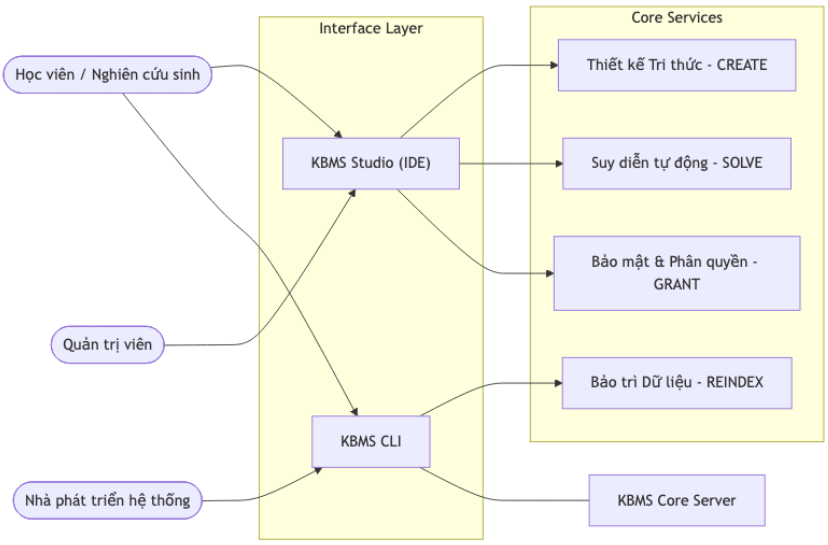
- **Admin**: Toàn quyền trên mọi Cơ sở tri thức (KB) và quản lý người dùng.
- **Researcher**: Quyền đọc/ghi/suy diễn trên các KB được cấp phép.
- **Guest**: Chỉ có quyền đọc (Read-only) trên các KB công khai.

3.3 Sơ đồ Hoạt động Tổng quát (Visual Overview)

Tài liệu này trình bày cái nhìn tổng quan về cách các thành phần tương tác và luồng dữ liệu xuyên suốt 4 tầng kiến trúc của KBMS.

3.3.1 Sơ đồ Use Case Tổng quát

Mô tả sự tương tác giữa các tác nhân bên ngoài và các dịch vụ lõi của hệ thống:

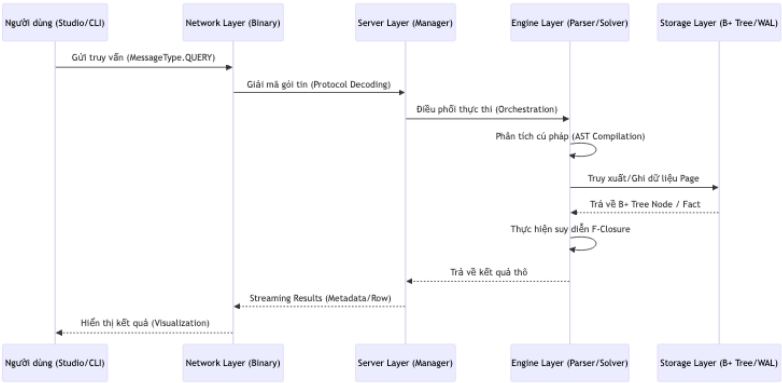


Hình 3.1: Sơ đồ Use Case tổng quát sự tương tác giữa người dùng và các dịch vụ lõi.

- **Interaction**: Người dùng tương tác thông qua hai môi trường (CLI/Studio).
- **Service Core**: Các yêu cầu được gửi tới Server để kích hoạt bộ suy diễn (RE), quản lý dữ liệu (KDL) hoặc bảo trì (KML).

3.3.2 Sơ đồ Sequence Hệ thống

Mô tả luồng ”sinh mệnh”của một yêu cầu từ khi xuất phát tại App Layer cho tới khi được lưu trữ bền vững tại Storage Layer:



Hình 3.2: Sơ đồ Sequence mô tả luồng sinh mệnh của một yêu cầu trong hệ thống.

Các giai đoạn chính:

1. **Request Stage**: App đóng gói yêu cầu nhị phân và gửi qua Network.
2. **Computing Stage**: Server Manager điều phối Parser để biên dịch câu lệnh và gửi tới Engine để tính toán/suy diễn.
3. **I/O Stage**: Engine tương tác với Storage Layer thông qua hệ thống Paging để đọc/ghi dữ liệu nhị phân.
4. **Response Stage**: Kết quả được ”Streaming”ngược lại cho khách hàng theo từng Row dữ liệu để tối ưu hóa bộ nhớ.

3.4 Đặc tả Yêu cầu Hệ thống Master (Master Requirements)

Tài liệu này là bản tổng hợp toàn diện nhất về mọi chức năng và yêu cầu kỹ thuật của KBMS, được đối soát trực tiếp với mã nguồn hệ thống (C# Server, Parser, Studio). Đây là ”nền tảng xương sống”cho việc thiết kế và phát triển toàn bộ hệ thống.

3.4.1 Yêu cầu Chức năng (Functional Requirements)

ID	Nhóm Chức năng	Mô tả Chi tiết	Mức ưu tiên
01	App Layer	KBMS Studio, CLI, REPL	High
02	Network Layer	Binary Protocol, Streaming	High
03	Server Layer	Reasoning Engine, Parser	Critical
04	Storage Layer	WAL, B+ Tree, Encryption	Critical

Bảng 3.2: Đặc tả Yêu cầu Chức năng Hệ thống Master

3.4.2 Yêu cầu Chức năng theo Tầng Kiến trúc (4 Tiers)

a. Tầng Ứng dụng (App Layer)

- **KBMS Studio (IDE Chuyên sâu)**:
  - **Monaco Engine**: Soạn thảo KBQL với Syntax Highlighting, IntelliSense và Squiggles báo lỗi thời gian thực.
  - **Knowledge Designer (VGE)**: Trực quan hóa đồ thị tri thức (Graph Nodes & Edges), thiết kế KDL kéo thả.
  - **Management Dashboard**: Giám sát RAM/CPU/Disk/Network, quản lý User, Session và Log Analyzer.
- **KBMS CLI (Interactive Tool)**:
  - **REPL & LineEditor**: Hỗ trợ phím tắt (‘Home/End‘, ‘Escx2‘), lịch sử lệnh và thụt đầu dòng tự động.
  - **Response Parser**: Vẽ bảng động, hỗ trợ Multi-line Cell và chế độ hiển thị dọc (‘\G‘).
  - **Batch Processing**: Lệnh ‘SOURCE‘ thực thi tệp tin kịch bản ‘.kbql‘.

b. Tầng Mạng (Network Layer - Binary Protocol)

- **Custom Binary Protocol**: Mã hóa nhị phân tối ưu (1 byte Type, 4 byte Length).
- **Message Types (12+ loại)**:
  - ‘1-LOGIN / 5-LOGOUT‘: Xác thực và kết thúc phiên.
  - ‘2-QUERY‘: Gửi lệnh thực thi.
  - ‘6-METADATA / 7-ROW / 8-FETCH_DONE‘: Luồng kết quả streaming.
  - ‘4-ERROR‘: Báo lỗi line-accurate (Line/Column).
  - ‘10-STATS / 11-LOGS_STREAM / 12-SESSIONS‘: Dữ liệu quản trị hệ thống.
  - ‘13-MANAGEMENT_CMD‘: Lệnh điều khiển Server từ xa.

c. Tầng Máy chủ (Server Layer - Core & Reasoning)

- **KnowledgeManager**: Điều phối luồng giữa Parser, Engine và Storage.
- **Reasoning Engine (Bộ máy Suy diễn)**:
  - **Thuật toán F-Closure**: Suy diễn tiến đến điểm đóng.
  - **Recursive Sub-closure**: Suy diễn tổ hợp các thành phần con.
  - **SameVariables Propagation**: Truyền thụ tính chất giữa các biến trùng tên.
- **Parser & Compiler**:

  - **Lexer**: Nhận diện 190+ từ khóa.
  - **Parser**: 50+ loại AST Nodes hỗ trợ KDL, KQL, KML, TCL, KCL.
- **Security & Auth**: Xác thực Base64, Role-Based Access Control (ROOT, ADMIN, RESEARCHER).
- **System Services**: ‘Bootstrapper‘, ‘SystemUpdater‘, ‘V2ToV3Converter‘.

d. Tầng Lưu trữ (Storage Layer - Physical Engine)

- **Physical Paging**: Trang dữ liệu 8KB cố định.
- **Indexing**: B+ Tree tối ưu tìm kiếm $O(\log n)$.
- **WAL (Write-Ahead Logging)**: Nhật ký phục hồi dữ liệu đảm bảo ACID.
- **Encryption**: Mã hóa tĩnh AES-256 đối với tệp ‘.dat‘ và ‘.kbf‘.

3.4.3 Bản đồ Chức năng KBQL (Detailed Registry)

Dưới đây là danh sách đầy đủ các câu lệnh hệ thống hỗ trợ:

Quản lý Kiểm soát (KCL - Security)

- ‘CREATE/ALTER/DROP USER‘: Quản lý tài khoản người dùng hệ thống.
- ‘GRANT/REVOKE‘: Phân quyền truy cập theo từng Cơ sở tri thức (KB).

Định nghĩa Tri thức (KDL - Design)

- ‘CREATE/DROP KNOWLEDGE BASE‘: Quản lý thực thể KB.
- ‘CREATE/ALTER/DROP CONCEPT‘: Quản lý các lớp đối tượng và thuộc tính.
- ‘CREATE/DROP RULE & EQUATION‘: Định nghĩa luật logic và phương trình toán học.
- ‘CREATE FUNCTION/OPERATOR‘: Mở rộng khả năng tính toán.
- ‘ADD/REMOVE COMPUTATION & HIERARCHY‘: Thiết lập quan hệ tính toán và kế thừa.
- ‘CREATE INDEX‘: Tối ưu hóa hiệu năng truy vấn.

Truy cập & Suy diễn (KQL/RE - Solve)

- ‘SELECT / INSERT / UPDATE / DELETE‘: Thao tác dữ liệu tri thức.
- ‘SOLVE‘: Kích hoạt bộ máy suy diễn tìm lời giải cho mục tiêu.
- ‘EXPLAIN / DESCRIBE / SHOW‘: Giải thích luồng suy diễn và xem cấu trúc tri thức.

Bảo trì & Hệ thống (KML - Maintenance)

- ‘IMPORT / EXPORT / BULK INSERT‘: Di trú dữ liệu quy mô lớn.
- ‘REINDEX / CHECKPOINT / VACUUM‘: Tối ưu hóa và dọn dẹp lưu trữ vật lý.
- ‘MIGRATION V2‘: Nâng cấp dữ liệu từ phiên bản cũ.

Quản lý Giao dịch (TCL - Transaction)

- ‘BEGIN / COMMIT / ROLLBACK‘: Đảm bảo tính nguyên tố (Atomicity) cho các chuỗi lệnh.

3.4.4 Yêu cầu Phi chức năng (Non-functional)

- **ACID Compliance**: Đảm bảo an toàn dữ liệu 100% thông qua WAL.
- **High Performance**: Độ trễ < 10ms trên LAN, hỗ trợ Asynchronous I/O.
- **Security**: Mã hóa AES-256, Master Key Security.
- **Scalability**: Stateless socket handling, hỗ trợ hàng trăm kết nối đồng thời.

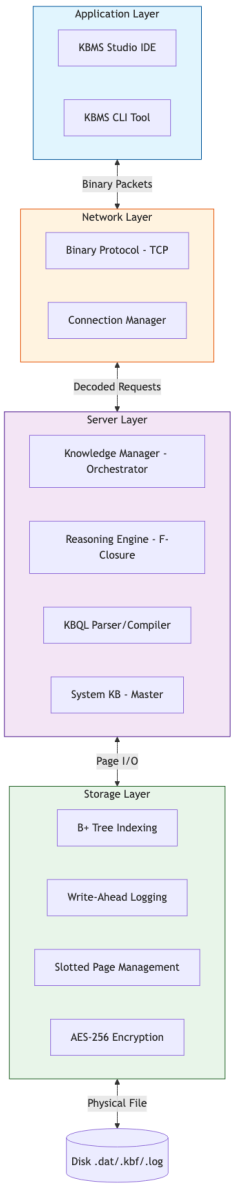
CHƯƠNG 4: Kiến trúc Hệ thống và các Tầng xử lý

4.1 Tổng quan Kiến trúc (System Overview)

Hệ thống KBMS (phiên bản V3) được thiết kế dựa trên mô hình kiến trúc 4 tầng (4-Tier Architecture) hiện đại [1]. Cấu trúc này giúp tách biệt rõ ràng trách nhiệm giữa việc tương tác người dùng, truyền dẫn mạng, xử lý logic và lưu trữ vật lý [3].

4.1.1 Mô hình Kiến trúc 4 Tầng

Cấu trúc phân tầng của KBMS bao gồm:

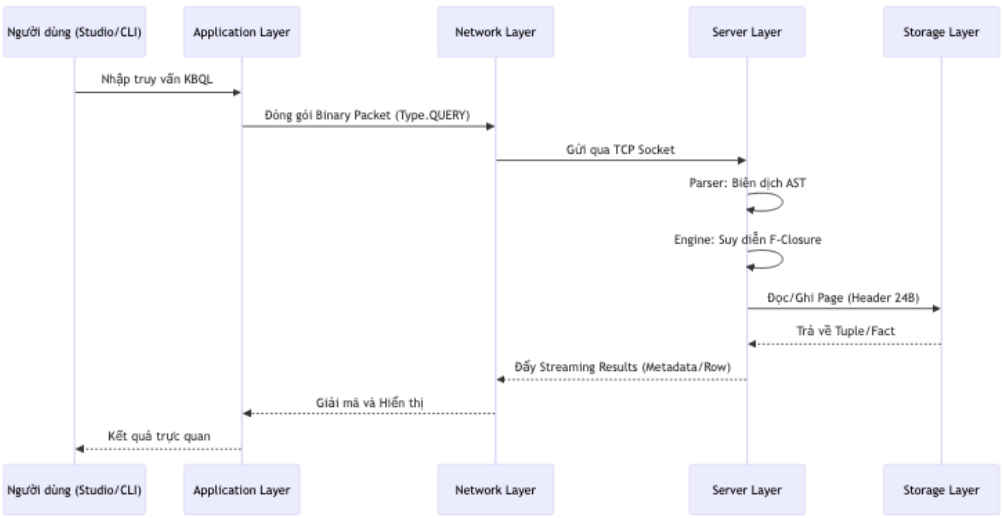


Hình 4.1: Kiến trúc 4 tầng chuẩn hóa của hệ thống KBMS V3.

1. **Application Layer**: Giao diện người dùng (Studio/CLI), chịu trách nhiệm thu thập yêu cầu và hiển thị kết quả.
2. **Network Layer**: Giao thức truyền dẫn nhị phân, đảm bảo kết nối ổn định và hiệu năng cao.
3. **Server Layer**: ”Bộ não”của hệ thống, thực hiện biên dịch ngôn ngữ KBQL và suy diễn tri thức.
4. **Storage Layer**: Quản lý dữ liệu bền vững trên đĩa cứng thông qua các thuật toán B+ Tree và WAL.

4.1.2 Luồng Yêu cầu Tổng quát

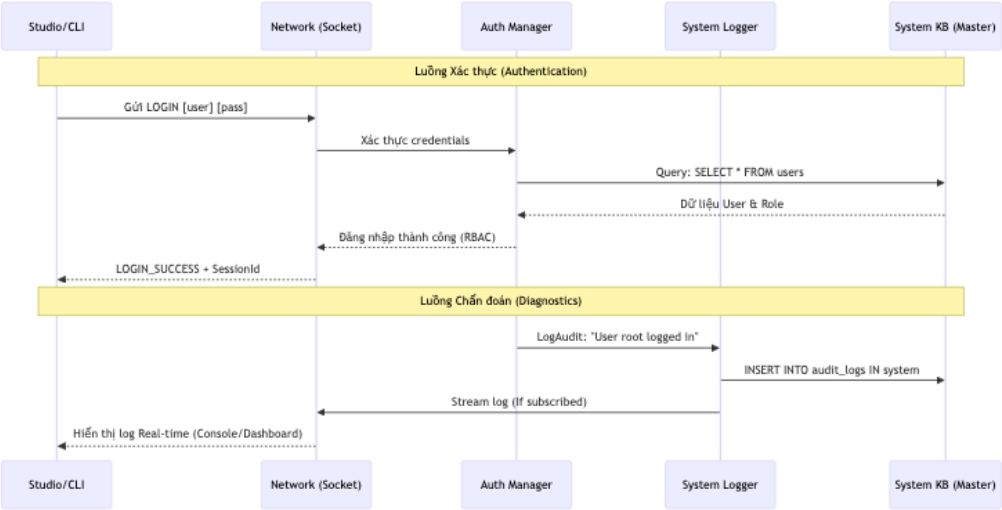
Mọi yêu cầu từ người dùng đều đi xuyên suốt qua 4 tầng này để đạt được kết quả cuối cùng:



Hình 4.2: Luồng xử lý yêu cầu đi xuyên suốt 4 tầng kiến trúc.

4.1.3 Bảo mật & Chẩn đoán Hệ thống

Bên cạnh luồng dữ liệu chính, KBMS duy trì một ”mạch quản trị”song song để đảm bảo tính an toàn và minh bạch:



Hình 4.3: Sơ đồ luồng bảo mật và chẩn đoán hệ thống song song.

4.1.4 Công nghệ Sử dụng (Technology Stack)

Tầng	Thành phần chính	Công nghệ & Thư viện
Application	Studio IDE	React, Electron, Monaco Editor, Tailwind CSS
Network	Binary Protocol	TCP Sockets, UTF-8 Encoding, Little-Endian
Server	Core Engine	.NET Core 8, C#, F-Closure Algorithm
Storage	Physical Layer	B+ Tree, WAL, Slotted Page, AES-256 Encryption

Bảng 4.1: Mô tả chức năng 4 tầng kiến trúc KBMS

4.2 Tầng Ứng dụng (Application Layer)

Tầng Ứng dụng là điểm chạm đầu tiên giữa người dùng và hệ thống KBMS. Nó bao gồm hai thành phần chính phục vụ các mục đích khác nhau.

4.2.1 KBMS Studio (IDE Chuyên nghiệp)

KBMS Studio được xây dựng trên nền tảng React và Electron, đóng vai trò là một môi trường phát triển tri thức toàn diện (Knowledge IDE).

- **Chức năng chính**:
  - **Monaco Engine**: Soạn thảo mã nguồn KBQL với tính năng IntelliSense và báo lỗi trực tiếp.
  - **Visual Knowledge Designer**: Cho phép thiết kế mô hình tri thức bằng đồ thị kéo thả.
  - **Management Dashboard**: Giám sát hiệu năng Server, quản lý User và xem Audit Logs.
- **Công nghệ**: React, TypeScript, Monaco Editor, Tailwind CSS.

4.2.2 KBMS CLI (Command Line Interface)

KBMS CLI là công cụ dòng lệnh mạnh mẽ dành cho các nhà quản trị và nhà phát triển hệ thống.

- **Chức năng chính**:
  - **REPL (Read-Eval-Print Loop)**: Môi trường tương tác dòng lệnh trực tiếp với Server.
  - **Batch Processing**: Thực thi các kịch bản tri thức (.kbql) quy mô lớn thông qua lệnh ‘SOURCE‘.
  - **Advanced Formatting**: Hỗ trợ hiển thị kết quả dưới dạng bảng hoặc hàng dọc (‘\G‘).
- **Công nghệ**: .NET Core, C#, ‘LineEditor‘ custom library.

4.3 Tầng Mạng (Network Layer)

Tầng Mạng đóng vai trò là ”mạch máu”truyền dẫn thông tin giữa các thành phần Application Layer và Server Layer.

4.3.1 Giao thức Nhị phân (Custom Binary Protocol)

Để tối ưu hóa hiệu năng và giảm thiểu băng thông, KBMS không sử dụng JSON/XML để giao tiếp mà sử dụng một giao thức nhị phân tùy chỉnh trên nền TCP.

- **Cấu trúc Gói tin**: Gồm 1 byte ‘MessageType‘ và 4 byte ‘PayloadLength‘, sau đó là ‘Payload‘ dữ liệu thực tế.
- **12+ Message Types**:
  - ‘LOGIN‘: Xác thực người dùng.
  - ‘QUERY‘: Gửi yêu cầu KBQL.
  - ‘LOGS_STREAM‘: Yêu cầu streaming log thời gian thực.
  - ‘MANAGEMENT_CMD‘: Lệnh quản trị hệ thống (Dành cho ROOT).
- **Encoding**: Sử dụng chuẩn UTF-8 cho chuỗi văn bản và định dạng nhị phân Little-Endian cho các số nguyên/thực.

4.3.2 Quản lý Kết nối (Connection Manager)

- **Asynchronous Socket I/O**: Server sử dụng mô hình lập trình bất đối xứng để phục vụ hàng trăm kết nối đồng thời với độ trễ tối thiểu.
- **Session Isolation**: Mỗi kết nối được gán một ‘SessionId‘ duy nhất, giúp tách biệt dữ liệu tạm thời và ngữ cảnh làm việc (Context) giữa các người dùng.
- **Heartbeat & Timeout**: Đảm bảo giải phóng tài nguyên hệ thống (RAM/Socket) ngay khi kết nối bị ngắt đột ngột sau 30 giây.

4.4 Tầng Máy chủ (Server Layer)

Tầng Máy chủ (Server Layer) là thành phần phức tạp nhất của KBMS, chịu trách nhiệm cho toàn bộ quá trình biên dịch ngôn ngữ và suy diễn tự động.

4.4.1 Knowledge Manager (Orchestrator)

Knowledge Manager là bộ điều phối trung tâm của Server. Nó chịu trách nhiệm:

- **KB Isolation**: Sử dụng ‘StoragePool‘ để đảm bảo mỗi người dùng có thể làm việc trên một Cơ sở tri thức độc lập mà không gây xung đột tài nguyên.
- **Thread-safe Operations**: Quản lý các lệnh nạp/hủy KB thông qua cơ chế khóa (Mutual Exclusion) để duy trì tính toàn vẹn của danh lục hệ thống.
- **Execution Routing**: Điều phối luồng làm việc giữa Parser (Biên dịch) và Inference Engine (Suy diễn).

4.4.2 KBQL Parser & Compiler

- **Phân tích Cú pháp**: Sử dụng thuật toán đệ quy xuống (Recursive Descent) để biên dịch câu lệnh KBQL thành cấu trúc AST (Abstract Syntax Tree).
- **Hỗ trợ**: Biên dịch các tập lệnh KDL (Định nghĩa), KQL (Truy vấn), KCL (Bảo mật), KML (Bảo trì) và TCL (Giao dịch).

4.4.3 Reasoning Engine (Bộ máy Suy diễn)

- **Forward Chaining**: Thuật toán suy diễn tiến dựa trên điểm đóng **F-Closure**.
- **Mô hình COKB**: Thực hiện suy diễn trên các đối tượng tính toán phức tạp, giải quyết các bài toán về Quan hệ (IS-A, PART-OF) và Phương trình toán học.

4.4.4 Dịch vụ Bảo mật & Xác thực (Security & Auth)

Hệ thống bảo vệ tri thức thông qua cơ chế xác thực đa tầng và phân quyền dựa trên vai trò (RBAC):

- **Authentication**: Đối soát thông tin đăng nhập với ‘UserCatalog‘ lưu trong System KB. Mật khẩu được mã hóa an toàn.
- **Authorization (RBAC)**: Phân chia 3 cấp độ truy cập:
  - **ROOT**: Quyền cao nhất, quản lý toàn bộ hệ thống, user và cấu hình.
  - **ADMIN**: Quản lý các Cơ sở tri thức (KB) và Concept.
  - **RESEARCHER**: Quyền đọc và thực thi suy diễn (Read-only / Solve).

4.4.5 Hệ thống Nhật ký & Chẩn đoán (Native Logging)

KBMS V3 sở hữu cơ chế nhật ký độc đáo, biến log thành một dạng tri thức có thể truy vấn:

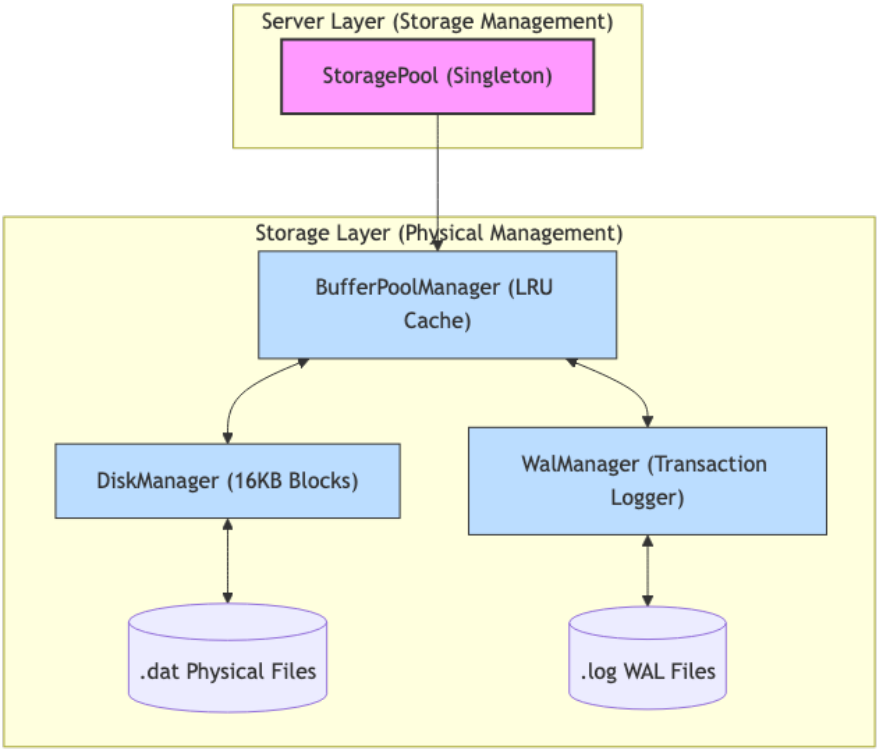
- **System Logs**: Lưu trữ các sự kiện vận hành, lỗi hệ thống và hiệu năng RAM/CPU.
- **Audit Logs**: Ghi lại mọi câu lệnh KBQL của người dùng, thời gian thực hiện và trạng thái thành công/thất bại.
- **Persistence**: Mọi log đều được ‘SystemLogger‘ chèn trực tiếp vào các concept ‘system_logs‘ và ‘audit_logs‘ bên trong **System KB**.
- **Real-time Streaming**: Hỗ trợ đẩy log trực tiếp tới Studio thông qua WebSockets/TCP phục vụ giám sát thời gian thực.

4.5 Tầng Lưu trữ (Storage Layer)

Tầng Lưu trữ chịu trách nhiệm đảm bảo tính bền vững (Durability), an toàn (Security) và điều phối truy cập đồng thời cho tri thức.

4.5.1 Ánh xạ Thành phần Hệ thống

Hệ thống quản lý việc lưu trữ qua một phân cấp các Manager chuyên biệt:



Hình 4.4: Bản đồ giải phẫu thành phần tầng lưu trữ vật lý.

- **StoragePool (Singleton)**: Quản lý danh sách các KBs đang nạp và đảm bảo Thread-safety khi truy cập đa luồng.
- **BufferPoolManager (LRU Cache)**: Điều phối bộ nhớ đệm RAM theo từng trang (Page).
- **DiskManager (Physical Handler)**: Giao tiếp trực tiếp với hệ điều hành để đọc/ghi tệp nạp theo khối (16KB).
- **WalManager (Transaction Logger)**: Quản lý tệp nhật ký '.log' để đảm bảo ACID.

4.5.2 Quản lý Trang (Physical Paging)

- **Slotted Page Layout**: Sử dụng cấu trúc trang 16KB với **24 Bytes Header** (PageId, LSN, Prev, Next, FreePtr, Count).
- **Hệ thống Slot**: Cho phép quản lý các bản ghi có độ dài thay đổi cực kỳ hiệu quả mà không làm phân mảnh đĩa.

4.5.3 Cấu trúc Chỉ mục (B+ Tree Indexing)

- **Hiệu năng**: Tìm kiếm và truy xuất bản ghi với độ phức tạp $O(\log n)$.
- **Clustered Index**: Tích hợp dữ liệu trực tiếp vào các nút lá (Leaf Nodes) của cây B+ Tree để tối thiểu hóa số lần đọc đĩa.

4.5.4 Điều phối Truy cập Đồng thời (Latching & Pinning)

Hệ thống hỗ trợ đa người dùng thao tác cùng lúc thông qua cơ chế Page Latching:

- **Page Pinning**: Khi một Query cần dữ liệu, Bpm sẽ cung cấp một Frame trong RAM và tăng 'PinCount'. Trang có 'PinCount > 0' sẽ bị khóa vĩnh viễn trong RAM (Pinned), thuật toán LRU không thể loại bỏ nó cho tới khi được 'Unpin'.
- **LRU Eviction**: Khi Cache đầy, các trang có 'PinCount = 0' và ít được sử dụng nhất sẽ bị trục xuất (Evicted) sau khi đã được đẩy (Flush) xuống đĩa nếu là trang bẩn (Dirty).
- **Write-Ahead Logging (WAL)**: Mọi thay đổi dữ liệu đều được ghi nhật ký trước khi thực hiện trên RAM, đảm bảo khả năng phục hồi (Recovery) bất chấp mọi sự cố nguồn điện.

4.5.5 Bảo mật Dữ liệu (Encryption)

- **AES-256**: Mã hóa toàn diện dữ liệu tĩnh (At-rest). Tệp dữ liệu thô trên đĩa hoàn toàn không thể đọc được nếu không có Master Key của hệ thống.

4.6 Đối soát Yêu cầu Kỹ thuật (Requirements Spec)

Tài liệu này tổng hợp toàn bộ các yêu cầu Chức năng (Functional) và Phi chức năng (Non-functional) dựa trên kiến trúc 4 tầng của KBMS.

4.6.1 Yêu cầu Chức năng (Functional Requirements)

Hệ thống cung cấp đầy đủ các nhóm chức năng từ mức ứng dụng đến mức lưu trữ:

Tầng Kiến trúc	Nhóm Chức năng	Hành động cụ thể
Application	Thiết kế & Giám sát	IDE Monaco, Kéo thả đồ thị, Dashboad thông số, REPL nâng cao.
Network	Kết nối & Truyền dẫn	Handshake nhị phân, Heartbeat, Giải mã Streaming Result Set.
Server	Xử lý & Suy diễn	Biên dịch KBQL, Suy diễn F-Closure, Giải phương trình, RBAC.
Storage	Lưu trữ & Phục hồi	B+ Tree Search, WAL Recovery, Checkpoint, AES Encryption.

Bảng 4.2: Đặc tả yêu cầu phi chức năng hệ thống KBMS

4.6.2 Yêu cầu Phi chức năng (Non-functional Requirements)

Đảm bảo hệ thống đạt được các tiêu chuẩn kỹ thuật chuyên nghiệp:

Độ bền vững & Nhất quán (ACID Compliance)

- **Atomicity**: Mọi thay đổi dữ liệu là nguyên tố thông qua nhật ký WAL.
- **Consistency**: Sau mọi lệnh KDL/KQL, tri thức phải ở trạng thái nhất quán về mặt logic.
- **Durability**: Dữ liệu đã cam kết (Commit) không bị mất mát sau sự cố.

Hiệu năng & Khả năng đáp ứng (Performance)

- **Độ trễ**: Thời gian phản hồi trung bình < 10ms trên mạng LAN.
- **Tải**: Hỗ trợ đồng thời 256+ kết nối mà không làm giảm hiệu suất suy diễn.
- **Truy xuất**: Tìm kiếm bản ghi $O(\log n)$ trên đĩa cứng quy mô TB.

Bảo mật (Security)

- **Xác thực**: Cơ chế LOGIN nhị phân với mật khẩu băm (Hashed).
- **Phân quyền**: RBAC đa tầng (Role-Based Access Control) đến từng Cơ sở tri thức.
- **Mã hóa**: AES-256 cho dữ liệu tĩnh trên đĩa cứng.

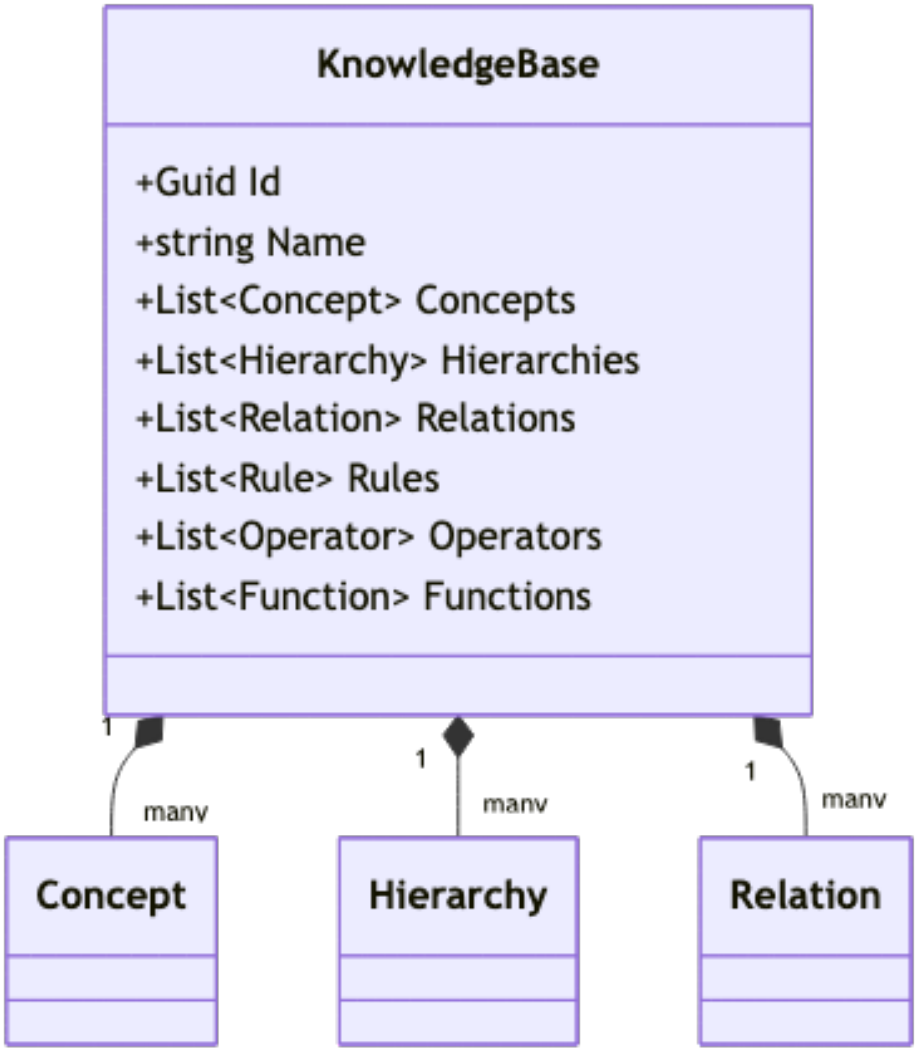
Khả năng Giám sát (Observability)

- **Logging**: Tự động ghi nhật ký mọi hành vi (Audit) và lỗi hệ thống (System Log).
- **Real-time**: Stream log từ Server về Studio với độ trễ < 1s.
- **Queryable Logs**: Cho phép dùng KBQL để truy vấn ngược lại lịch sử vận hành.

CHƯƠNG 5: Định nghĩa Mô hình và Khái niệm

5.1 Định nghĩa Mô hình tri thức dạng COKB

Mô hình **COKB (Computational Objects Knowledge Base)** [1] là sự kết hợp giữa lập trình hướng đối tượng và logic toán học, cho phép biểu diễn các đối tượng có khả năng tự tính toán và suy diễn. Một hệ quản trị cơ sở tri thức KBMS V3 được xây dựng dựa trên cốt lõi là bộ 6 thành phần hình thức:

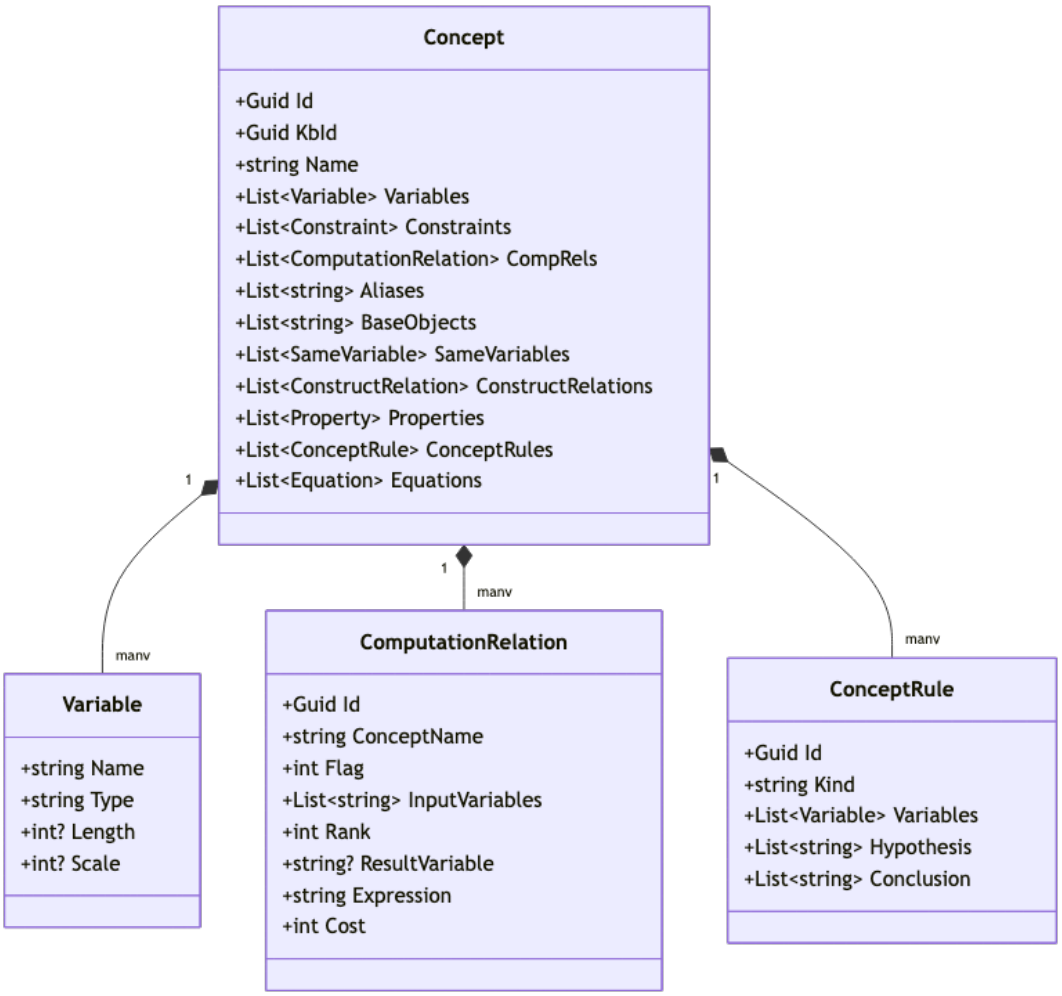


Hình 5.1: Sơ đồ lớp mô tả cấu trúc thực thể của KnowledgeBase trong KBMS V3.

$$COKB = (C, H, R, Ops, Funcs, Rules) \tag{5.1}$$

5.1.1 Thành phần C (Concepts - Tập các khái niệm)

Đây là thành phần quan trọng nhất, định nghĩa cấu trúc của các lớp đối tượng trong hệ thống. Mỗi khái niệm $c \in C$ được đặc tả bởi một bộ thành phần nội tại cực kỳ chi tiết:

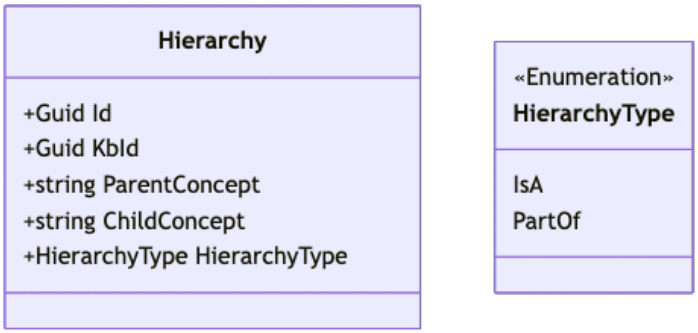


Hình 5.2: Sơ đồ lớp đặc tả cấu trúc nội tại phức hợp của một Concept.

1. **Variables (Biến)**: Tập các thuộc tính xác định đặc tính của đối tượng. Mỗi biến bao gồm ‘Name‘, ‘Type‘, ‘Length‘ và ‘Scale‘.
2. **Constraints (Ràng buộc)**: Các điều kiện logic (‘Expression‘) mà đối tượng phải thỏa mãn.
3. **Equations (Phương trình)**: Các công thức toán học xác định mối liên hệ định lượng giữa các biến.
4. **ComputationRelations (Quan hệ tính toán)**: Đặc tả khả năng tính toán của khái niệm thông qua ‘Rank‘, ‘Flag‘ và ‘Cost‘.
5. **ConceptRules (Luật nội tại)**: Các luật dẫn cục bộ (‘Hypothesis‘ -> ‘Conclusion‘) chỉ có giá trị trong phạm vị khái niệm.
6. **Cấu trúc mở rộng**: Bao gồm ‘Aliases‘ (Tên gọi khác), ‘BaseObjects‘ (Đối tượng cơ sở), ‘SameVariables‘ (Biến tương đương) và ‘ConstructRelations‘ (Quan hệ tạo lập).

5.1.2 Thành phần H (Hierarchy - Phân cấp tri thức)

Thành phần H quản lý các mối quan hệ cấu trúc giữa các Concepts thông qua thực thể ‘Hierarchy‘:

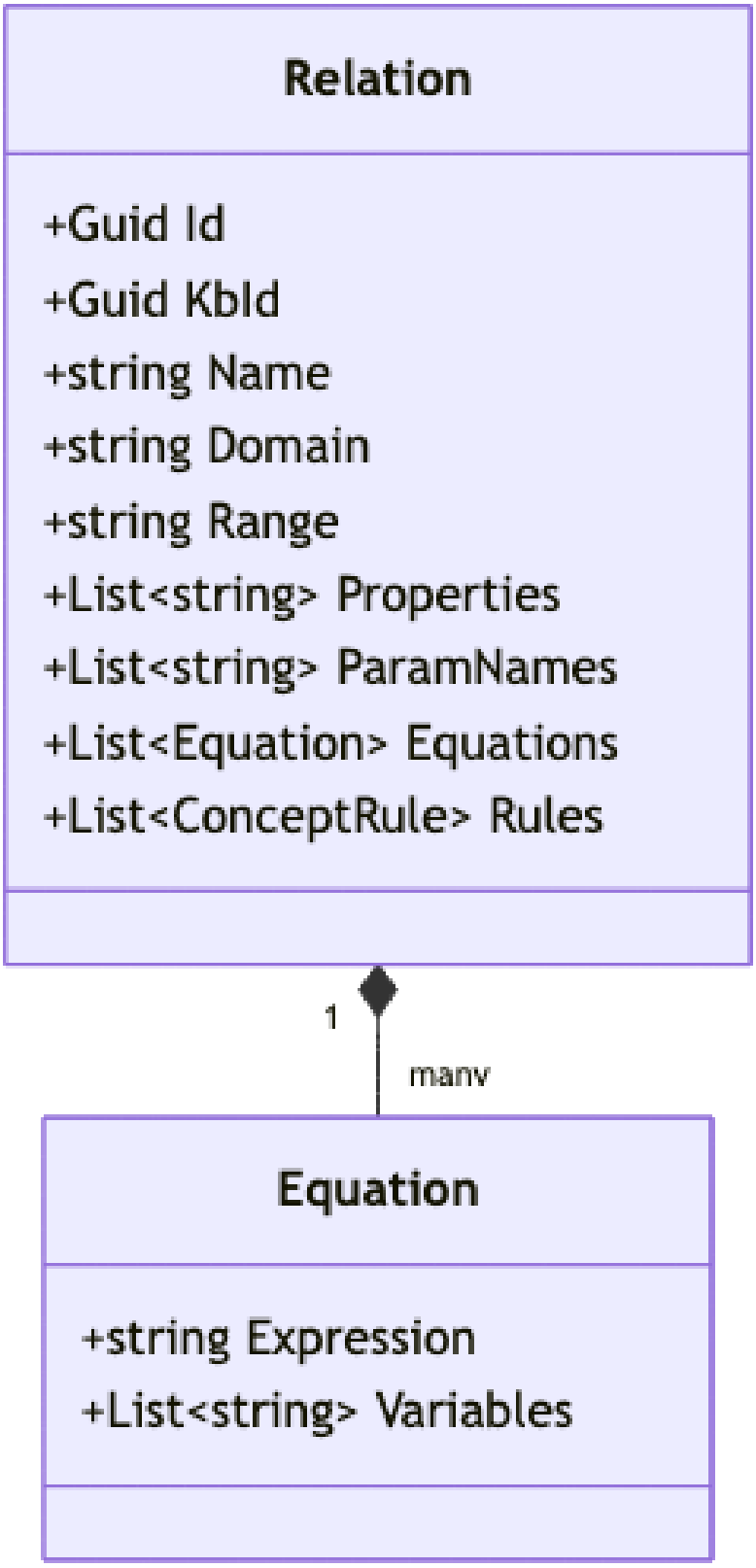


Hình 5.3: Sơ đồ minh họa quan hệ Parent-Child thông qua HierarchyType.

- **ParentConcept & ChildConcept**: Xác định điểm đầu và điểm cuối của liên kết phân cấp.
- **HierarchyType**: Bao gồm ‘IsA‘ (Kế thừa tri thức) và ‘PartOf‘ (Cấu trúc thành phần).

5.1.3 Thành phần R (Relations - Quan hệ ngữ nghĩa)

Quan hệ ngữ nghĩa R trong KBMS V3 không chỉ là các liên kết tĩnh mà còn mang tính toán học và logic:

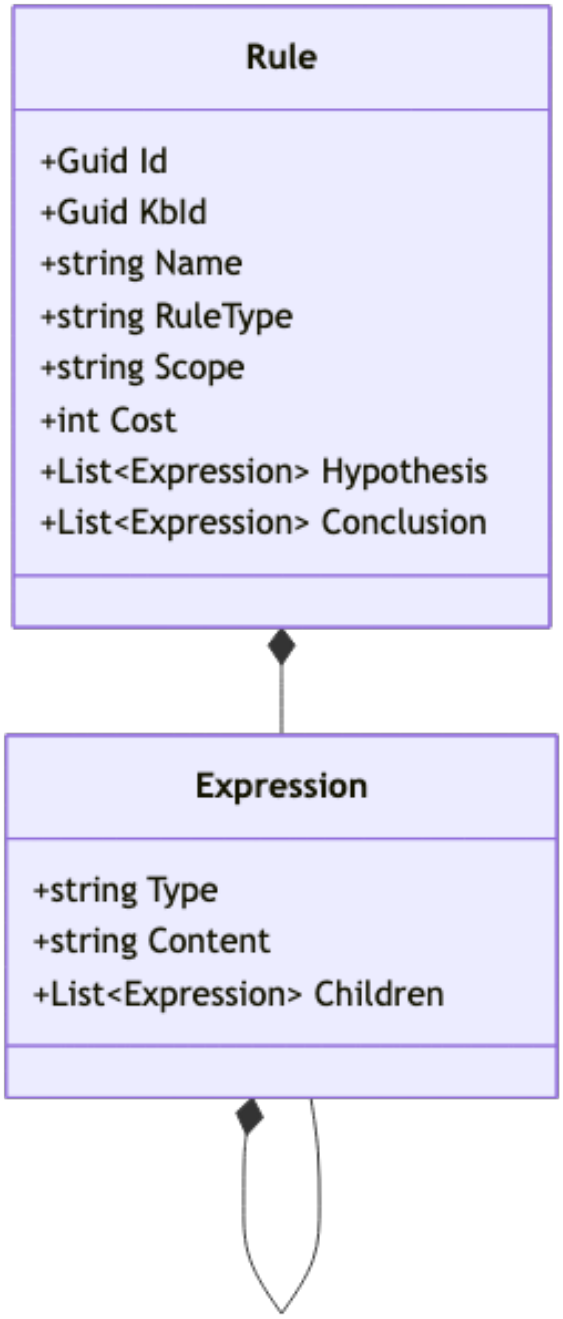


Hình 5.4: Sơ đồ lớp đặc tả Relation với Domain, Range và tri thức nội tại.

- **Domain & Range**: Xác định miền xác định và miền giá trị của quan hệ.
- **Properties**: Các tính chất của quan hệ như đối xứng, phản xạ, bắc cầu.
- **Knowledge Integration**: Mỗi quan hệ có thể chứa ‘Equations‘ và ‘Rules‘ riêng để hỗ trợ suy diễn.

5.1.4 Thành phần Rules & Logic (Bộ máy suy diễn)

Bộ máy suy diễn sử dụng các luật dẫn toàn cục để tìm bao đóng tri thức. Mỗi ‘Rule‘ được cấu trúc chặt chẽ:

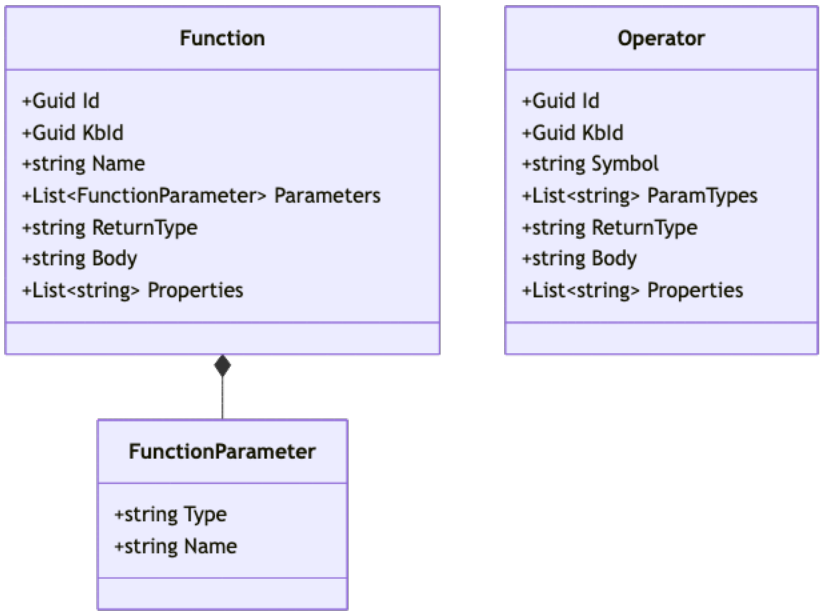


Hình 5.5: Sơ đồ lớp đặc tả cấu trúc Rule và cấu trúc đệ quy Expression.

- **RuleType**: Phân loại luật (Deduction, Default, Constraint, Computation).
- **Scope**: Phạm vi áp dụng của luật (tên Concept).
- **Hypothesis & Conclusion**: Tập hợp các ‘Expression‘ (Biểu thức).
- **Cấu trúc đệ quy Expression**: Mỗi biểu thức có thể chứa các biểu thức con (‘Children‘), cho phép biểu diễn các công thức logic và toán học có độ phức tạp bất kỳ.

5.1.5 Thành phần Ops & Funcs (Bộ máy thực thi)

Đây là các thành phần thực thi tính toán (Executable logic) của hệ thống:

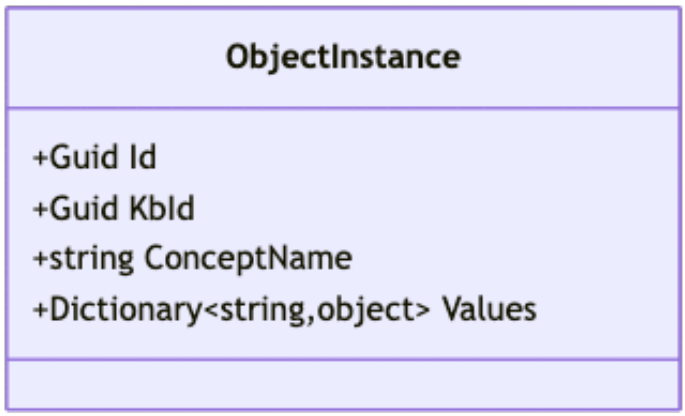


Hình 5.6: Sơ đồ lớp đặc tả thành phần Function và Operator.

- **Operators (Toán tử)**: Đặc tả qua 'Symbol', 'ParamTypes' và 'Body' thực thi.
- **Functions (Hàm)**: Gồm tập 'Parameters' (kiểu 'FunctionParameter'), 'ReturnType' và mã nguồn thực thi 'Body'.

5.1.6 Thực thể Đối tượng (ObjectInstance)

Thực thể (**ObjectInstance**) là các thể hiện cụ thể mang giá trị thực tế của một Concept:



Hình 5.7: Sơ đồ lớp đặc tả ObjectInstance và cơ chế lưu trữ giá trị Dictionary.

- **ConceptName**: Tên khái niệm mà thực thể thuộc về.
- **Values**: Một 'Dictionary<string, object>' lưu trữ tập hợp các cặp (Tên thuộc tính, Giá trị thực). Cơ chế này cho phép KBMS V3 quản lý dữ liệu đối tượng một cách linh hoạt và tối ưu bộ nhớ.

5.1.7 Ví dụ minh họa thực tế cho mô hình COKB

Để làm rõ các khái niệm lý thuyết trên, chúng ta xem xét một hệ tri thức về **Hình học phẳng (Geometry)** tập trung vào thực thể **Tam giác**.

Ví dụ về Concept (Khái niệm)

Định nghĩa khái niệm ‘TamGiac‘ trong KBMS:

- **Variables**: ‘a‘ (cạnh 1), ‘b‘ (cạnh 2), ‘c‘ (cạnh 3), ‘p‘ (nửa chu vi), ‘S‘ (diện tích).
- **Constraints**: $a + b > c, a + c > b, b + c > a$ (Điều kiện tồn tại tam giác).
- **Equations**: $p = (a + b + c)/2$ và $S = \sqrt{p(p - a)(p - b)(p - c)}$ (Công thức Heron).

Ví dụ về Hierarchy (Phân cấp)

- **IsA**: ‘TamGiacVuong‘ là một ‘TamGiac‘ (Kế thừa toàn bộ biến và phương trình của ‘TamGiac‘ nhưng thêm ràng buộc $a^2 + b^2 = c^2$).
- **PartOf**: ‘Dinh‘ (Đỉnh) là một thành phần thuộc về ‘TamGiac‘.

Ví dụ về Relation (Quan hệ)

- **Relation**: ‘DongDang(t1: TamGiac, t2: TamGiac)‘
- **Properties**: Đối xứng (Symmetric), Bắc cầu (Transitive).
- **Inference logic**: Nếu $t1.a/t2.a = t1.b/t2.b = t1.c/t2.c$ thì $t1$ đồng dạng $t2$.

Ví dụ về Rule & Logic (Luật dẫn)

- **Rule**: ‘R1: TamGiac(a==b, b==c) -> TamGiac{Loai=”Deu”}‘
- **Ý nghĩa**: Nếu ba cạnh bằng nhau thì kết luận đây là tam giác đều.

Ví dụ về Ops & Funcs (Thực thi)

- **Function**: ‘Power(x, n)‘ - Tính lũy thừa x^n.
- **Operator**: ‘^‘ - Toán tử được định nghĩa bằng hàm ‘Power‘.

Ví dụ về ObjectInstance (Thực thể)

Một thực thể cụ thể của ‘TamGiac‘ trong bộ nhớ RAM:

- **ID**: ‘550e8400-e29b-41d4-a716-446655440000‘
- **Concept**: ‘TamGiac‘
- **Values**: ‘{ ”a”: 3, ”b”: 4, ”c”: 5 }‘

CHƯƠNG 6: Ngôn ngữ Truy vấn Tri thức KBQL

6.1 Giới thiệu về Ngôn ngữ Truy vấn KBQL

KBQL (Knowledge Base Query Language) là ngôn ngữ truy vấn chính được sử dụng trong hệ quản trị KBMS. KBQL không chỉ kế thừa các cú pháp SQL tiêu chuẩn để thao tác với dữ liệu mà còn mở rộng các khả năng suy diễn tri thức (Reasoning) dựa trên logic vị từ và tập luật (Rules).

6.1.1 Triết lý Thiết kế

KBQL được thiết kế dựa trên ba trụ cột chính:

1. **Sự quen thuộc (Familiarity):** Cú pháp gần gũi với SQL giúp người dùng dễ dàng tiếp cận.
2. **Tính tri thức (Knowledge-Driven):** Tích hợp sâu các khái niệm về Concept, Fact và Rule thay vì chỉ dừng lại ở Table/Row.
3. **Tự động suy diễn (Automatic Inference):** Kết quả truy vấn có thể được tự động cập nhật hoặc suy luận thông qua bộ máy Suy diễn (Inference Engine) mà người dùng không cần viết logic thủ công.

6.1.2 Các Thành phần Chính của KBQL

Nhóm Lệnh	Chức năng	Các lệnh tiêu biểu
KDL (Knowledge Definition)	Định nghĩa cấu trúc tri thức, luật, phân cấp	‘CREATE KB‘, ‘CONCEPT‘, ‘RULE‘, ‘HIERARCHY‘, ‘RELATION‘
KML (Knowledge Manipulation)	Thao tác trên tập các sự kiện (Facts)	‘INSERT‘, ‘UPDATE‘, ‘DELETE‘, ‘IMPORT‘, ‘EXPORT‘
KQL (Knowledge Query)	Truy vấn và yêu cầu suy diễn	‘SELECT‘, ‘SOLVE‘, ‘SHOW‘, ‘EXPLAIN‘, ‘DESCRIBE‘
KCL (Knowledge Control)	Quản lý người dùng và quyền truy cập	‘GRANT‘, ‘REVOKE‘, ‘CREATE/ALTER/DROP USER‘
TCL (Transaction Control)	Quản lý giao dịch và tính toàn vẹn	‘BEGIN‘, ‘COMMIT‘, ‘ROLLBACK‘
Admin (Maintenance)	Bảo trì và tối ưu hóa hệ thống	‘MAINTENANCE (VACUUM, REINDEX, CHECK)‘

Bảng 6.1: Phân loại Từ khóa dành riêng (Reserved Keywords)

6.1.3 Hỗ trợ Chỉnh sửa (ALTER Support)

Không phải tất cả các đối tượng trong KBMS đều hỗ trợ lệnh ‘ALTER‘. Dưới đây là bảng tra cứu:

Đối tượng	Hỗ trợ ALTER	Ghi chú
Concept	Có	Hỗ trợ thêm/xóa biến, luật, ràng buộc, quan hệ nội bộ.
Knowledge Base	Có	Hỗ trợ thay đổi mô tả (Description).
User	Có	Hỗ trợ đổi mật khẩu và vai trò quản trị.
Relation	Không	Cần ‘DROP‘ và ‘CREATE‘ lại.
Hierarchy	Không	Sử dụng ‘ADD/REMOVE HIERARCHY‘.
Rule (Toàn cục)	Không	Cần ‘DROP‘ và ‘CREATE‘ lại (Khác với Rule nội bộ Concept).
Operator/Function	Không	Cần ‘DROP‘ và ‘CREATE‘ lại.

Bảng 6.2: Khả năng hỗ trợ lệnh ALTER cho từng đối tượng KBQL

6.1.4 Kiểu Dữ liệu Hỗ trợ (Data Types)

KBQL cung cấp bộ kiểu dữ liệu phong phú để định nghĩa Concept:

Nhóm	Kiểu dữ liệu	Mô tả
Số học	‘INT‘, ‘BIGINT‘, ‘DECIMAL‘, ‘FLOAT‘, ‘DOUBLE‘	Các kiểu số nguyên và số thực.
Chuỗi	‘VARCHAR(n)‘, ‘CHAR(n)‘, ‘TEXT‘, ‘STRING‘	Lưu trữ văn bản (hỗ trợ độ dài tùy chỉnh).
Logic	‘BOOLEAN‘	Giá trị ‘true‘ hoặc ‘false‘.
Thời gian	‘DATE‘, ‘DATETIME‘, ‘TIMESTAMP‘	Quản lý thời gian và sự kiện.
Tri thức	‘OBJECT‘, ‘<ConceptName>‘	Tham chiếu đến một đối tượng hoặc một Khái niệm khác.
Đặc biệt	‘NULL‘	Trạng thái rỗng.

Bảng 6.3: Phân loại Kiểu dữ liệu nguyên thuỷ trong KBQL

6.1.5 Khái niệm Cốt lõi

Concept (Khái niệm)

Thay vì dùng ”Table”, KBQL sử dụng **Concept**. Một Concept đại diện cho một thực thể hoặc một lớp đối tượng trong thế giới thực, bao gồm các biến (Variables) định nghĩa thuộc tính của nó.

Fact (Sự kiện)

Mỗi bản ghi dữ liệu trong một Concept được coi là một **Fact**. Tập hợp các Fact tạo nên cơ sở dữ liệu hiện tại.

Rule (Luật)

Rule định nghĩa mối quan hệ logic giữa các Fact. Khi dữ liệu (Fact) thay đổi, các Rule liên quan có thể được kích hoạt để tạo ra Fact mới hoặc cập nhật Fact hiện có (Forward Chaining).

Tiếp theo, chúng ta sẽ đi sâu vào chi tiết các lệnh định nghĩa (DDL).

6.2 Định nghĩa Tri thức (KDL)

KDL là tập hợp các lệnh dùng để định nghĩa cấu trúc, logic và các mối quan hệ trong Cơ sở tri thức (Knowledge Base).

6.2.1 Quản lý Cơ sở Tri thức (Knowledge Base)

Lệnh khởi tạo hoặc xóa bỏ một vùng chứa tri thức logic.

- **CREATE KNOWLEDGE BASE <name> [DESCRIPTION "<text>"]**: Khởi tạo KB mới.
- **ALTER KNOWLEDGE BASE <name> SET (DESCRIPTION: "<text>")**: Cập nhật mô tả cho KB.
- **DROP KNOWLEDGE BASE <name>**: Xóa toàn bộ dữ liệu và cấu trúc của KB.
- **USE <name>**: Chuyển ngữ cảnh làm việc sang KB được chỉ định.

6.2.2 Định nghĩa Khái niệm (Concept)

Concept là thực thể cốt lõi trong KBMS, tương tự như Class trong OOP hay Table trong SQL nhưng linh hoạt hơn.

Tạo Concept (CREATE CONCEPT)

Lệnh này định nghĩa cấu trúc của một Khái niệm. Bạn có thể định nghĩa theo kiểu liệt kê biến đơn giản hoặc sử dụng các khối chuyên sâu:

```
CREATE CONCEPT <name> (
    VARIABLES (<var>: <type>, ...),
    ALIASES (<alias1>, <alias2>),
    BASE_OBJECTS (<obj1>, <obj2>),
    CONSTRAINTS (<if_then_constraints>),
    SAME_VARIABLES (<var1>, <var2>),
    CONSTRUCT_RELATIONS (<rel_definitions>),
    PROPERTIES (<prop1>, <prop2>),
    RULES (<logic_rules>),
    EQUATIONS (<math_equations>)
);
```

Chi tiết các khối nâng cao:

- **ALIASES**: Định nghĩa các tên gọi khác cho Concept.
- **BASE_OBJECTS**: Liệt kê các đối tượng cơ sở cấu thành nên Concept này (thường dùng trong PART_OF).
- **CONSTRAINTS**: Các ràng buộc logic nội bộ phải thỏa mãn.
- **SAME_VARIABLES**: Khai báo các biến ở các thành phần khác nhau nhưng trỏ cùng một giá trị tri thức.
- **CONSTRUCT_RELATIONS**: Tự động tạo lập quan hệ khi dữ liệu được nạp vào.
- **PROPERTIES**: Các đặc tính metadata của Concept.
- **RULES / EQUATIONS**: Định nghĩa luật và phương trình ngay bên trong Concept thay vì tạo lệnh rời.

Chỉnh sửa Concept (ALTER CONCEPT)

Hỗ trợ thay đổi cấu trúc mà không làm mất dữ liệu hiện có.
Cấu trúc lệnh

```
ALTER CONCEPT <name> (
    ADD (
        VARIABLE <var>: <type>,
        RULE (<rule_definition>),
        CONSTRAINT (<expression>),
        EQUATION '<expression>',
        PROPERTY <key>: <value>
    ),
    DROP (
        VARIABLE <name>,
        RULE <name>,
        CONSTRAINT <name>
    ),
    RENAME (VARIABLE <old> TO <new>)
);
```

6.2.3 Quản lý Phân cấp (Hierarchy)

Định nghĩa quan hệ cha-con hoặc thành phần giữa các Concept.

- **IS_A / ISA**: Quan hệ kế thừa (ví dụ: *Square IS_A Rectangle*).
- **PART_OF**: Quan hệ thành phần (ví dụ: *Engine PART_OF Car*).

Cú pháp

```
ADD HIERARCHY <child> IS_A <parent>;
ADD HIERARCHY <part> PART_OF <whole>;
REMOVE HIERARCHY <parent> IS_A <child>;
```

6.2.4 Định nghĩa Quan hệ (Relation)

Mô tả các mối liên kết ngữ nghĩa giữa hai Concept.

Cú pháp

```
CREATE RELATION <name>
FROM <domain> TO <range>
[PARAMS (<p1>, ...)]
[PROPERTIES (<transitive, symmetric, functional, ...>)];
```

6.2.5 Định nghĩa Luật (Rule)

Định nghĩa logic suy diễn tự động (Forward Chaining).

Cú pháp

```
CREATE RULE <name>
SCOPE <concept>
IF <condition>
THEN <action>;
```

Ví dụ: ‘CREATE RULE FeverCheck SCOPE Patient IF temp > 37.5 THEN SET status = ’Fever’;‘

6.2.6 Toán tử & Hàm Tùy biến (Operator & Function)

Mở rộng khả năng tính toán của hệ thống.

- **CREATE OPERATOR <symbol>**: Định nghĩa toán tử mới (ví dụ: ‘|-|‘).
- **CREATE FUNCTION <name>**: Định nghĩa hàm con xử lý logic.

Cú pháp chung

```
CREATE {OPERATOR | FUNCTION} <identity>
PARAMS (<p1> <type1>, ...)
RETURNS <type>
BODY '<script_logic>';
```

6.2.7 Công thức tự động (Computation)

Gán công thức tính toán trực tiếp vào các biến của Concept.

Cú pháp

```
ADD COMPUTATION TO <concept>
VARIABLES <inputs>, <result>
FORMULA '<expression>';
```

6.2.8 Chỉ mục (Index)

Tối ưu hóa tốc độ truy vấn trên các biến cụ thể của Concept (sử dụng B+ Tree).

Cú pháp

```
CREATE INDEX <index_name> ON <concept_name> (<variable_list>);
DROP INDEX <index_name>;
```

6.2.9 Sự kiện tự động (Trigger)

Tự động thực thi một câu lệnh KBQL khi có sự kiện dữ liệu xảy ra.

Cú pháp

```
CREATE TRIGGER <name>
ON ( {INSERT|UPDATE|DELETE} OF <concept> )
DO ( <kbql_statement> );
```

Ví dụ: ‘CREATE TRIGGER LogInsert ON (INSERT OF Patient) DO (PRINT ’New patient added’);‘

6.3 Thao tác tri thức (KML)

KML là tập hợp các lệnh dùng để chèn, cập nhật, xóa các Sự kiện (Facts) và chuyển đổi dữ liệu trong Cơ sở tri thức.

6.3.1 Chèn Sự kiện (INSERT)

Lệnh chèn một hoặc nhiều Fact vào một Concept.

Chèn một dòng đơn

```
INSERT INTO <concept_name> ATTRIBUTE (<val1>, <val2>, ...);
```

Chèn hàng loạt (Bulk Insert)

Dùng để nạp dữ liệu lớn một cách nhanh chóng.

```
INSERT BULK INTO <concept_name> ATTRIBUTE (
    (<val1a>, <val2a>, ...),
    (<val1b>, <val2b>, ...),
    ...
);
```

6.3.2 Cập nhật Sự kiện (UPDATE)

Sửa đổi giá trị các biến của các Fact hiện có dựa trên điều kiện lọc.

Cú pháp

```
UPDATE <concept_name>
ATTRIBUTE (SET <var1>: <new_val1>, <var2>: <new_val2>)
WHERE <conditions>;
```

Lưu ý: Khi cập nhật một thuộc tính, hệ thống sẽ tự động chạy lại các Luật (Rules) liên quan để đảm bảo tính nhất quán của tri thức (Consistency).

6.3.3 Xóa Sự kiện (DELETE)

Loại bỏ các Fact khỏi Concept.

Cú pháp

```
DELETE FROM <concept_name> WHERE <conditions>;
```

6.3.4 Chuyển đổi Dữ liệu (IMPORT / EXPORT)

Công cụ sao lưu và phục hồi tri thức từ các tệp tin bên ngoài.

Xuất dữ liệu (EXPORT)

```
EXPORT (
    CONCEPT: <name>,
    FILE: '<path>',
    FORMAT: {CSV|JSON|XML}
);
```

Nhập dữ liệu (IMPORT)

```
IMPORT (
    CONCEPT: <name>,
    FILE: '<path>',
    FORMAT: {CSV|JSON|XML}
);
```

6.3.5 Đặc điểm của KML trong KBMS

Khác với các hệ quản trị CSDL thông thường, mọi thao tác KML trong KBMS đều có thể kích hoạt:

1. **Reasoning Engine:** Các luật suy diễn tiến sẽ được kích hoạt ngay lập tức sau khi ‘INSERT‘ hoặc ‘UPDATE‘.
2. **Trigger Engine:** Các sự kiện tự động (‘Trigger‘) sẽ được thực thi nếu có định nghĩa cho sự kiện tương ứng.
3. **Validation:** Dữ liệu được kiểm tra kiểu dữ liệu và các ràng buộc (Constraints) ngay tại tầng Parser và Storage.

6.4 Truy vấn và Suy diễn (KQL)

KQL là tập hợp các lệnh dùng để lấy thông tin, thực hiện truy vấn dữ liệu và yêu cầu hệ thống thực hiện các phép suy diễn tri thức phức tạp.

6.4.1 Truy vấn Dữ liệu (SELECT)

Lệnh ‘SELECT‘ trong KBQL tương tự SQL nhưng được tối ưu để làm việc với các Concept và thuộc tính tri thức.

Cú pháp đầy đủ

```
SELECT [<columns> | * | AGGREGATE(<var>)]
FROM <concept> [AS <alias>]
[JOIN <concept> ON <condition>]
[WHERE <conditions>]
[GROUP BY <vars>]
[HAVING <condition>]
[ORDER BY <vars> {ASC|DESC}]
[LIMIT <n> OFFSET <m>];
```

Các tính năng đặc biệt

- **Hàm Tổng hợp (Aggregates):** Hỗ trợ ‘COUNT‘, ‘SUM‘, ‘AVG‘, ‘MIN‘, ‘MAX‘.
- **HAVING:** Mệnh đề lọc sau khi đã thực hiện ‘GROUP BY‘.
- **Hàm Tính toán CALC():** Thực hiện các biểu thức toán học phức tạp ngay trong lúc truy vấn. *Ví dụ:* ‘SELECT name, CALC(price * 1.1) AS price_tax FROM Product;‘

6.4.2 Giải quyết Tri thức (SOLVE)

Đây là lệnh đặc trưng của KBMS, dùng để kích hoạt bộ máy giải quyết vấn đề (Problem Solver) dựa trên các công thức và luật đã định nghĩa.

Cú pháp

```
SOLVE ON CONCEPT <name>
GIVEN <inputs>
FIND <outputs>
[SAVE]
[USING <rule_or_relation_list>];
```

Cách hoạt động

1. **Input:** Nạp các giá trị đã biết vào phần ‘GIVEN‘.
2. **Inference:** Hệ thống sử dụng Forward Chaining và Equation Solving để tìm giá trị của các biến trong phần ‘FIND‘.
3. **Persist (Tùy chọn):** Nếu có từ khóa ‘SAVE‘, các kết quả vừa tìm thấy sẽ được lưu trực tiếp thành một Fact mới vào Concept tương ứng.
4. **Result:** Trả về kết quả sau khi đã suy diễn thành công.

6.4.3 Kiểm tra Hệ thống (SHOW)

Dùng để liệt kê các thành phần đang có trong Cơ sở Tri thức.

- **SHOW CONCEPTS**: Liệt kê tất cả các Khái niệm.
- **SHOW RULES**: Liệt kê các luật suy diễn.
- **SHOW RELATIONS**: Liệt kê các quan hệ giữa các Concept.
- **SHOW HIERARCHIES**: Xem cấu trúc phân cấp cây tri thức.
- **SHOW OPERATORS / FUNCTIONS**: Xem các toán tử và hàm tùy biến.

6.4.4 Phân tích & Mô tả (DESCRIBE / EXPLAIN)

Công cụ dành cho lập trình viên để hiểu sâu về cấu trúc và cách thực thi của hệ thống.

- **DESCRIBE {CONCEPT|KB|RULE|...} <name>**: Hiển thị chi tiết cấu trúc định nghĩa của một đối tượng.
- **EXPLAIN (<kbql_statement>)**: Hiển thị kế hoạch thực thi (Execution Plan) của một câu lệnh, rất hữu ích để tối ưu hóa truy vấn.

6.4.5 Truy vấn Metadata (Hệ thống)

Bạn có thể truy vấn trực tiếp vào danh mục hệ thống để lấy thông tin metadata.
Ví dụ: ‘SELECT * FROM <concept_name>.variables;‘ trả về danh sách các biến và kiểu dữ liệu của Concept đó.

6.5 Kiểm soát truy cập (KCL)

KCL là tập hợp các lệnh dùng để quản lý hệ thống bảo mật, tài khoản và quyền hạn truy cập của người dùng trong KBMS.

6.5.1 Quản lý Người dùng (USER Management)

KBMS cho phép định cấu hình người dùng và vai trò để bảo vệ tri thức khỏi các truy cập không mong muốn.

Tạo Người dùng mới (CREATE USER)

```
CREATE USER <username>
PASSWORD '<password>'
[ROLE {ADMIN|SERVICE|USER}];
```

Chỉnh sửa Người dùng (ALTER USER)

Sử dụng để đổi mật khẩu hoặc trạng thái quản trị.

```
ALTER USER <username> (
    SET (PASSWORD: '<new_pass>', ADMIN: true)
);
```

Xóa Người dùng (DROP USER)

```
DROP USER <username>;
```

6.5.2 Quản trị Quyền hạn (GRANT / REVOKE)

Sử dụng để cấp hoặc thu hồi quyền thao tác trên các Concept và Tri thức cụ thể.

Cấp quyền (GRANT)

```
GRANT {SELECT, INSERT, UPDATE, DELETE, ...}
ON CONCEPT <name>
TO <username>;
```

Thu hồi quyền (REVOKE)

```
REVOKE {SELECT, INSERT, UPDATE, DELETE, ...}
ON CONCEPT <name>
FROM <username>;
```

6.5.3 Các thực thể Quản trị đặc biệt (System Admin)

- **ADMIN**: Có toàn quyền trên tất cả các KB, Concept và người dùng khác.
- **SERVICE**: Có quyền truy cập vào các API hệ thống nhưng không thể thay đổi cấu trúc Metadata.
- **USER**: Quyền hạn mặc định, cần được cấp phép cụ thể cho từng đối tượng.

6.6 TCL: Ngôn ngữ Kiểm soát Giao dịch (Transaction Control Language)

TCL là tập hợp các lệnh dùng để quản lý sự thực thi đồng nhất của các lệnh KBQL, đảm bảo tính toàn vẹn tri thức (ACID).

6.6.1 Một Giao dịch là gì? (Transaction)

Giao dịch là một đơn vị công việc bao gồm một hoặc nhiều lệnh thực thi trên KBMS. Nếu một phần của giao dịch thất bại, toàn bộ giao dịch có thể được hủy bỏ để không làm sai lệch tri thức hiện tại.

6.6.2 Các lệnh Giao dịch

6.6.3 Các lệnh Giao dịch

Bắt đầu Giao dịch (BEGIN TRANSACTION)

```
BEGIN TRANSACTION;
```

Từ thời điểm này, mọi thay đổi dữ liệu hoặc định nghĩa tri thức sẽ được thực thi tạm thời trong bộ đệm giao dịch.

Xác nhận Giao dịch (COMMIT)

```
COMMIT;
```

Lệnh này xác nhận toàn bộ các thay đổi trong giao dịch và lưu trữ vĩnh viễn vào các file B+ Tree và Catalog của hệ thống.

Hủy bỏ Giao dịch (ROLLBACK)

```
ROLLBACK;
```

Lệnh này hủy bỏ toàn bộ các lệnh từ lúc 'BEGIN TRANSACTION', đưa tri thức quay về trạng thái trước khi giao dịch bắt đầu.

6.6.4 Vai trò của Giao dịch trong KBMS

1. **Tính Nguyên tử (Atomicity):** Đảm bảo một bộ các Fact liên quan được nạp vào cùng lúc (ví dụ: Thông tin bệnh nhân và các triệu chứng chẩn đoán kèm theo).
2. **Tính Nhất quán (Consistency):** Ngăn chặn việc Rules thực hiện các thay đổi tri thức không đồng bộ giữa các Concept khác nhau.
3. **Cách ly (Isolation):** Các giao dịch đang diễn ra không ảnh hưởng đến các truy vấn suy diễn khác cho tới khi 'COMMIT' thành công.

6.7 Bảo trì Hệ thống (Maintenance)

Hệ quản trị KBMS cung cấp một bộ các lệnh quản trị chuyên sâu giúp tối ưu hóa bộ nhớ, chỉ mục và kiểm tra tính nhất quán của tri thức.

6.7.1 Lệnh Bảo trì (MAINTENANCE)

Lệnh 'MAINTENANCE' được dùng để thực hiện các tác vụ dọn dẹp và sửa lỗi hệ thống cấp thấp.

Cú pháp

```
MAINTENANCE (
    {VACUUM | REINDEX | CHECK CONSISTENCY} [ON CONCEPT <name>],
    ...
);
```

6.7.2 Các hành động cụ thể

Thu gọn Bộ nhớ (VACUUM)

Khi Fact bị xóa, không gian bộ nhớ trong cây B+ Tree không được giải phóng ngay lập tức. Lệnh 'VACUUM' thực hiện việc sắp xếp lại các Page dữ liệu và thu hồi dung lượng đĩa cứng dư thừa.

Ví dụ: 'MAINTENANCE (VACUUM);'

Đánh lại Chỉ mục (REINDEX)

Cập nhật lại toàn bộ các Index gắn với Concept. Hữu ích khi hệ thống gặp sự cố mất điện hoặc phát hiện sai lệch chỉ mục.

Ví dụ: ‘MAINTENANCE (REINDEX ON CONCEPT Patient);‘

Kiểm tra Tính nhất quán (CHECK CONSISTENCY)

Kiểm tra xem các liên kết giữa Fact, Rule và Hierarchy có còn hợp lệ hay không. Đảm bảo tri thức không bị mâu thuẫn.

Ví dụ: ‘MAINTENANCE (CHECK CONSISTENCY ON CONCEPT (*));‘

6.7.3 Tần suất thực hiện bản khuyến nghị

- **VACUUM:** Sau mỗi lần xóa dữ liệu quy mô lớn (Batch delete).
- **REINDEX:** Định kỳ hàng tuần hoặc sau khi thực hiện việc thay đổi Metadata phức tạp.
- **CHECK CONSISTENCY:** Thực hiện trước khi triển khai hệ thống suy diễn vào môi trường quan trọng.

6.8 Biểu thức và Hàm số (Expressions)

Biểu thức là thành phần cốt lõi xuất hiện trong các mệnh đề ‘WHERE‘, ‘IF‘, ‘SET‘ và hàm ‘CALC()‘. KBQL tích hợp một bộ máy đánh giá biểu thức mạnh mẽ hỗ trợ đầy đủ các phép toán và hàm học thuật.

6.8.1 Toán tử Cơ bản (Operators)

Toán tử Số học

- ‘+‘, ‘-‘, ‘*‘, ‘/‘: Các phép toán cơ bản.
- ‘^‘: Lũy thừa.
- ‘%‘: Chia lấy dư.

Toán tử So sánh

- ‘=‘, ‘!=‘ (hoặc ‘<>‘): So sánh bằng và không bằng.
- ‘<‘, ‘<=‘, ‘>‘, ‘>=‘: So sánh thứ tự.

Toán tử Logic

- ‘AND‘, ‘OR‘, ‘NOT‘: Các phép logic Boolean.

6.8.2 Hàm Tích hợp (Built-in Functions)

KBMS tích hợp sẵn các hàm toán học sau đây để phục vụ tính toán tri thức:

Hàm	Giải thích	Ví dụ
‘Abs(x)‘	Giá trị tuyệt đối	‘Abs(-10)‘ → 10
‘Sqrt(x)‘	Căn bậc hai	‘Sqrt(16)‘ → 4
‘Pow(x, y)‘	Lũy thừa	‘Pow(2, 3)‘ → 8
‘Round(x, n)‘	Làm tròn đến n chữ số	‘Round(3.1415, 2)‘ → 3.14
‘Floor(x)‘ / ‘Ceiling(x)‘	Làm tròn xuống / lên	‘Floor(2.9)‘ → 2
‘Sin‘, ‘Cos‘, ‘Tan‘	Các hàm lượng giác	‘Sin(0)‘ → 0
‘Factorial(n)‘	Tính giai thừa	‘Factorial(5)‘ → 120

Bảng 6.4: Danh mục Hàm tích hợp (Built-in Functions) trong KBQL

6.8.3 Cách sử dụng trong CALC()

Hàm ‘CALC()‘ cho phép nhúng trực tiếp biểu thức vào kết quả trả về của ‘SELECT‘.
Ví dụ: ‘SELECT name, CALC(Sqrt(a^2 + b^2)) AS hypotenuse FROM Triangles;‘

6.8.4 Tương tác với Biến (Variables)

Bên trong các biểu thức, bạn có thể tham chiếu trực tiếp đến tên các biến của Concept hiện tại. Nếu có ‘JOIN‘, hãy sử dụng Alias để phân biệt: ‘e.salary * 1.1‘.

6.8.5 Lưu ý về Hiệu năng

Các biểu thức phức tạp trong ‘WHERE‘ có thể làm chậm tốc độ truy vấn nếu không có Index phù hợp. Hãy sử dụng lệnh ‘EXPLAIN‘ để kiểm tra cách hệ thống đánh giá các biểu thức của bạn.

6.9 Các trường hợp Chuyên gia (Expert Cases)

Tài liệu này trình bày các tình huống sử dụng KBQL ở mức độ nâng cao, khai thác tối đa sức mạnh của bộ máy suy diễn và giải toán của KBMS.

6.9.1 Giải ngược Phương trình (Inverse Problem Solving)

Hệ thống KBMS không chỉ tính toán một chiều mà còn có khả năng giải ngược các tham số đầu vào khi biết kết quả đầu ra thông qua các thuật toán **Newton-Raphson** và **Brent**.

Kịch bản: Tính toán mạch điện

Chúng ta biết luật Ohm ‘U = I * R‘. Nếu biết ‘U‘ và ‘I‘, hệ thống phải tự tìm ‘R‘.

```
CREATE CONCEPT Resistor (
    VARIABLES (u: DECIMAL, i: DECIMAL, r: DECIMAL),
    EQUATIONS (u = i * r)
);

-- Trường hợp 1: Biết I và R, tìm U (Tính xuôi)
SOLVE ON CONCEPT Resistor GIVEN i = 2, r = 10 FIND u; -- Kết quả: u = 20

-- Trường hợp 2: Biết U và I, tìm R (Tính ngược - Expert)
SOLVE ON CONCEPT Resistor GIVEN u = 220, i = 2 FIND r; -- Kết quả: r = 110
```

6.9.2 Suy diễn Chùm (Chained Inference)

Thể hiện khả năng lan truyền tri thức qua nhiều lớp luật (Forward Chaining).

Kịch bản: Hệ thống Cảnh báo sớm

```
CREATE RULE R1 SCOPE Sensor IF temp > 100 THEN SET status = 'Overheat';
CREATE RULE R2 SCOPE Sensor IF status = 'Overheat' AND pressure > 50 THEN SET alert = 'Critical';
CREATE RULE R3 SCOPE Sensor IF alert = 'Critical' THEN DO (PRINT 'EMERGENCY SHUTDOWN');

-- Khi chèn dữ liệu thỏa mãn chuỗi:
INSERT INTO Sensor ATTRIBUTE (110, 60);
-- Kết quả: R1 kích hoạt -> status='Overheat' -> R2 kích hoạt -> alert='Critical' -> R3 kích hoạt
```

6.9.3 Phân cấp Kế thừa Đa tầng (Recursive Property Propagation)

Khi định nghĩa Hierarchy, các luật ở Concept cha sẽ tự động ”chảy”xuống toàn bộ các con, cháu.

Kịch bản: Phân loại Hình học

```
CREATE CONCEPT Shape (VARIABLES (color: STRING));
CREATE CONCEPT Polygon (...);
CREATE CONCEPT Triangle (...);

ADD HIERARCHY Polygon IS_A Shape;
ADD HIERARCHY Triangle IS_A Polygon;

CREATE RULE ShapeColor SCOPE Shape IF color = 'Red' THEN SET is_hot = true;

-- Khi chèn một Triangle màu đỏ:
INSERT INTO Triangle ATTRIBUTE ('Red', ...);
-- Kết quả: Mặc dù luật định nghĩa ở Shape, nhưng Triangle vẫn nhận được is_hot = true.
```

6.9.4 Ràng buộc nội bộ phức tạp (Internal Constraints)

Sử dụng khối ‘CONSTRAINTS‘ để bảo vệ dữ liệu ở mức tri thức, không cho phép các Fact mâu thuẫn tồn tại.

Kịch bản: Quản lý nhân sự

```
CREATE CONCEPT Employee (
    VARIABLES (age: INT, experience: INT),
    CONSTRAINTS (
        IF age < experience + 18 THEN DO (ERROR 'Tuổi không hợp lệ so với kinh nghiệm')
    )
);

-- Lệnh này sẽ bị Parser/Engine chặn lại:
INSERT INTO Employee ATTRIBUTE (20, 5); -- (20 < 5 + 18) => Lỗi.
```

6.9.5 Giải hệ phương trình 2 ẩn (Newton-Raphson 2D)

KBMS hỗ trợ giải đồng thời các hệ thức toán học.

Kịch bản: Tìm giao điểm đường thẳng

```
CREATE CONCEPT Intersection (
    VARIABLES (x: DECIMAL, y: DECIMAL),
    EQUATIONS (
        y = 2 * x + 1,
        y = -x + 4
    )
);

SOLVE ON CONCEPT Intersection FIND x, y;
-- Kết quả: x = 1, y = 3 (Hệ thống tự giải hệ phương trình).
```

6.9.6 Kết hợp Trigger và Rule (The Chain Reaction)

Dùng Trigger để kích hoạt một luồng suy diễn ở một Concept hoàn toàn khác.

Kịch bản: Đồng bộ kho và đơn hàng

```
CREATE TRIGGER SyncStock ON (INSERT OF Order)
DO (UPDATE Product ATTRIBUTE (SET stock: stock - 1) WHERE id = new.product_id);

-- Khi cập nhật kho, các luật về 'Out of stock' sẽ tự động chạy:
CREATE RULE OutOfStock SCOPE Product IF stock < 0 THEN SET status = 'Refill';
```

6.10 Xác thực Ngôn ngữ (Language Validation)

Để đảm bảo mọi câu lệnh KBQL đều được diễn dịch chính xác trước khi gửi tới Engine, hệ thống KBMS trải qua quy trình kiểm thử cú pháp nghiêm ngặt.

6.10.1 Kiểm thử Cú pháp (Parsing Test)

Tầng Parser được kiểm soát bởi tệp **'ParserTests.cs'** với hơn 21,000 dòng kịch bản kiểm thử, bao phủ mọi trường hợp từ câu lệnh đơn giản đến phức tạp nhất.

Nhóm Kiểm thử	Mô tả	Trạng thái
KDL Validation	Kiểm tra định nghĩa Concept, Hierarchy, Relation.	**Passed**
KQL Validation	Kiểm tra các phép toán Join, Filter, Order By, Limit.	**Passed**
Expression Eval	Kiểm tra các biểu thức toán học phức tạp ('CALC()').	**Passed**
Error Handling	Kiểm tra khả năng báo lỗi đúng dòng, đúng cột.	**Passed**

Bảng 6.5: Phân bổ kịch bản kiểm thử theo nhóm ngôn ngữ

6.10.2 Bằng chứng Thực nghiệm (Evidence)

Kết quả chạy bộ kiểm thử 'ParserTests':

Thiếu ảnh: placeholder_parser_test_log.png

Hình 6.1: Kết quả kiểm thử bộ phân tích cú pháp (Parser) với hơn 21,000 kịch bản.

6.10.3 Nhật ký Lexer (Lexing Proof)

Trước khi phân tích cú pháp, Lexer thực hiện tách từ tố (Tokenization). Tệp 'LexerTests.cs' đảm bảo mọi ký tự đặc biệt và từ khóa đều được nhận diện đúng.

Thiếu ảnh: placeholder_lexer_token_stream.png

Hình 6.2: Luồng Token được phân tách bởi Lexer trước khi đưa vào Parser.

CHƯƠNG 7: Cơ chế Lưu trữ và Quản lý Dữ liệu nhị phân

7.1 Tham chiếu Thuật ngữ Kỹ thuật (Technical Glossary)

Để đảm bảo tính nhất quán và giúp người đọc dễ dàng tiếp cận các nội dung kỹ thuật chuyên sâu trong Chương 7, phần này định nghĩa các thuật ngữ nền tảng về hệ quản trị cơ sở dữ liệu (DBMS) được áp dụng trong KBMS V3.

7.1.1 Đơn vị lưu trữ và Cấu trúc dữ liệu

- **Page (Trang):** Đơn vị lưu trữ cơ bản và nhỏ nhất mà Hệ quản trị bộ nhớ (Storage Engine) có thể đọc hoặc ghi. Trong KBMS, mỗi trang có kích thước cố định là 16KB.
- **Tuple (Bản ghi):** Một tập hợp dữ liệu nhị phân đại diện cho một thực thể (ví dụ: một đối tượng tam giác). Một trang có thể chứa nhiều Tuples.
- **Slotted Page:** Kỹ thuật tổ chức trang bằng cách sử dụng một mảng các khe cắm (Slots) ở đầu trang để trỏ đến dữ liệu nằm ở cuối trang, giúp quản lý các bản ghi có độ dài thay đổi hiệu quả.
- **B+ Tree:** Một cấu trúc cây tự cân bằng cho phép tìm kiếm, truy cập tuần tự, chèn và xóa dữ liệu trong thời gian logarit. Đây là cấu trúc chính để lưu trữ chỉ mục trong KBMS.

7.1.2 Quản lý Bộ nhớ và I/O

- **I/O (Input/Output):** Thao tác đọc/ghi dữ liệu giữa bộ nhớ RAM và thiết bị lưu trữ vật lý (SSD/HDD). I/O là thao tác tốn kém nhất về hiệu năng.
- **Buffer Pool (Bộ đệm):** Một vùng nhớ RAM trung gian dùng để lưu trữ tạm thời các trang dữ liệu từ đĩa. Giúp giảm thiểu số lần phải đọc/ghi trực tiếp xuống đĩa.
- **LRU (Least Recently Used):** Thuật toán thay thế trang trong bộ đệm, ưu tiên giữ lại các trang được truy cập thường xuyên nhất và loại bỏ các trang ít được dùng nhất khi bộ đệm đầy.
- **Flush:** Hành động ghi một trang dữ liệu từ RAM xuống đĩa cứng để đảm bảo tính vĩnh cửu.

7.1.3 Độ bền và An toàn dữ liệu

- **WAL (Write-Ahead Logging):** Nguyên tắc "Ghi nhật ký trước khi ghi dữ liệu". Mọi thay đổi phải được ghi vào file log trước khi cập nhật vào file dữ liệu chính để phòng ngừa mất dữ liệu khi sập nguồn.
- **LSN (Log Sequence Number):** Số thứ tự duy nhất được gán cho mỗi bản ghi nhật ký (Log Record) để theo dõi trình tự thời gian của các giao dịch.
- **ACID:** Các tính chất đảm bảo độ tin cậy của giao dịch: Nguyên tử (Atomicity), Nhất quán (Consistency), Cô lập (Isolation), và Bền vững (Durability).
- **AES-256:** Tiêu chuẩn mã hóa khóa đối xứng với độ dài khóa 256-bit, được KBMS dùng để bảo vệ dữ liệu tĩnh trên đĩa.

7.1.4 Cơ chế chuyển đổi dữ liệu

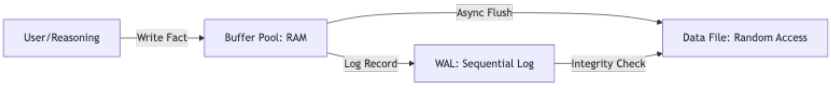
- **Serialization (Tuần tự hóa):** Quá trình chuyển đổi cấu trúc dữ liệu hoặc trạng thái đối tượng (trong C#) thành một định dạng (chuỗi byte) có thể lưu trữ hoặc truyền đi.
- **Deserialization (Giải tuần tự hóa):** Quá trình ngược lại, tái tạo đối tượng từ chuỗi byte nhị phân.

7.2 Tổng quan về Kiến trúc Lưu trữ (Storage Architecture)

Cỗ máy lưu trữ của KBMS (phiên bản V3) chịu trách nhiệm quản lý việc ghi dữ liệu tri thức xuống đĩa cứng một cách bền vững và truy xuất chúng với hiệu năng cao nhất. Kiến trúc này được thiết kế theo mô hình phân tầng từ vật lý đến logic.

7.2.1 Hệ thống Phân tầng I/O (Multi-layered I/O)

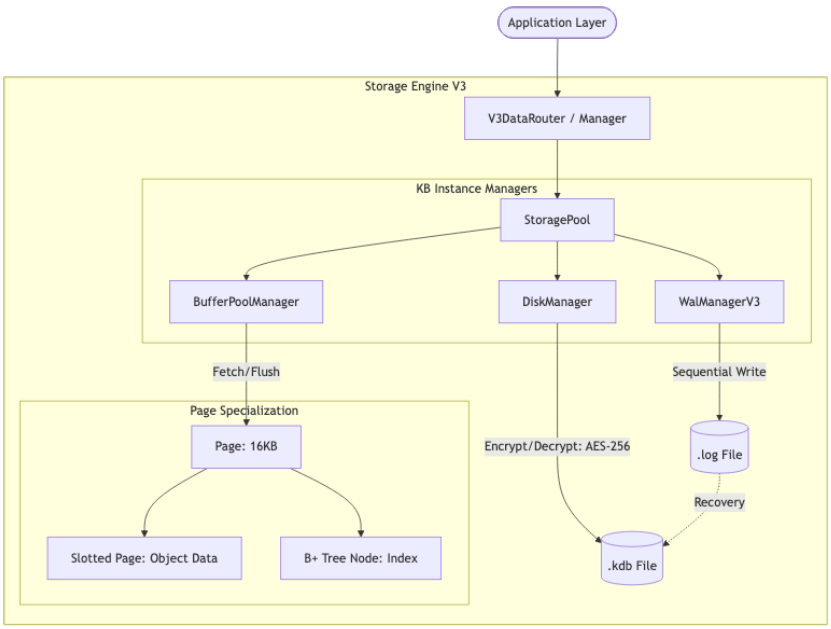
Dữ liệu trong KBMS luân chuyển qua 3 trạng thái chính để đảm bảo sự cân bằng giữa tốc độ truy cập RAM và độ an toàn của đĩa cứng:



Hình 7.1: Sơ đồ chiến lược luồng I/O đa tầng của KBMS.

7.2.2 Sơ đồ Kiến trúc Thành phần (Component Architecture)

Kiến trúc nội tại của Storage Engine V3 được tổ chức theo mô hình quản lý tập trung (‘StoragePool‘), đảm bảo tính cô lập giữa các Cơ sở tri thức (KB) khác nhau.



Hình 7.2: Sơ đồ kiến trúc thành phần chi tiết của Storage Engine V3.

7.2.3 Quản lý Bộ đệm (Buffer Pool Manager)

Để giảm thiểu I/O chậm chạp, KBMS duy trì một vùng nhớ RAM gọi là **Buffer Pool**. Thành phần này hoạt động như một lớp trung gian thông minh giữa RAM và Đĩa.

Thuật toán thay thế LRU (Least Recently Used)

KBMS sử dụng chính sách LRU để điều phối các trang trong bộ đệm:

- **Pinning (Ghim trang):** Khi một trang đang được truy cập, ‘PinCount‘ tăng lên. Trang có ‘PinCount > 0‘ sẽ không bị đẩy ra khỏi RAM.
- **Dirty Pages:** Những trang có thay đổi dữ liệu sẽ được đánh dấu ‘IsDirty = true‘.
- **Eviction (Trục xuất):** Khi bộ đệm đầy, hệ thống tìm trang có ‘PinCount = 0‘ và không được sử dụng trong thời gian lâu nhất để giải phóng. Nếu trang đó là ”Dirty”, nó sẽ được ghi xuống đĩa (Flush) trước khi bị thay thế.

7.2.4 Hạn mức Kỹ thuật của Kiến trúc V3

Kiến trúc phân trang 16KB cho phép KBMS mở rộng mạnh mẽ:

Thông số	Giá trị	Giải thích
Kích thước Trang	**16 KB**	Khớp với block size tối ưu của SSD hiện đại.
Dung lượng tối đa	**~35.25 TB**	Giới hạn bởi 2.1 tỷ Page ID (4-byte).
Tốc độ Look-up	**O(1)**	Nhờ sử dụng Hash Table để quản lý Frame thay vì quét mảng.
Mã hóa tĩnh	**AES-256**	Được tích hợp trực tiếp tại tầng DiskManager.

Bảng 7.1: Thông số định lượng của Storage Engine V3

7.2.5 Lộ trình Triển khai Kỹ thuật

Tiến trình xây dựng Backend cho Storage Engine được thực thi qua 4 giai đoạn:

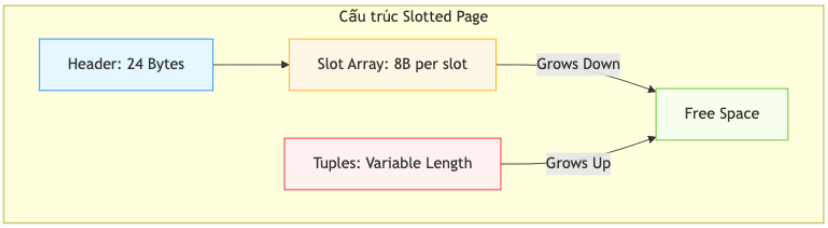
1. **Tầng Vật lý:** Xây dựng ‘DiskManager‘ hỗ trợ ‘Random Access‘ và ‘AES Encryption‘.
2. **Tầng Quản lý RAM:** Lập trình ‘BufferPoolManager‘ và cơ chế thay thế trang LRU.
3. **Tầng Cấu trúc Dữ liệu:** Thiết kế giải thuật ‘Slotted Page‘ cho dữ liệu biến thiên.
4. **Tầng Chỉ mục:** Xây dựng cấu trúc cây ‘B+ Tree‘ để kết nối hàng triệu trang thành một hệ thống đồng nhất.

7.3 Giải phẫu Trang vật lý (Page Anatomy)

KBMS V3 sử dụng mô hình phân trang (Paging) và cấu trúc **Slotted Page** để tối ưu hóa truy xuất ngẫu nhiên và quản lý không gian trống hiệu quả cho các bản ghi có độ dài biến thiên.

7.3.1 Cấu trúc nội tại của Trang (Internal Structure)

Một trang dữ liệu chuẩn 16KB được phân rã thành 4 thành diện diện tích chính: **Header**, **Slot Array**, **Free Space**, và **Tuples**.



Hình 7.3: Giải phẫu cấu trúc trang nhị phân Slotted-Page của KBMS.

- **Header (24 Bytes)**: Nằm ở đầu trang, chứa các metadata điều phối.
- **Slot Array**: Danh sách các ”khe cắm”trỏ tới dữ liệu, phát triển theo chiều **tiến** (từ đầu trang xuống). Mỗi slot chiếm 8 bytes: ‘[Offset (4B) | Length (4B)]‘.
- **Free Space**: Vùng nhớ trống nằm giữa Slot Array và Tuples.
- **Tuples (Dữ liệu)**: Nội dung thực tế, được chèn theo chiều **lùi** (từ cuối trang ngược lên trên) để tận dụng tối đa không gian trống ở giữa.

7.3.2 Đặc tả Header (24-Byte Layout)

Cấu trúc Header được thiết kế để hỗ trợ tối đa cho việc phục hồi dữ liệu (Recovery) và điều hướng B+ Tree:

Byte Offset	Trường dữ liệu	Ý nghĩa
0 - 3	**PageId**	ID duy nhất của trang trong tệp dữ liệu.
4 - 7	**LSN**	Log Sequence Number phục vụ cơ chế ghi nhật ký WAL.
8 - 11	**PrevPageId**	Con trỏ tới trang trước trong chuỗi lá B+ Tree.
12 - 15	**NextPageId**	Con trỏ tới trang sau phục vụ quét tuần tự (Scan).
16 - 19	**FreeSpacePointer**	Offset đánh dấu ranh giới bắt đầu của vùng dữ liệu.
20 - 23	**TupleCount**	Tổng số lượng bản ghi (Slots) hiện có trong trang.

Bảng 7.2: Bảng 11

7.3.3 Quản lý bản ghi biến thiên (Slotted Page Logic)

Cơ chế này cho phép KBMS xử lý các bản ghi có kích thước khác nhau (ví dụ: một đối tượng có tên ngắn và một đối tượng có tên dài) mà không cần định nghĩa lại cấu trúc trang:

1. **Chèn bản ghi:** Dữ liệu mới được đẩy vào cuối vùng ‘Free Space‘, đồng thời một Slot mới được thêm vào đầu trang để trỏ đến vị trí đó.
2. **Định danh RID (Record ID):** Một bản ghi được xác định duy nhất bởi cặp ‘(PageId, SlotId)‘.
3. **Xóa & Thu hồi:** Khi xóa, Slot tương ứng được đánh dấu là trống. KBMS cung cấp lệnh ‘VACUUM‘ để dồn dịch dữ liệu và thu hồi không gian trống bị phân mảnh.

7.3.4 Lợi ích của kiến trúc 16KB

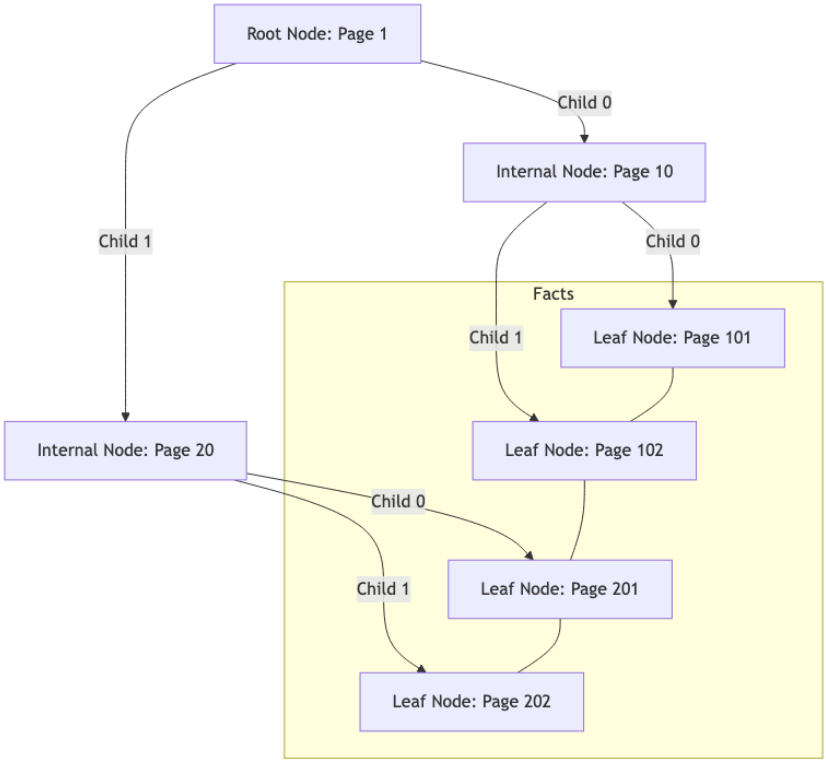
1. **Block Alignment:** 16KB là bội số của block size trên hầu hết các dòng SSD hiện đại, giúp tránh hiện tượng ”Read-Modify-Write”.
2. **Binary Search:** Nhờ Slot Array có kích thước cố định, việc tìm kiếm một bản ghi trong trang có độ phức tạp là **O(log N)** bằng tìm kiếm nhị phân trên mảng Slot.

7.4 Chỉ mục Cây B+ Tree (Indexing Structure)

KBMS sử dụng cây B+ Tree làm cấu trúc dữ liệu chính để quản lý và truy xuất các thực thể (‘ObjectInstance‘) trong mỗi Concept. B+ Tree là lựa chọn tối ưu cho việc truy vấn dữ liệu từ bộ nhớ ngoài (Disk) vì nó giảm thiểu số lần đọc đĩa (I/O) cần thiết.

7.4.1 Cấu trúc Đa tầng (Tiered Structure)

Cây B+ Tree trong KBMS V3 thường duy trì từ 3 đến 5 tầng tùy thuộc vào khối lượng tri thức:



Hình 7.4: Sơ đồ cấu trúc đa tầng của cây B+ Tree trong việc quản lý thực thể.

1. **Nút Gốc (Root Node):** Điểm bắt đầu của mọi truy vấn. Nếu cây chỉ có 1 trang, Root đồng thời là Leaf.
2. **Nút Trung gian (Internal Nodes):** Chứa các khóa dẫn hướng (Keys) và con trỏ trang (‘PageId‘). Các nút này không chứa dữ liệu thực tế.
3. **Nút Lá (Leaf Nodes):** Tầng cuối cùng của cây, nơi chứa toàn bộ dữ liệu của các ‘Tuple‘. Các nút lá được liên kết với nhau bằng con trỏ ‘NextPageId‘ tạo thành một danh sách liên kết kép, cho phép quét dữ liệu theo phạm vi (Range Scan) cực nhanh.

7.4.2 Các Thuật toán Vận hành Cốt lõi

Tìm kiếm (Search) - $O(\log_b n)$

Hệ thống bắt đầu từ Root, so sánh khóa cần tìm với các ranh giới trong Nút trung gian để chọn ‘PageId‘ tiếp theo, cho đến khi chạm tới Nút lá. Tại đây, ‘Slotted Page‘ sẽ thực hiện tìm kiếm nhị phân trên ‘Slot Array‘ để lấy dữ liệu.

Chèn và Tách trang (Insert & Split)

Khi một trang bị đầy (Overflow):

1. **Tách trang:** Một trang mới được cấp phát.
2. **Phân bổ:** Một nửa số lượng bản ghi được chuyển sang trang mới.
3. **Cập nhật cha:** Khóa phân tách được chèn vào Nút trung gian cấp trên. Nếu Nút cha cũng đầy, quá trình tách sẽ đệ quy ngược lên trên, có thể làm tăng chiều cao của cây.

Xóa và Gộp trang (Delete & Merge)

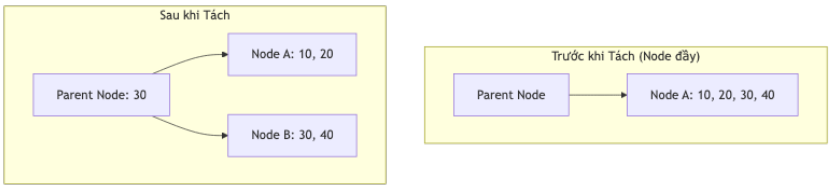
Khi số lượng bản ghi trong một trang giảm xuống dưới 50% (Underflow):

- **Vay mượn (Borrow):** Hệ thống kiểm tra các trang lân cận để chuyển bớt bản ghi sang.
- **Gộp trang (Merge):** Nếu không thể vay mượn, hai trang lân cận sẽ được gộp làm một và trang thừa sẽ được giải phóng về cho 'DiskManager'.

7.4.3 Clustered Index trong KBMS

Khác với các hệ quản trị CSDL truyền thống thường tách rời Index và Data, KBMS V3 sử dụng mô hình **Clustered Index**:

- Dữ liệu thực tế của 'ObjectInstance' nằm trực tiếp tại các Nút lá của cây.
- **Lợi ích:** Tiết kiệm một lần truy xuất đĩa (Disk Seek) vì sau khi tìm thấy khóa trong Index, hệ thống có ngay dữ liệu mà không cần trỏ đến một file dữ liệu riêng biệt.
- **Hiệu năng:** Cực kỳ hiệu quả cho các truy vấn theo dải (ví dụ: 'SELECT * FROM TamGiac WHERE canh_a > 5').



Hình 7.5: Cơ chế tách nút (Split) tự động khi trang dữ liệu bị đầy.

7.5 Ánh xạ Mô hình sang Dữ liệu nhị phân (Model Serialization)

Cỗ máy lưu trữ V3 (Storage Engine V3) đóng vai trò là xương sống bền vững cho KBMS, thực hiện việc chuyển đổi hai chiều giữa các thực thể lập trình (C# Objects) và các khối nhị phân lưu trữ trên đĩa (Binary Pages).

7.5.1 Chiến lược Nhất quán và Linh hoạt

Hệ thống phân tách chiến lược tuần tự hóa thành hai luồng chính dựa trên đặc thù dữ liệu:

1. **Dữ liệu mật độ cao (ObjectInstance):** Sử dụng định dạng 'Tuple' nhị phân nén với bảng Offset để tối ưu hóa hiệu năng quét (Scan) và tìm kiếm (Index).
2. **Siêu dữ liệu & Lược đồ (Concept, Relation, Rule, KB):** Sử dụng định dạng JSON-to-Binary (UTF-8) để duy trì tính linh hoạt tối đa khi cấu trúc mô hình tri thức thay đổi.

7.5.2 Đặc tả Kỹ thuật của 'ObjectInstance' Tuple

Mỗi bản ghi đối tượng ('ObjectInstance') được gói gọn trong một đơn vị 'Tuple'. Cấu trúc nhị phân của nó tuân thủ sơ đồ phân bổ bộ nhớ sau:

Thành phần	Kích thước	Mô tả
Field Count	2 Bytes	Số lượng trường dữ liệu có trong bản ghi.
Offsets Array	$N \times 2$ Bytes	Mảng vị trí kết thúc của từng trường (tính từ đầu Tuple).
Data Payload	Biến thiên	Chứa dữ liệu thực tế của các trường đã được mã hóa.

Bảng 7.3: Bảng 12

Ánh xạ Logic trường dữ liệu

Trong KBMS V3, các trường dữ liệu của một ‘ObjectInstance‘ được sắp xếp theo thứ tự cố định:

- **Trường 0 (ID):** Chứa ‘GUID‘ (16 bytes) định danh duy nhất cho đối tượng.
- **Trường 1 (Schema):** Một chuỗi ký tự UTF-8 chứa tên các biến (Variables) ngăn cách bởi dấu gạch đứng ‘|‘. Đây là ”snapshot”của lược đồ tại thời điểm ghi.
- **Trường 2..N:** Chứa giá trị của các biến tương ứng dưới dạng chuỗi UTF-8.

7.5.3 Tuần tự hóa Siêu dữ liệu (Metadata Serialization)

Các đối tượng mang tính định nghĩa (Definitions) được quản lý bởi các Catalog chuyên biệt thông qua gói ‘System.Text.Json‘:

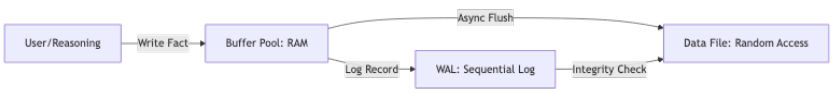
Quy trình lưu trữ Concept & KnowledgeBase

1. **Chuyển đổi:** Đối tượng C# → Chuỗi JSON → Mảng Byte (UTF-8).
2. **Đóng gói:** Mảng byte này được coi như một ‘Tuple‘ duy nhất và chèn vào các trang thuộc danh mục hệ thống (‘catalog:concepts‘ hoặc ‘catalog:kbs‘).
3. **Phân vùng:**
   - ‘KbCatalog‘: Quản lý danh sách các cơ sở tri thức toàn cục.
   - ‘ConceptCatalog‘: Quản lý lược đồ tri thức riêng cho từng KB.

7.5.4 Cơ chế Phục hồi Kiểu dữ liệu (Hydration & Casting)

Vì tầng lưu trữ nhị phân xử lý dữ liệu thô (Raw bytes/Strings) để đảm bảo tốc độ, việc khôi phục các kiểu dữ liệu logic (Strong Typing) được thực hiện ở tầng ‘V3DataRouter‘:

1. **Giải nén:** Đọc ‘Tuple‘ từ trang ‘SlottedPage‘, tách các trường dựa trên bảng Offset.
2. **Khớp dữ liệu:** Lấy danh sách tên biến từ Trường 1 (Schema).
3. **Ép kiểu (Late-Binding Casting):** Đối chiếu với định nghĩa của ‘Concept‘ hiện tại để chuyển đổi chuỗi sang kiểu dữ liệu đích:
   - ‘INT‘/‘LONG‘ → ‘long.Parse()‘
   - ‘FLOAT‘/‘DOUBLE‘ → ‘double.Parse()‘
   - ‘BOOLEAN‘ → ‘bool.Parse()‘
   - ‘DECIMAL‘ → ‘decimal.Parse()‘
     (a) **Xử lý tiến hóa:** Nếu Concept có thêm biến mới chưa có trong dữ liệu cũ, giá trị ‘null‘ sẽ được gán tự động để bảo toàn logic suy diễn.



Hình 7.6: Luồng dữ liệu qua các tầng xử lý từ Model đến Disk.

7.6 Độ bền & Nhật ký ghi trước (Durability & WAL)

Đảm bảo dữ liệu không bị mất mát khi hệ thống bị ngắt điện đột ngột là nhiệm vụ của ‘WalManager‘. Thành phần này thực hiện các giao thức để đảm bảo tính **Durability** (Độ bền vững) trong mô hình ACID.

7.6.1 Giao thức Write-Ahead Logging (WAL)

KBMS tuân thủ nguyên tắc vàng của hệ quản trị cơ sở dữ liệu: **Ghi Nhật ký trước khi ghi Dữ liệu**.

1. **Log First:** Mọi thay đổi (Insert, Update, Delete) được mô tả thành một đối tượng ‘LogRecord‘ và ghi vào tệp ‘.log‘ trên đĩa trước khi dữ liệu thực tế được cập nhật vào Buffer Pool.
2. **Sequential I/O:** Việc ghi Log là thao tác ghi tuần tự (Sequential), giúp tận dụng tối đa băng thông của ổ cứng mà không bị trễ do tìm kiếm đầu đọc (Disk Seek).
3. **Atomic Write:** Một giao dịch chỉ được coi là thành công khi ‘FileStream.Flush(true)‘ xác nhận dữ liệu Log đã nằm an toàn trên bề mặt vật lý của đĩa.

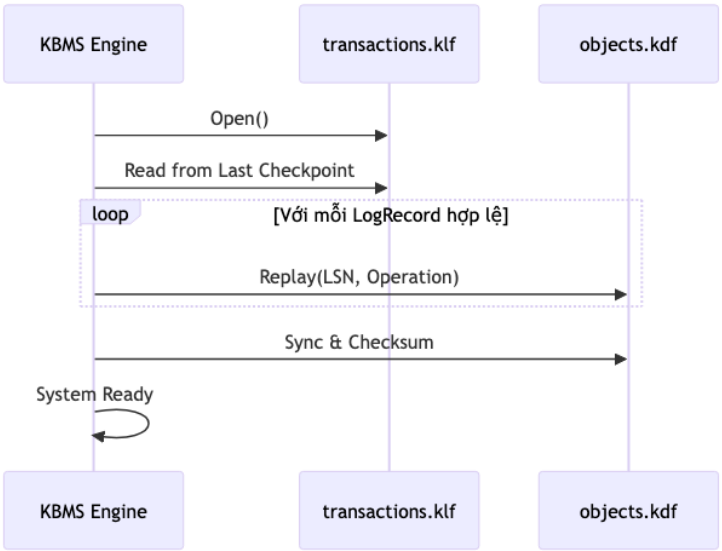
7.6.2 Số tuần tự nhật ký (LSN - Log Sequence Number)

LSN là ”dấu vân tay thời gian”của mọi thay đổi trong hệ thống:

- Mỗi bản ghi Log có một LSN tăng dần duy nhất.
- Mỗi trang dữ liệu (‘Page‘) lưu trữ LSN của thao tác ghi cuối cùng tác động lên nó (tại Offset 4-7 trong Header).
- **Quy tắc an toàn:** Một trang dữ liệu chỉ được phép ghi xuống đĩa (Flush) nếu LSN của trang đó nhỏ hơn hoặc bằng LSN lớn nhất đã được ghi thành công vào file Log.

7.6.3 Quy trình Phục hồi sau sự cố (Recovery)

Khi KBMS khởi động lại sau một vụ sập nguồn đột ngột (Crash), nó thực hiện quy trình **Forward Recovery**:



Hình 7.7: Cơ chế tái hiện (Redo) dữ liệu từ nhật ký WAL để khôi phục trạng thái nhất quán.

1. **Phân tích (Analysis):** Xác định vị trí bản ghi Log cuối cùng và điểm kiểm tra (Checkpoint) gần nhất.
2. **Tái hiện (Redo Phase):** Hệ thống đọc file Log và thực hiện lại các thao tác nếu ‘LSN_Log > LSN_Page‘. Điều này đảm bảo mọi giao dịch đã được xác nhận (Committed) nhưng chưa kịp ghi xuống file ‘.dat‘ sẽ được phục hồi đầy đủ.
3. **Nhất quán:** Sau khi hoàn tất, hệ thống sẵn sàng phục vụ các truy vấn mới.

7.6.4 Cơ chế Checkpoint

Để ngăn file Log phình to vô hạn, định kỳ KBMS thực hiện **Checkpoint**:

- Ghi toàn bộ các trang ”Dirty”từ Buffer Pool xuống đĩa.
- Đánh dấu vị trí an toàn trong file Log.
- Xóa bỏ các bản ghi Log cũ đã được lưu trữ an toàn trên file dữ liệu chính, giúp giải phóng không gian lưu trữ.

7.7 Quản lý Vật lý & Mã hóa (Physical Disk Manager)

Tầng thấp nhất của Storage Engine chịu trách nhiệm giao tiếp trực tiếp với hệ điều hành và bảo vệ dữ liệu tĩnh (Data-at-Rest) thông qua các thuật toán mã hóa tiên tiến.

7.7.1 Ánh xạ Trang vật lý (Physical Block Mapping)

KBMS V3 không lưu trữ dữ liệu dưới dạng luồng liên tục mà chia thành các khối cố định (Blocks) trên đĩa.

- **Page ID to Offset:** Mọi yêu cầu truy xuất trang ‘PageId‘ được ‘DiskManager‘ chuyển đổi thành vị trí chính xác trong file thông qua công thức:

$$Offset = PageId \times 16416 \tag{7.1}$$

- **Truy cập ngẫu nhiên ($O(1)$):** Sử dụng phương thức ‘FileStream.Seek‘ để nhảy trực tiếp đến khối dữ liệu cần thiết mà không phải quét toàn bộ tệp, đảm bảo hiệu năng ổn định ngay cả khi tệp cơ sở tri thức lên đến hàng trăm Gigabytes.

7.7.2 Mã hóa dữ liệu tĩnh (AES-256 Encryption)

Mọi dữ liệu trong KBMS V3 đều được mã hóa ở mức trang (Page-level encryption) trước khi được ghi xuống đĩa cứng để ngăn chặn việc đánh cắp dữ liệu vật lý.

- **Thuật toán:** AES-256 (Advanced Encryption Standard).
- **Quy trình ghi:**
  1. Dữ liệu 16KB được lấy từ Buffer Pool.
  2. Mã hóa bằng ‘MasterKey‘ của KB.
  3. Thêm 32 bytes IV (Initialization Vector) và Padding.
  4. Ghi khối **16,416 bytes** xuống đĩa.
     - **Quy trình đọc:** Thực hiện ngược lại để giải mã và đưa dữ liệu thô vào RAM.

Thành phần	Kích thước (Bytes)	Ghi chú
Dữ liệu logic	16,384	16KB dữ liệu thật (đã bao gồm 24B Header).
Phần bù/IV	32	Phần bù mã hóa và vector khởi tạo.
Khối vật lý	**16,416**	**Kích thước thực tế chiếm dụng trên đĩa**.

Bảng 7.4: Bảng 13

7.7.3 Quản lý tệp tin và Cấp phát (Allocation)

- **Cấp phát trước (Pre-allocation):** Khi một trang mới được yêu cầu, ‘DiskManager‘ thực hiện ghi đè các bytes ‘0x00‘ (kỹ thuật Zero-fill) để giữ chỗ trên đĩa, giúp giảm thiểu hiện tượng phân mảnh tệp (Fragmentation) ở cấp độ hệ điều hành.
- **Xác nhận đĩa (Persistence):** Các thay đổi quan trọng luôn đi kèm lệnh ‘Flush‘ để ép hệ điều hành ghi trực tiếp xuống phiến đĩa vật lý, bỏ qua các tầng cache của OS để tránh mất dữ liệu khi mất điện đột ngột.

7.8 Thực tế hóa: Từ Mô hình đối tượng sang Chuỗi nhi phân

Để kết nối lý thuyết mô hình (Chương 5) và hạ tầng lưu trữ (Chương 7), chúng ta sẽ theo dõi lộ trình của một đối tượng cụ thể đi qua các tầng xử lý của KBMS.

7.8.1 Đối tượng đầu vào (C# Layer)

Giả sử người dùng nhập vào một thực thể của khái niệm ‘TamGiac‘ (đã định nghĩa ở mục 5.7):

```
var t1 = new ObjectInstance {
    Id = new Guid("550e8400-e29b-41d4-a716-446655440000"),
    ConceptName = "TamGiac",
    Values = new Dictionary<string, object> {
        { "a", 3 }, { "b", 4 }, { "c", 5 }, { "p", 6 }, { "S", 6 }
    }
};
```

7.8.2 Tuần tự hóa thành Tuple (Serialization)

Khi thực hiện lệnh ‘INSERT‘, ‘V3DataRouter‘ sẽ chuyển đổi đối tượng trên thành khối nhị phân ‘Tuple‘ với cấu trúc sau:

Field	Nội dung	Kiểu	Kích thước
0	‘550e8400-e29b-41d4-a716-446655440000‘	GUID	16 Bytes
1	‘a	b	c
2	‘3‘	UTF-8 String	1 Byte
3	‘4‘	UTF-8 String	1 Byte
4	‘5‘	UTF-8 String	1 Byte
5	‘6‘	UTF-8 String	1 Byte
6	‘6‘	UTF-8 String	1 Byte

Bảng 7.5: Bảng 14

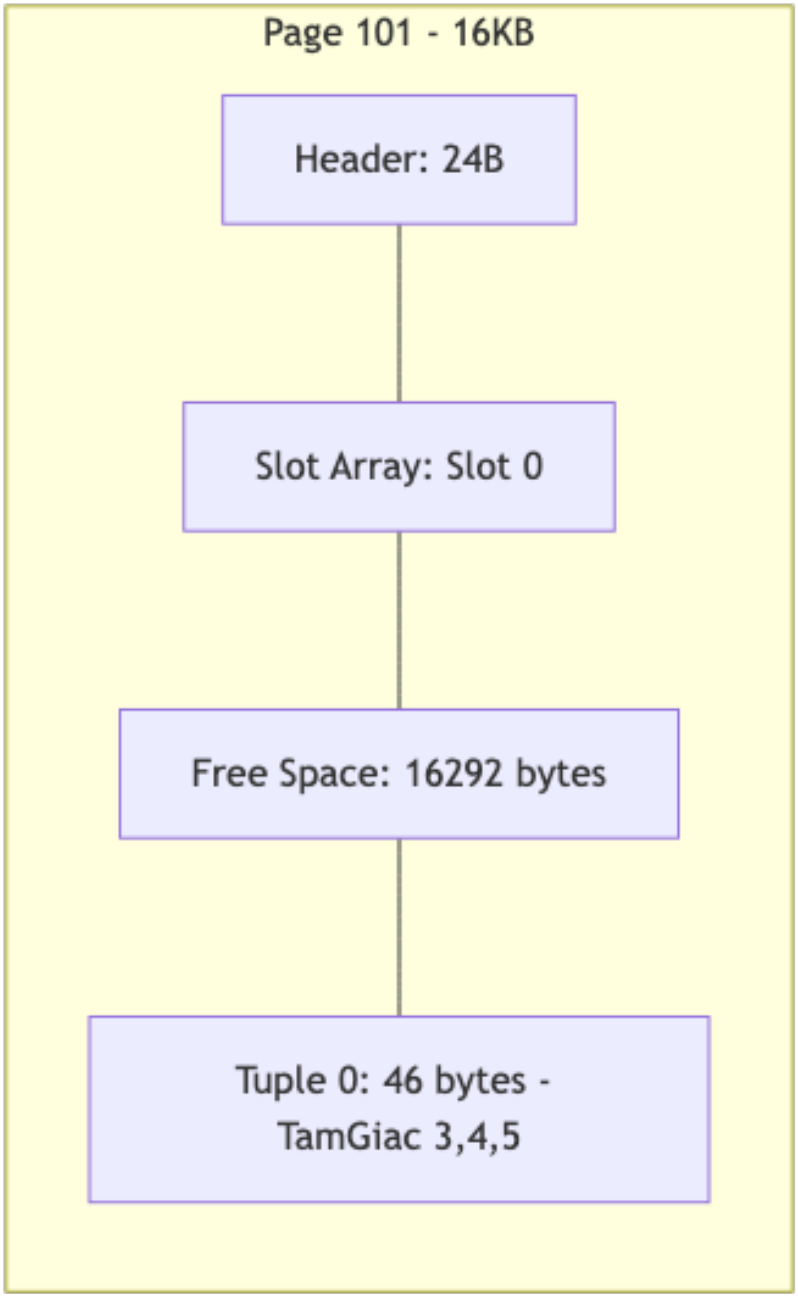
Tính toán kích thước Tuple:

- **Header:** 2B (FieldCount) + $7 \times 2B$ (Offsets) = **16 Bytes**.
- **Payload:** $16 + 9 + 1 + 1 + 1 + 1 + 1$ = **30 Bytes**.
- **Tổng cộng:** $16 + 30 =$ **46 Bytes**.

7.8.3 Bản đồ vị trí trong Trang (Slotted Page Mapping)

Giả sử ‘Tuple‘ này được chèn vào một trang trống (‘PageId=101‘).

1. **Header (24B):** Cập nhật ‘TupleCount = 1‘, ‘FreeSpacePointer = 16338‘ ($16384 - 46$).
2. **Slot Array:** Slot đầu tiên (Index 0) chứa giá trị ‘[Offset: 16338, Length: 46]‘.
3. **Data Area:** 46 bytes dữ liệu nằm ở cuối trang (từ vị trí 16338 đến 16383).



Hình 7.8: Minh họa vị trí thực tế của đối tượng TamGiac trong bộ nhớ nhị phân.

7.8.4 Biểu diễn dưới dạng mã Hex (Hex Dump Trace)

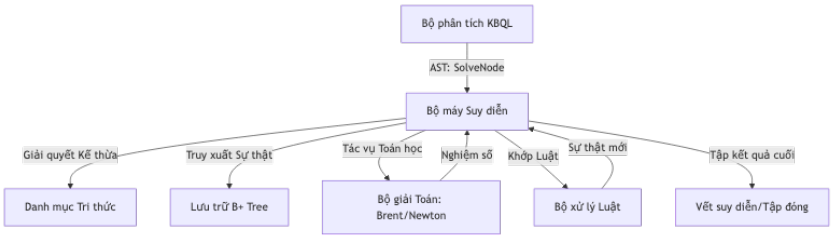
Dưới đây là mô phỏng 64 bytes đầu tiên của trang dữ liệu trên đĩa (đã giải mã AES):

```
Offset    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F    Decoded
-------------------------------------------------------------------------
00000000  65 00 00 00 01 00 00 00 FF FF FF FF FF FF FF FF    e...............
          [ PageId: 101 ] [ LSN: 1  ] [ PrevPageId: -1  ]
00000010  FF FF FF FF D2 3F 00 00 01 00 00 00 D2 3F 00 00    .....?.......?..
          [ NextPage: -1] [ FSP:16338] [ Count: 1 ] [Slot0: Off=16338, Len=46]
```

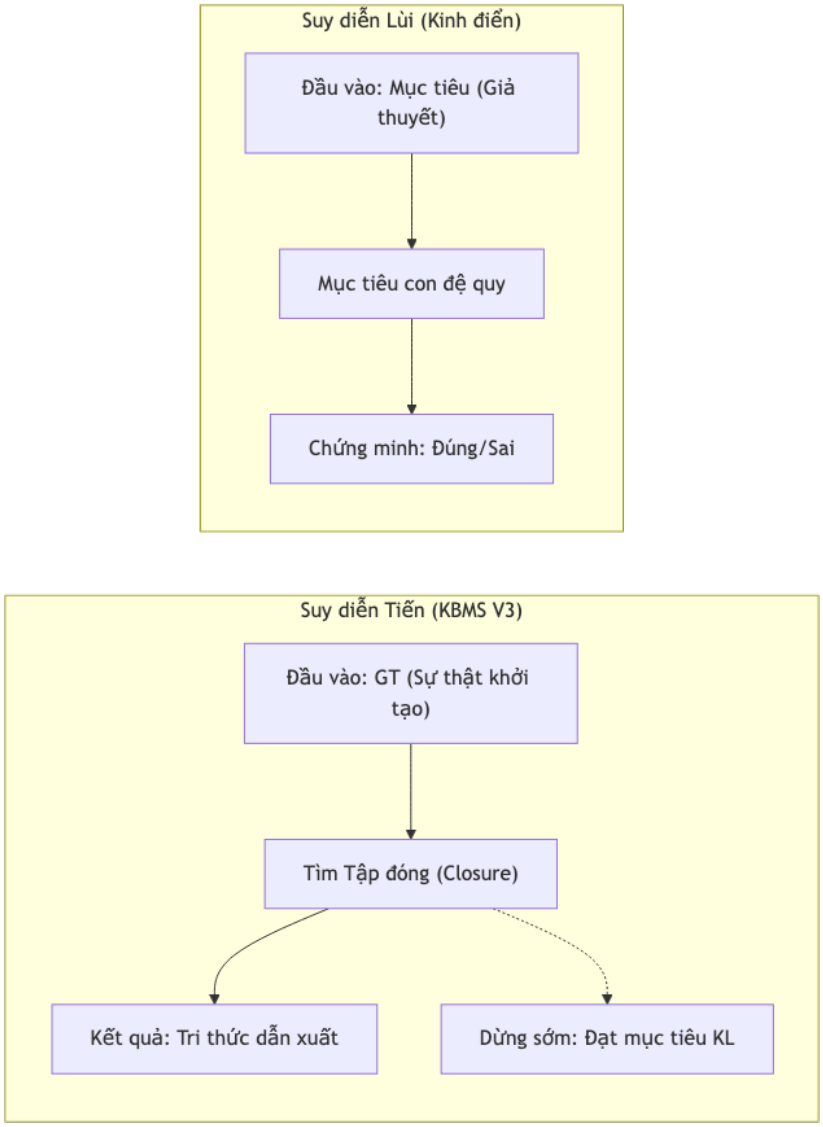
CHƯƠNG 8: Bộ máy Suy diễn và Thuật toán Bao đóng

8.0.1 Kiến trúc Điều phối Reasoning

Bộ máy suy diễn không hoạt động độc lập mà đóng vai trò là ”bộ não”trung tâm điều phối giữa ngôn ngữ truy vấn và dữ liệu vật lý.



Hình 8.1: Sơ đồ điều phối giữa Engine suy diễn và các phân hệ ngoại vi.



Hình 8.2: So sánh chiến lược Suy diễn tiến (Forward) và Suy diễn lùi (Backward).

8.0.2 Nguyên lý Đóng kín Tri thức (Knowledge Closure)

Trực giác của KBMS dựa trên khái niệm **Tập đóng (Closure)**. Cho một tập hợp các sự thật đầu vào (**GT**) và một tập hợp các luật/phương trình (**Rules**), mục tiêu là tìm ra tập hợp lớn nhất các sự thật có thể được suy ra một cách logic.

$$Closure(GT, Rules) = GT \cup \{\text{Kết luận từ các Luật và Phương trình được kích hoạt}\} \tag{8.1}$$

Hệ thống sẽ lặp lại quá trình suy diễn cho đến khi đạt được trạng thái **Fixed-point** (điểm dừng) - nơi việc tiếp tục suy diễn không sinh ra thêm bất kỳ sự thật mới nào [6]. Việc tìm tập đóng này dựa trên thuật toán F-Closure tối ưu cho các vật thể tính toán [1].

8.0.3 Biểu diễn Tri thức (Knowledge Representation)

Trong KBMS, tri thức được biểu diễn dưới dạng các đối tượng (Concepts) và các ràng buộc (Constraints) giữa chúng.

Các thành phần của một Concept

- **Variables (Biến):** Các thuộc tính của đối tượng (ví dụ: ‘Price‘, ‘Quantity‘).
- **Rules (Luật):** Logic ‘IF-THEN‘ để suy diễn ra giá trị hoặc sự kiện mới.

- **Equations (Phương trình):** Các công thức toán học mô tả mối quan hệ giữa các biến.
- **Constraints (Ràng buộc):** Các điều kiện mà dữ liệu buộc phải thỏa mãn.
- **Relations (Quan hệ):** Mối liên kết giữa các Concept (IS-A, PART-OF, ...).

8.0.4 Quy trình Suy diễn (Reasoning Lifecycle)

Khi một bản ghi dữ liệu (Fact) được đưa vào hệ thống, bộ máy suy diễn thực hiện các bước sau:

1. **Preprocessing (Tiền xử lý):** Nạp toàn bộ metadata của Concept, bao gồm các luật kế thừa từ IS-A và Part-Of.
2. **Initialization (Khởi tạo):** Đưa các Fact đầu vào vào tập **GT (Ground Truth)**.
3. **Cyclic Evaluation (Đánh giá vòng lặp):** Hệ thống liên tục quét qua tập hợp các Rules và Equations cho đến khi không còn Fact nào mới được sinh ra.
4. **Verification (Kiểm chứng):** Sau khi suy diễn xong, hệ thống kiểm tra lại toàn bộ các ‘Constraints‘ để đảm bảo kết quả suy diễn không vi phạm bất kỳ quy tắc nào.

8.0.5 Chiến lược Suy diễn: Forward vs. Backward Chaining

Trong lý thuyết Hệ chuyên gia, có hai chiến lược chính:

- **Forward Chaining (Suy diễn tiến)**: Từ sự thật (**GT**) suy ra kết luận. Thích hợp cho việc ”khám phá”mọi tri thức có thể từ một tập dữ liệu nhỏ.
- **Backward Chaining (Suy diễn lùi)**: Từ mục tiêu (**KL**) tìm ngược lại xem cần những sự thật nào. Thích hợp cho việc ”kiểm chứng”giả thuyết trong một cơ sở tri thức khổng lồ.

Tại sao KBMS V3 chọn Forward Chaining?

Hệ thống KBMS hiện đại được tối ưu hóa cho các **Mô hình Vật thể Tính toán (Computational Objects)**. Trong các mô hình này (như Hình học hay Tài chính), các biến số và công thức có mối quan hệ ràng buộc chặt chẽ và cục bộ. Việc sử dụng **Goal-Directed Forward Chaining** mang lại các ưu điểm:

1. **Hiệu năng**: Tìm tập đóng (Closure) nhanh hơn việc duyệt cây mục tiêu đệ quy (SLD-resolution) trong các hệ thống chứa nhiều phương trình toán học.
2. **Tính đầy đủ**: Đảm bảo mọi hệ quả của một thay đổi dữ liệu đều được cập nhật vào GT.
3. **Dừng sớm (Early Exit)**: Mặc dù là suy diễn tiến, Server sẽ dừng ngay lập tức khi tất cả các biến mục tiêu trong danh sách **KL** đã được tìm thấy (Dòng code 378).

Giới hạn An toàn (Safety Limit)

Để ngăn chặn các vòng lặp vô hạn trong trường hợp tri thức bị mâu thuẫn hoặc đệ quy quá sâu, bộ máy suy diễn áp dụng giới hạn: **Tối đa 50 vòng lặp (Iterations)**. Nếu sau 50 lần quét mà hệ thống vẫn sinh ra Fact mới, nó sẽ tự động dừng và báo cáo trạng thái Closure hiện tại (Dòng code 93).

IS-A (Kế thừa)

- **Ý tưởng:** Nếu A **IS-A** B, thì mọi biến và luật của B sẽ được ”truyền”sang A.
- **Ví dụ:** ‘Square‘ IS-A ‘Rectangle‘. Khi tính diện tích cho ‘Square‘, hệ thống có thể dùng luôn luật ‘Area = Width * Height‘ của ‘Rectangle‘.

PART-OF (Thành phần)

- **Ý tưởng:** Mô tả cấu trúc phân rã của đối tượng. Một đối tượng lớn được tạo từ nhiều đối tượng con.
- **Ví dụ:** ‘Car‘ PART-OF ‘Engine‘, ‘Wheel‘. Hệ thống tự động ánh xạ các biến từ các phần con vào đối tượng cha để suy diễn tổng thể.

CONSTRUCT_RELATIONS (Quy tắc quan hệ)

- **Ý tưởng:** Định nghĩa các quy luật chung cho một mối quan hệ cho trước (ví dụ: Quan hệ ‘CongRuence‘ - Bằng nhau trong hình học).
- **Ứng dụng:** Khi hai đối tượng thỏa mãn quan hệ này, các phương trình tương ứng (như ‘Object1.Size = Object2.Size‘) sẽ được tự động tiêm vào hệ thống để giải.

8.0.6 Giải trình và Truy vết (Traceability)

Bộ máy suy diễn KBMS không phải là một ”Black Box”(Hộp đen). Nó cung cấp khả năng giải thích lý do tại sao một giá trị được tạo ra.

- **Derivation Trace:** Lưu trữ nguồn gốc (Source) của mỗi biến được suy diễn, bao gồm cả các biến đầu vào được dùng tại bước đó.
- **Cơ chế:** Mỗi lần một Rule hoặc Equation kích hoạt thành công, một bản ghi Trace sẽ được đẩy vào kết quả truy vấn.

8.0.7 Kế hoạch Triển khai Mã nguồn Engine (Implementation Strategy)

Để hiện thực hóa bản thiết kế kỹ thuật trên thành phân hệ phần mềm hoạt động thực tế, lộ trình triển khai (Backend C#) được chia làm 4 giai đoạn chính (Phases):

Giai đoạn 1: Tiền xử lý & Xây dựng Cấu trúc Tri thức (Preprocessing)

Xây dựng lớp Abstract Syntax Tree (AST) tiếp nhận từ bộ phân tích (Parser). Ở giai đoạn này, dữ liệu đầu vào (Sự thật - Fact) được ”Làm phẳng”(Flattening) bằng cách quét chuỗi kế thừa ‘IS-A‘ và ‘PART-OF‘ nhằm lôi kéo tất cả Metadata cần thiết sang đối tượng con. Quá trình tiền xử lý biến danh mục tri thức thô thành một đối tượng sẵn sàng chạy (Executable Concept).

Giai đoạn 2: Lõi Suy diễn Lan truyền (Deductive Core & Propagation)

Xây dựng vòng lặp chính của cơ chế **Goal-Directed Forward Chaining** (FClosure).

- Cài đặt luồng nhận diện chuỗi biến tương đương và sao chép/lan truyền giá trị tức thời (**SameVariables Propagation**).
- Chạy luồng khớp mệnh đề IF-THEN (**Rule Matching**) để đẩy liên tiếp kết luận logic vào Tập Sự Thật (Ground Truth).

Giai đoạn 3: Tích hợp Lõi giải Toán học (Mathematical Solvers)

Phát triển các Adapter kết nối lớp thuật toán tính toán với hệ thống:

- Adapter Phương trình 1 ẩn (**Brent’s Method**): Quét qua các biểu thức còn 1 biến và tiến hành tính toán hội tụ nghiệm số tự động.
- Adapter Hệ phương trình (**Newton-Raphson 2D**): Nhận diện cụm 2 phương trình chung 2 ẩn, định cấu hình ma trận Jacobian và cập nhật vòng lặp đệ quy.

Giai đoạn 4: Bảo vệ Hệ thống & Truy vết (Safety Constraints & Traceability)

Đảm bảo Engine chạy độc lập không gây vắt kiệt RAM máy chủ bằng cách cài giới hạn an toàn $N = 50$ **vòng lặp tối đa (Iterations)**. Nếu sau 50 chu kỳ duyệt hệ thống vẫn chưa trả về điểm đóng (Closure), thuật toán tự ngắt kết nối. Song song đó, viết bộ thu thập nhật ký truy xuất (**Trace System**) để xuất định dạng dữ liệu cây giải trình thành dạng JSON chuyển qua cho Front-End UI hiển thị.

8.0.8 Các Thuật toán Suy diễn Cốt lõi (Forward Chaining - FClosure)

Thuật toán này là xương sống của KBMS, thực hiện việc mở rộng tập tri thức GT cho đến khi đạt điểm đóng (Closure).

Các Giai đoạn Thực thi (Phases)

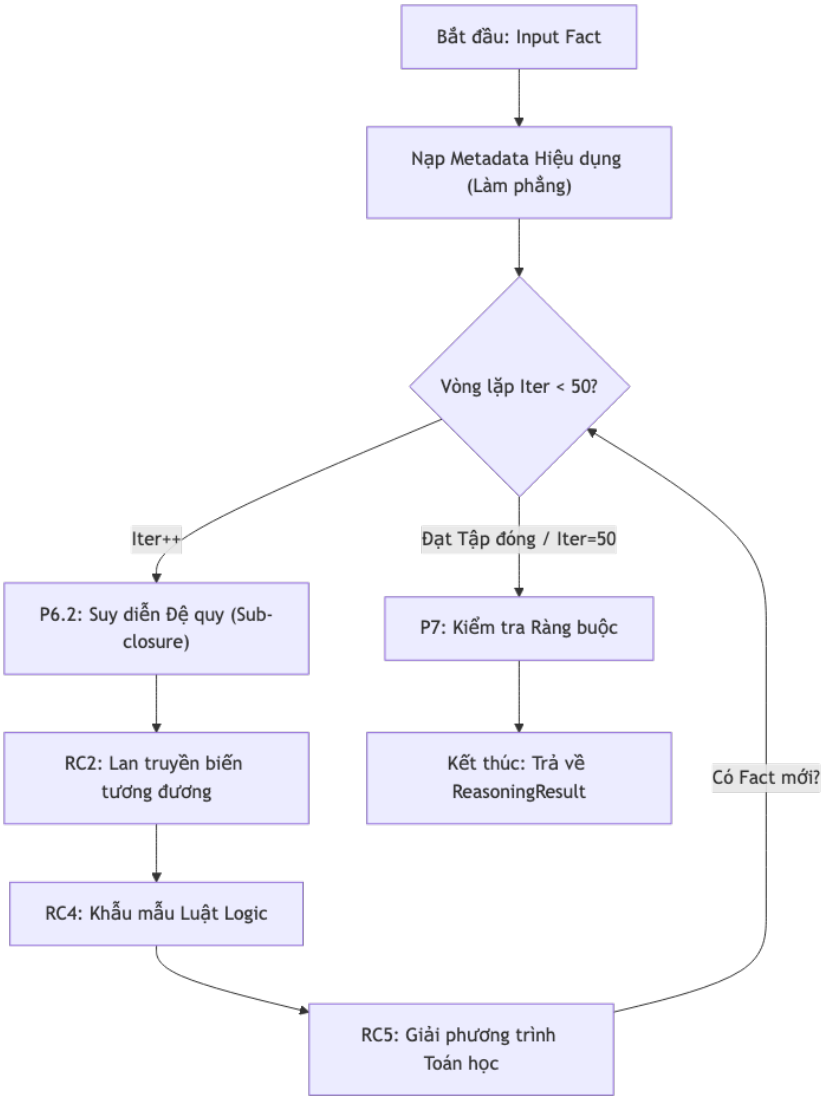
Bên trong vòng lặp chính, bộ máy thực hiện các bước sau theo thứ tự ưu tiên:

1. **Recursive Sub-closure (Phase 6.2)**: Duyệt qua các biến có kiểu dữ liệu là một Concept khác. Hệ thống sẽ tự động gọi hồi quy ‘FindClosure‘ cho các đối tượng con này để đồng bộ tri thức từ dưới lên.
2. **SameVariables Propagation (RC2)**: Lan truyền giá trị giữa các cặp biến được đánh dấu tương đương. Ví dụ: ‘a = b‘ thì nếu biết ‘a‘, hệ thống tự động gán giá trị cho ‘b‘.
3. **Computation Relations (RC3)**: Thực thi các quan hệ tính toán trực tiếp (**Rf**) từ biểu thức.
4. **Logic Rules (RC4)**: Khớp mẫu các luật ‘IF-THEN‘. Nếu mẫu khớp, các kết luận mới được đẩy vào GT.
5. **Equation Solving (RC5)**: Giải các phương trình toán học và hệ phương trình để tìm các ẩn số mới.

Ví dụ Chạy khô (FClosure Dry Run)

(Bảng ví dụ Triangle đã được cung cấp ở phần trước)

Sơ đồ Luồng FClosure (Chi tiết Server-side)



Hình 8.3: 1. Sơ đồ Luồng FClosure

8.0.9 Giải phương trình 1D (Brent's Method - RC5)

KBMS sử dụng thuật toán Brent để tìm nghiệm của phương trình $f(x) = 0$ trong một khoảng cô lập.

Độ hội tụ (Convergence)

Thuật toán kết hợp giữa Bisection (đảm bảo hội tụ) và Secant/Inverse Quadratic Interpolation (đảm bảo tốc độ).

- **Tốc độ**: Hội tụ siêu tuyến tính với bậc $\phi \approx 1.618$.
- **Ưu điểm**: Không yêu cầu tính đạo hàm $f'(x)$.

8.0.10 Giải hệ phương trình 2D (Newton-Raphson - RC5)

Được kích hoạt khi phát hiện 2 phương trình có chung 2 biến chưa biết (Dòng code 341-374).

Ma trận Jacobian & Công thức cập nhật

Hệ thống tính toán ma trận Jacobian **J** bằng phương pháp sai phân hữu hạn (Dòng code 573):

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix} \tag{8.2}$$

Sau đó cập nhật giá trị theo công thức (Dòng code 579-583):

$$\Delta \mathbf{x} = -\mathbf{J}^{-1}\mathbf{F}(\mathbf{x}) \tag{8.3}$$

8.0.11 Phân tích Độ phức tạp (Complexity Analysis)

Thuật toán	Mục tiêu	Độ phức tạp	Lý do kỹ thuật (Server-side)
FClosure	Suy diễn tổng thể	$O(50 \times (R + E + SV))$	Giới hạn $N = 50$ vòng lặp tối đa. R: số luật, E: số phương trình, SV: số cặp biến tương đương.
Newton-Raphson	Giải hệ 2D	$O(50 \times 4 \times eval)$	Newton lặp max 50 lần; mỗi lần tính 4 đạo hàm riêng lẻ ($eval$).
Brent's Method	Giải PT 1D	$O(1/\phi^n)$	Hội tụ siêu tuyến tính, số bước lặp n thường nhỏ (< 20).
Flattening	Gộp tri thức	$O(H \times V)$	H: chiều sâu kế thừa, V: tổng số biến trong hệ thống.
Trace System	Giải thích	$O(D)$	D: Độ sâu vết suy diễn (Reasoning Depth).

Bảng 8.1: Bảng ký hiệu thuật toán F-Closure

8.0.12 Nhật ký Suy diễn (Reasoning Trace Example)

Khi bạn thực hiện lệnh 'SOLVE', KBMS trả về một bản giải trình chi tiết:

```
{
  "closure": { "a": 3, "b": 4, "c": 5, "S": 6 },
  "traces": [
    { "target": "c", "source": "CosineLaw", "inputs": ["a", "b", "angleC"], "value": 5.0 },
    { "target": "p", "source": "PerimeterFormula", "inputs": ["a", "b", "c"], "value": 6.0 },
    { "target": "S", "source": "HeronFormula", "inputs": ["p", "a", "b", "c"], "value": 6.0 }
```

```
  ]
}
```

8.1 Xác thực Suy diễn (Reasoning Validation)

Bộ máy suy diễn là trung tâm trí tuệ của KBMS, do đó tính chính xác tuyệt đối của thuật toán F-Closure là bắt buộc.

8.1.1 Kiểm thử Thuật toán F-Closure

Hệ thống sử dụng tệp **'Phase5ForwardChainingTests.cs'** để mô phỏng các kịch bản suy diễn phức tạp:

Kịch bản	Dữ liệu đầu vào	Kết quả mong đợi
Simple Rule	'Emp.salary = 100'	'salary = 110' (Nếu bonus = 0.1)
Multi-step	'Update Rule R1' -> 'Triggers R2'	Điểm đóng F-Closure cuối cùng.
Recursive	Luật đệ quy lồng nhau.	Dừng đúng lúc tại điểm đóng.

Bảng 8.2: Tổng hợp kết quả kiểm thử bộ máy suy diễn

Minh chứng Mã nguồn (F-Closure):

Thiếu ảnh: code_test_f_closure.png

Hình 8.4: Kịch bản kiểm thử thuật toán F-Closure nâng cao.

Minh chứng Kết quả (Log Trace):

Thiếu ảnh: result_test_f_closure.png

Hình 8.5: Kết quả tìm tập đóng và vết suy diễn tương ứng.

8.1.2 Kiểm thử Logic Toán học (Solver Testing)

Xác thực tính năng giải hệ phương trình thông qua 'Newton-Raphson' và 'Brent Solver'.

Thiếu ảnh: placeholder_solver_iterations_log.png

Hình 8.6: Nhật ký log của Engine khi tìm nghiệm phương trình bậc hai

CHƯƠNG 9: Giao thức Mạng và Truyền tải Dữ liệu

9.1 Giao thức Mạng (Network Protocol)

9.2 Giao thức Truyền dẫn Nhị phân (Binary Protocol)

KBMS sử dụng một giao thức truyền tin nhị phân (Binary Protocol) tùy chỉnh chạy trên nền TCP để đảm bảo tốc độ truyền tải tri thức nhanh nhất và khả năng xử lý đồng thời (concurrency) tốt.

9.2.1 Cấu trúc Gói tin (Packet Structure)

Mỗi thông điệp (Message) gửi đi giữa Client và Server được đóng gói theo định dạng sau:

Trường	Kích thước	Mô tả
Total Length	4 bytes (Int32)	Tổng độ dài các trường tính từ *SessLen* trở đi (bỏ qua MessageType).
Message Type	1 byte	Loại thông điệp (Query, Result, Error, ...).
SessLen	2 bytes (UInt16)	Độ dài của chuỗi Session ID.
Session ID	N bytes (UTF-8)	Định danh phiên làm việc của người dùng.
ReqLen	2 bytes (UInt16)	Độ dài của chuỗi Request ID.
Request ID	M bytes (UTF-8)	Định danh duy nhất cho mỗi yêu cầu truy vấn.
Payload	P bytes (UTF-8)	Nội dung thực tế (ví dụ: câu lệnh KBQL hoặc kết quả JSON).

Bảng 9.1: Đặc tả cấu trúc gói tin nhị phân KBMS

9.2.2 Các loại Thông điệp (Message Types)

Máy chủ được trang bị **12 Giao thức Chuẩn** bao quát mọi tình huống Truy vấn, Phân mảnh dữ liệu, Bảo mật và Telemetry Hệ thống (Thiết kế gốc từ ‘MessageType.cs‘):

1. **Bảo mật & Xác thực (Security Auth):**
   - ‘LOGIN (0x01)‘: Xin cấp quyền phiên làm việc bằng thông tin (User/Pass).
   - ‘LOGOUT (0x05)‘: Đóng kết nối định danh và trả phiên tài nguyên hệ thống.
1. **Truy vấn (Query Routing):**
   - ‘QUERY (0x02)‘: Chở luồng văn bản gốc (Chuỗi lệnh KBQL).
   - ‘RESULT (0x03)‘: Gói kết quả suy diễn ở dạng tổng quát (JSON String) .
   - ‘ERROR (0x04)‘: Payload chứa tọa độ ‘Line‘, ‘Column‘ và ‘Stacktrace‘ dùng để tô đỏ biên dịch viên của Designer.
1. **Kỹ thuật Trả Lô Dữ Liệu Lớn (Tabular Data Streaming - TDS)** KBMS xử lý Hàng nghìn Fact mà không làm treo RAM Client bằng luồng C# phân đoạn kinh điển:
   - ‘METADATA (0x06)‘: Truyền trước sơ đồ lưới dữ liệu (Tên Field, Bề rộng UI Column).

- ‘ROW (0x07)‘: Trình bày Payload theo chuỗi vòng lặp ”Lô”(Ví dụ: Batch 100 Rows/Lần gói) gửi liên tục qua luồng Stream TCP.
- ‘FETCH_DONE (0x08)‘: Cờ khóa hạ màn chặn chuỗi vòng lặp đọc. Tổng hợp độ trễ thực thi CSDL (Execution Time - ms).

1. **Hệ Quản trị Trực tiếp (Telemetry System)** Dành riêng cho các Client mang quyền hệ thống cấp cao (ROOT Role):
   - ‘STATS (0x0A / 10)‘: Theo dõi Metrics máy chủ thời gian thực.
   - ‘LOGS_STREAM (0x0B / 11)‘: Đăng ký lắng nghe Live Log Socket đang chảy của Hệ thống.
   - ‘SESSIONS (0x0C / 12)‘: Trích xuất danh bạ quản lý Users Online, kiểm soát giới hạn Concurrent Connections.
   - ‘MANAGEMENT_CMD (0x0D / 13)‘: Đỉnh cao phân quyền - Dùng để chạy lệnh thao tác cấp hệ điều hành (Chỉnh sửa Setting, Kill Session, Cấp Role).

9.2.3 Thuật toán Xử lý Luồng tin (Streaming Algorithm)

Để tránh hiện tượng treo ứng dụng khi truyền tải lượng tri thức lớn, KBMS áp dụng cơ chế **Asynchronous Packet Processing**:

1. **Header Parsing:** Đọc 4 byte đầu tiên để xác định kích thước gói tin cần nhận.
2. **Buffer Allocation:** Cấp phát vùng nhớ vừa đủ cho gói tin đó.
3. **Exact Reading:** Sử dụng hàm ‘ReadExact‘ để đảm bảo nhận đủ dữ liệu trước khi giải mã (Decode).
4. **Payload Dispatching:** Chuyển nội dung UTF-8 vào bộ phân tích cú pháp (Parser) hoặc hiển thị lên UI.

9.2.4 Bảo mật và Kết nối

- **TCP Keep-Alive:** Duy trì kết nối ổn định giữa KBMS Studio và Server.
- **Session Management:** Cho phép người dùng thực hiện nhiều truy vấn đồng thời trên cùng một kết nối mà không bị lẫn lộn dữ liệu (nhờ vào Request ID).
- **Error Propagation:** Khi Server gặp lỗi suy diễn, nó sẽ đóng gói toàn bộ Stack Trace và vị trí lỗi vào một ‘Error Message‘ đặc biệt để Client hiển thị nháy đỏ trên trình soạn thảo bài toán.

9.2.5 Kiến trúc Xử lý Chịu tải (Concurrency Threading)

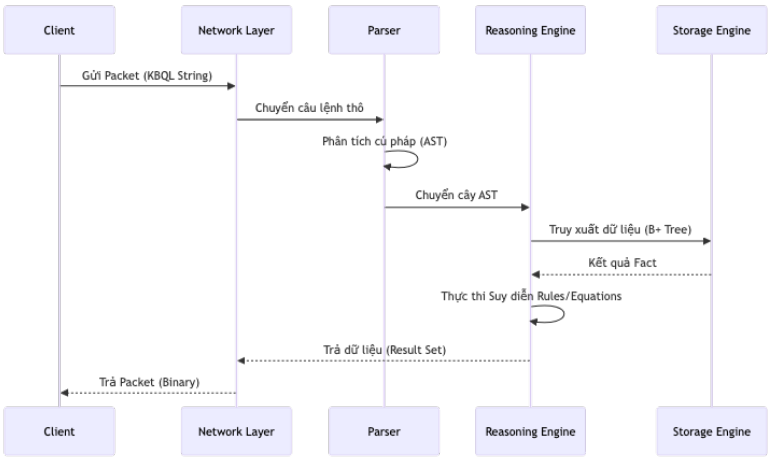
Để duy trì hiệu năng cao phục vụ cho hàng ngàn điểm kết nối TCP, KBMS Server loại bỏ hoàn toàn mô hình 1-Thread-Per-Client truyền thống, thay vào đó áp dụng cấu trúc xử lý Đa luồng Bất đồng bộ (Async/Await) đỉnh cao của .NET Core:

- **Asynchronous I/O Socket:** Cổng nhận (‘AcceptTcpClientAsync‘) sẽ quăng toàn bộ tiến trình phân tích Packet vào ThreadPool để tránh tắt nghẽn luồng lắng nghe chính (‘_ = HandleClientAsync(client)‘).
- **Tránh Va Chạm Thread Mạng:** Thuật toán gói dữ liệu sử dụng cờ báo ‘SemaphoreSlim‘ để khóa luồng ghi byte, tránh trường hợp Response từ hai yêu cầu bất đồng bộ (Async) va chạm và ghi chồng byte lên nhau trên cùng Network Stream.
- **Thread-Safe Sessions:** Việc thu thập và phát tán Request / Log liên quan tới client được lưu trữ tập trung qua lớp ‘ConnectionManager‘ bằng bộ từ điển an toàn luồng ‘ConcurrentDictionary‘.

Mỗi khi người dùng gửi một câu lệnh KBQL từ Client, Server sẽ thực hiện một quy trình xử lý đa tầng để hiểu ý định (Intent) và thực thi đúng các phép toán/suy diễn.

9.2.6 Vòng đời của một Truy vấn (Query Lifecycle)

Dưới đây là sơ đồ tuần tự mô tả các bước từ khi Client gửi câu lệnh đến khi nhận được kết quả cuối cùng:

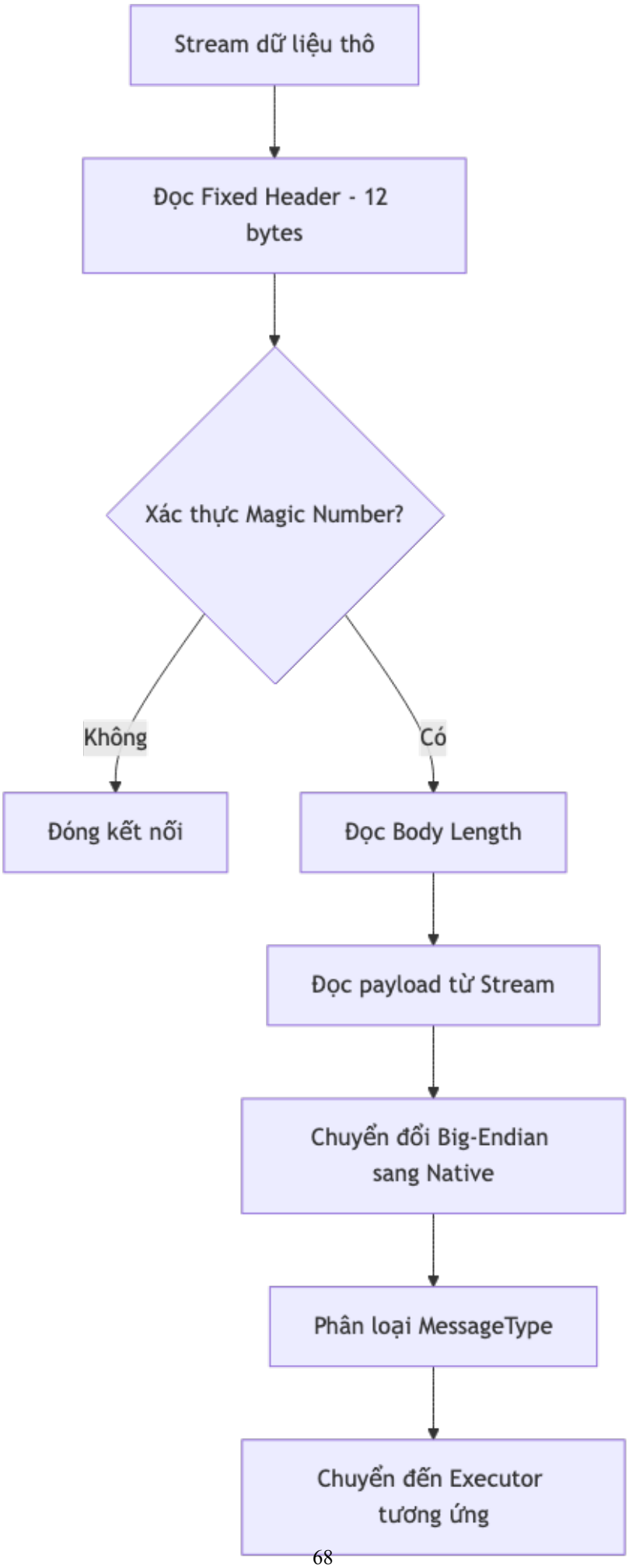


Hình 9.1: Vòng đời xử lý truy vấn xuyên suốt 4 tầng kiến trúc

Hình: Sơ đồ Tuần tự Vòng đời Truy vấn (Query Lifecycle)

9.2.7 Luồng xử lý Packet (Packet Processing Flow)

Hệ thống xử lý các byte dữ liệu thô từ Stream và chuyển đổi chúng thành các cấu trúc thông điệp có ý nghĩa:


Stream dữ liệu thô
Đọc Fixed Header - 12 bytes
Xác thực Magic Number?
Không
Có
Đóng kết nối
Đọc Body Length
Đọc payload từ Stream
Chuyển đổi Big-Endian sang Native
Phân loại MessageType
Chuyển đến Executor tương ứng

*Hình: Sơ đồ# 09.2. Luồng Xử lý Truy vấn Mạng
*

9.2.8 Phân tích Từ vựng (Lexer)

Ý tưởng: Biến chuỗi văn bản (String) thô thành một danh sách các tín hiệu có nghĩa (Tokens).

- **Thuật toán Tokenizer:** Quét chuỗi ký tự, nhận diện các từ khóa (‘SELECT‘, ‘CREATE‘), định danh (tên Concept), toán tử và giá trị số/chuỗi.
- **Báo lỗi:** Nếu gặp ký tự lạ, Tokenizer ngay lập tức trả về lỗi kèm vị trí chính xác (Dòng, Cột).

9.2.9 Phân tích Cú pháp (Parser & AST)

Ý tưởng: Xây dựng cây cú pháp trừu tượng (Abstract Syntax Tree - AST) từ danh sách Tokens.

- **Kỹ thuật:** Sử dụng trình phân tích cú pháp **Recursive Descent** (Phân tích đi xuống đệ quy) để khớp các Tokens với bộ ngữ pháp của KBQL.
- **Cấu trúc AST:** Mỗi câu lệnh được biểu diễn thành một cây các đối tượng. Ví dụ, lệnh ‘SELECT‘ sẽ có các nhánh con là ‘Fields‘, ‘Source‘, ‘WhereCondition‘, và ‘Limit‘.

9.2.10 Thực thi và Cấp phát (Executor & Router)

Sau khi có AST từ ‘KBMS.Parser‘, gói tin bắt đầu chuyển giao cho Logic Cấu trúc lõi:

1. **Chồng Khớp (Binding) & Routing:** Từng lệnh tách lẻ trong chuỗi văn bản được cắt ra và chuyển qua Router ‘_knowledgeManager.Execute(ast, ...)‘ để thực thi tuần tự.
2. **Catalog Check:** Concept được truy vấn có tồn tại không?
3. **Schema & Type Check:** Các biến trong hàm ‘SELECT‘ hoặc ‘CALC()‘ có hợp lệ về kiểu dữ liệu hay không? Mọi sai phạm đều ngay lập tức bị tóm gọn ở lớp ‘KBMS.Parser.ParserException‘, đính kèm vào đó (Line, Column) và đẩy ngược ra Network Client dưới dạng JSON (‘Type: ERROR‘) nhằm nhắc người dùng sửa sai.

9.2.11 Lập kế hoạch Thực thi (Query Execution Plan)

Hệ thống quyết định cách thức thực hiện truy vấn tối ưu nhất:

- **Access Pattern:** Sử dụng Index (B+ Tree) hay quét tuần tự (Full Scan)?
- **Join Strategy:** Áp dụng phương pháp JOIN nào để kết hợp các Khái niệm?
- **Reasoning Injection:** Nếu truy vấn yêu cầu suy diễn (‘INFER‘ hoặc truy cập các biến suy diễn), hệ thống sẽ tiêm (Inject) thêm các bước gọi vào ‘ReasoningEngine‘ trước khi trả kết quả cuối cùng.

9.2.12 Ví dụ luồng xử lý thực tế

Câu lệnh: ‘SELECT name FROM Student WHERE age > 18;‘

1. **Lexer:** ‘[SELECT, name, FROM, Student, WHERE, age, >, 18, ;]‘
2. **Parser:** Tạo đối tượng ‘SelectStatement‘ với ‘Table=Student‘, ‘Filter=(age > 18)‘.
3. **Binder:** Xác nhận ‘Student‘ có trong KB, ‘name‘ và ‘age‘ là hợp lệ.
4. **Executor:**
   - Gọi ‘StorageEngine‘ để lấy dữ liệu từ B+ Tree của ‘Student‘.
   - Sử dụng ‘ReasoningEngine‘ để lọc các bản ghi có ‘age > 18‘.
   - Chỉ trả về trường ‘name‘ cho phía Network Layer.

9.2.13 Kế hoạch Triển khai Mã Nguồn Backend (Implementation Strategy)

Gắn liền với kiến trúc C# của hệ thống, luồng thiết kế cho tầng mạng và Bộ phân tích từ vựng (Network & Parser) được thực thi theo 4 Giai đoạn định hình thực tế:

Giai đoạn 1: Quản trị Kết nối (TCP Listener & Async Handle)

Trực tiếp thiết lập máy chủ ‘KbmsServer‘ trên kiến trúc Async/Await I/O. Quăng nhiệm vụ hứng Client đang rỗi vào vùng không chặn (Fire-and-forget qua ‘_ = HandleClientAsync()‘) nhằm khai phóng ThreadPool. Thiết lập hệ thống quét tệp chết (Cleanup Expired Sessions) nhằm đóng ngắt ‘Socket‘ lậu và hoàn trả RAM về máy chủ.

Giai đoạn 2: Mã hóa Giao thức Nhị phân (Packet Binary Protocol)

Tiến hành lập trình bộ vỏ ‘KBMNetwork‘ để viết định dạng truyền tải tuân thủ tài liệu cấu trúc gói ‘Protocol.cs‘. Viết các thao tác C# Core cấp thấp ‘Span<byte>‘, ‘BinaryPrimitives‘ xử lý ‘BigEndian‘ tạc khuôn truyền tin (Đọc đủ Byte Session, RequestID, và Payload).

Giai đoạn 3: Phân tích Cú Pháp Đệ Quy (Lexer & AST Parser)

Sử dụng tư duy trình biên dịch tạo module ‘KBMS.Parser‘. Lập trình ‘Token‘ và quét nhận dạng Regex cắt chuỗi chữ vô nghĩa thành mảng ký pháp Logic. Xây dựng một cỗ máy phân tích đi xuống (Recursive Descent Parser) quét nối Tokens thành Cây Cú Pháp ‘AST‘ (Abstract Syntax Tree), giúp CSDL hiểu ý đồ của con người.

Giai đoạn 4: Bảo mật & Phiên Giao dịch (Session & Security)

Quản trị hàng ngàn User kết nối TCP đồng thời thông qua tính năng cấu trúc ‘ConcurrentDictionary‘ của lớp ‘ConnectionManager.cs‘. Triển khai tính năng User Auth Protocol (LOGIN Message), xác minh Password mã hóa trên hệ từ điển ‘UserCatalog‘, rà soát quyền sở hữu và hạn chế tấn công. Bắt lỗi AST trả về cấu trúc lỗi (kéo theo Line/Col) trên gói JSON Network gửi tới UI của Designer.

9.3 Xác thực Giao thức (Protocol Validation)

Tầng mạng đảm bảo mọi thông điệp nhị phân được truyền dẫn không sai lệch và có độ trễ thấp nhất.

9.3.1 Kiểm thử Gói tin Nhị phân (Binary Packet Testing)

Sử dụng bộ kịch bản **‘Protocol.cs‘** để đóng gói và giải mã mockup:

- **Handshake**: Mô phỏng quá trình Client gửi gói tin LOGIN và chờ phản hồi RESULT.
- **Streaming**: Kiểm tra khả năng nhận diện dấu hiệu kết thúc luồng dữ liệu (‘FETCH_DONE‘).

Minh chứng Mã nguồn (Handshake):

Thiếu ảnh: code_test_binary_protocol.png

Hình 9.3: Kịch bản kiểm thử đóng gói và phân rã gói tin nhị phân.

Minh chứng Luồng dữ liệu (Gói tin):

Thiếu ảnh: terminal_test_protocol_hex.png

Hình 9.4: Nhật ký gói tin nhị phân (Little-endian) truyền trên Socket.

9.3.2 Kiểm thử Tải Kết nối (Concurrency Proof)

Mô phỏng 256 người dùng cùng thực thi truy vấn để kiểm tra giới hạn của ‘Asynchronous Sockets‘.

Minh chứng Kết quả (Concurrency):

Thiếu ảnh: terminal_test_concurrent_clients.png

Hình 9.5: Bằng chứng Server xử lý đồng thời nhiều Client mà không nghẽn.

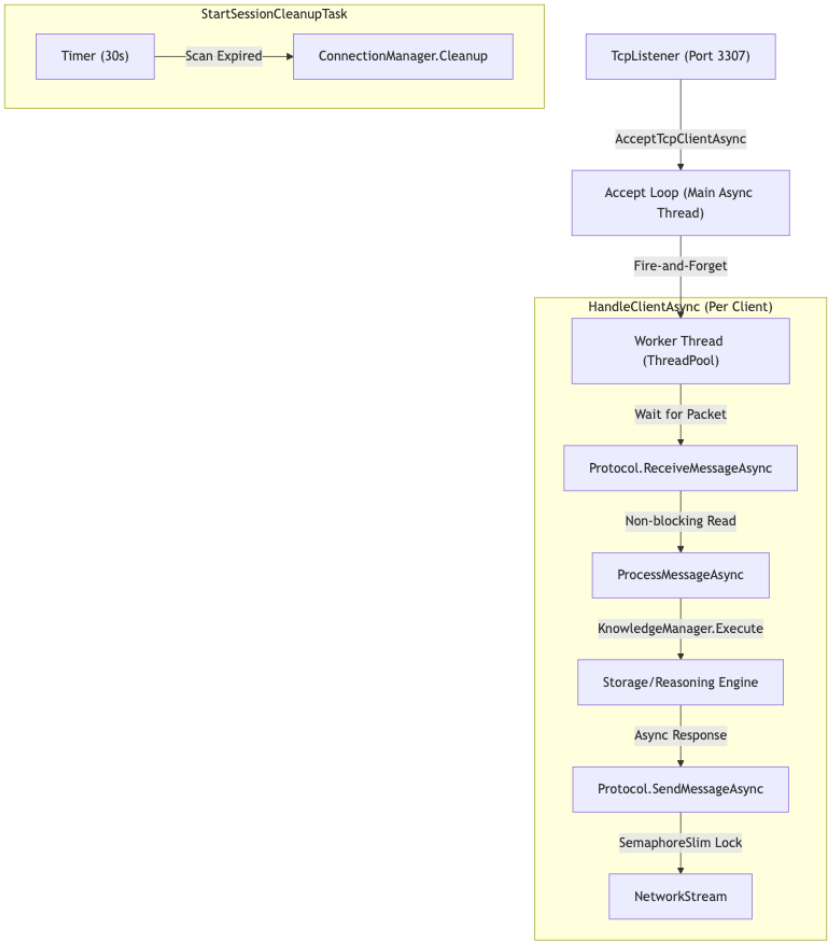
CHƯƠNG 10: Hệ thống Máy chủ và Quản trị Tài nguyên

10.1 Điều phối Mạng & Luồng Hoạt động

Máy chủ KBMS (‘KbmsServer.cs‘) được thiết kế trên mô hình xử lý hiệu năng cao, cho phép tiếp nhận hàng ngàn kết nối đồng thời nhờ vào kiến trúc **Task-based Asynchronous Pattern (TAP)** của .NET.

10.1.1 Mô hình Xử lý Luồng (Async Threading Model)

Khác với các hệ thống cũ sử dụng ‘1-Thread-Per-Client‘, KBMS sử dụng kỹ thuật bất đồng bộ thực thụ để giải phóng bộ nhớ và tăng khả năng chịu tải.



Hình 10.1: Mô hình xử lý bất đồng bộ (Asynchronous) sử dụng ThreadPool của .NET.

1. **Chấp nhận kết nối (Acceptance Loop)**: Vòng lặp ‘while‘ chính sử dụng ‘await _listener.AcceptTcpClientAsync(...)‘. Khi có client kết nối, hệ thống không đợi mà ngay lập tức spawn một tác vụ con (‘_ = HandleClientAsync(client)‘) rồi quay lại lắng nghe client tiếp theo.

2. **Xử lý Gói tin (Fire-and-Forget)**: Nhiệm vụ xử lý client được đẩy hoàn toàn vào ThreadPool của hệ điều hành. Điều này giúp Main Thread của Server không bao giờ bị nghẽn (Non-blocking).
3. **Hàng đợi Ghi (Semaphore Lock)**: Do Server có thể gửi nhiều thông điệp bất đồng bộ (ví dụ: đang gửi kết quả Query thì Server muốn gửi thêm Live Log), lớp 'Protocol' sử dụng 'SemaphoreSlim' để khóa luồng ghi nhị phân, đảm bảo các byte không bị trộn lẫn trên đường truyền.

10.1.2 Quản lý Phiên (Session Management)

Kéo dài sự tồn tại của kết nối và định danh người dùng thông qua lớp 'ConnectionManager.cs'.

- **Lưu trữ an toàn luồng**: Toàn bộ các phiên làm việc được lưu trong một 'ConcurrentDictionary'. Đây là cấu trúc dữ liệu tối ưu cho phép hàng trăm Thread con cùng lúc đọc/ghi thông tin Session mà không gây ra lỗi tranh chấp bộ nhớ (Race Conditions).
- **Dọn dẹp tài nguyên (Cleanup Task)**: Máy chủ khởi chạy một tiến trình ngầm 'StartSessionCleanupTask' định kỳ mỗi **30 giây**. Nó quét qua danh sách 'LastActivityAt' và tự động đóng các 'Socket' của client đã quá lâu không tương tác ('DefaultTimeoutSeconds'), giúp rảnh RAM cho những kết nối mới.

10.1.3 Vòng đời của một Giao dịch (Lifecycle)

Máy chủ vận hành theo một vòng lặp sự kiện (Event Loop) cho mỗi client:

1. **Receive**: Đọc Header (4 byte đầu) để biết kích thước gói.
2. **Dispatch**: Chuyển giao gói tin cho 'ProcessMessageAsync'.
3. **Execute**: Nếu là truy vấn, Server gọi 'KnowledgeManager' để xử lý.
4. **Respond**: Đóng gói kết quả và gửi ngược lại client.

10.2 Kiến trúc Bộ não Điều phối (Knowledge Engine)

'KnowledgeManager.cs' là thành phần trung tâm, đóng vai trò "Bộ não"của toàn bộ hệ thống KBMS. Nó kết nối giữa Tầng Ngôn ngữ (Parser/AST) và Tầng Lưu trữ/Suy diễn (Storage/Reasoning).

10.2.1 Cơ chế Bộ hạ lệnh (The Dispatcher)

Sau khi 'KBMS.Parser' bẻ gãy câu lệnh thành một cây cú pháp (AST), 'KnowledgeManager' sẽ thực hiện đệ quy qua danh sách các nút này và gọi hàm 'Execute(ast, user, currentKb)'.

```
// Luồng thực thi lõi trong KnowledgeManager.cs
public object Execute(AstNode ast, User user, string? currentKb) {
    var kbName = DetermineKbName(ast) ?? currentKb;
    var action = DetermineAction(ast);

    if (!CheckPrivilege(user, action, kbName))
        return ErrorResponse.PermissionErrorResponse(action, kbName);

    return ExecuteQuery(ast, kbName);
}
```

Chức năng chính:

- **Routing (Định tuyến)**: Xác định lệnh này thuộc về Knowledge Base (KB) nào dựa trên từ khóa 'USE' hoặc tham số câu lệnh.
- **Security Guard (Bảo vệ)**: Sử dụng 'CheckPrivilege' để lọc quyền của 'User' hiện tại (ROOT, ADMIN, WRITE, SELECT). Mọi hành động xâm phạm đều bị chặn ngay tại đây trước khi chạm vào dữ liệu.
- **Execution Pipeline**: Hàm 'ExecuteQuery' chứa một bộ chuyển mạch (Switch-case) khổng lồ điều hướng hơn **50 loại nút AST** khác nhau (từ 'CREATE_CONCEPT' đến 'SOLVE').

10.2.2 Các Công nghệ Liên kết Tri thức (Knowledge Binding)

Hệ thống điều phối của KBMS V3 mang tính đột phá nhờ các cơ chế sau:

Auto-expand Variables (Tự nở rộng biến)

Khi bạn khai báo một Concept chứa thuộc tính là một Concept khác (Ví dụ: ‘p: Point‘), ‘KnowledgeManager‘ sẽ tự động ”nở”biến này thành các thuộc tính con (‘p.x‘, ‘p.y‘).

- **Lợi ích**: Giúp bộ máy suy diễn (‘ReasoningEngine‘) có thể truy cập trực tiếp vào các biến cơ sở mà không cần phải parse lại cấu trúc object phức tạp (Flattening process).

Triggers Registry

‘KnowledgeManager‘ duy trì một danh sách các **Trigger** đang hoạt động. Ngay sau khi một lệnh ‘INSERT‘, ‘UPDATE‘ hoặc ‘DELETE‘ thành công, bộ điều phối sẽ tự động gọi hàm ‘FireTriggers‘ để kích hoạt các phản ứng dây chuyền (Chain reactions) trong tri thức.

V3 Data Router

Bộ điều phối không còn truy cập file trực tiếp. Nó giao tiếp thông qua ‘V3DataRouter‘. Router này chịu trách nhiệm xác định vị trí vật lý của dữ liệu trên hàng chục tệp tin ‘.kbf‘, ‘.dat‘ khác nhau, giúp KBMS mở rộng quy mô dữ liệu theo kiến trúc Multi-Database.

10.2.3 Quản lý Giao dịch (Transaction Control)

Khác với các hệ thống đơn giản, bộ điều phối của bạn hỗ trợ **Transaction Control Language (TCL)**:

- **BEGIN TRANSACTION**: Bật cờ ‘inTransaction‘ và khởi tạo bộ đệm (‘txBuffer‘).
- **COMMIT**: Phóng toàn bộ dữ liệu từ Buffer Pool xuống đĩa qua WAL.
- **ROLLBACK**: Xóa sạch bộ đệm, khôi phục trạng thái tri thức về điểm an toàn trước đó.

Thiếu ảnh: knowledge_manager_v3.png

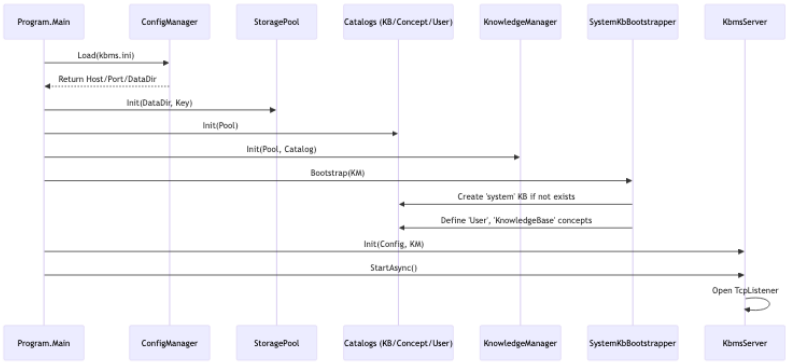
Hình 10.2: Sơ đồ tương tác giữa Knowledge Manager và các phân hệ cấp thấp.

10.3 Hạ tầng V3 (Infrastructure)

Phiên bản KBMS V3 giới thiệu một bước tiến lớn: **Tự khởi động bằng chính ngôn ngữ tri thức**. Mã nguồn tại ‘KBMS.Server/V3‘ chịu trách nhiệm thiết lập nền móng cho mọi Knowledge Base khác.

10.3.1 Cơ chế Tự nạp Hệ thống (System Bootstrapper)

Mỗi khi máy chủ KBMS khởi động lần đầu tiên trên một thư mục dữ liệu mới, ‘SystemKbBootstrapper.cs‘ sẽ được gọi để thực thi tiến trình ”Khai thiên lập địa”:



Hình 10.3: Luồng khởi động Server và đăng ký các Manager

Hình: Các bước khởi tạo hạ tầng từ Config đến Socket

1. **Khởi tạo ‘system‘ KB**: Hệ thống tự động lệnh ‘CreateKnowledgeBase(”system”)‘.
2. **Định nghĩa Siêu tri thức (Meta-Concepts)**: Bootstrapper nạp sẵn các khái niệm cốt lõi:
   - ‘User‘: Lưu trữ Username, Password (Hash), Role, Permissions.
   - ‘KnowledgeBase‘: Danh mục các KB hiện có.
   - ‘Session‘: Theo dõi lịch sử đăng nhập/đăng xuất.
     (a) **Bootstrap Admin**: Nếu chưa có người dùng nào, hệ thống tự động tạo tài khoản ‘root‘ mặc định.

10.3.2 Các Danh mục Chuyên biệt (Catalogs)

Khác với V2 (lưu mọi thứ chung 1 file), V3 tách biệt hoàn toàn để tối ưu hóa quản lý:

- **‘KbCatalog.cs‘**: Quản trị ”Phả hệ”của các tri thức. Nó lưu trữ Metadata của hàng chục KB và quản lý quyền truy cập.
- **‘ConceptCatalog.cs‘**: Quản lý hàng ngàn Concept và Rule của mỗi KB. Nó hỗ trợ nạp đè và thay đổi cấu trúc Concept mà không làm hỏng dữ liệu vật lý.
- **‘UserCatalog.cs‘**: Hệ thống quản trị người dùng trung tâm. Nó đóng vai trò là ”chứng minh thư”của cả máy chủ, cấp xác thực (Authentication) cho mọi kết nối TCP đi vào.

10.3.3 Hệ thống Cập nhật & Di trú (Updater & Migrator)

Để giữ cho hệ thống luôn ổn định giữa các phiên bản, KBMS cung cấp các công cụ hạ tầng trong thư mục này:

‘SystemUpdater.cs‘

Tự động quét và vá các Schema của ‘system‘ KB khi người dùng nâng cấp phần mềm. Điều này đảm bảo các tri thức cũ luôn tương thích với bộ suy diễn mới.

‘V2ToV3Converter.cs‘

Đây là ”Cây cầu nối”cho phép người dùng cũ của KBMS V2 chuyển toàn bộ tri thức sang V3. Nó thực hiện quét đĩa cũ, bóc tách ‘Objects‘ và tự động băm nhỏ dữ liệu để đẩy vào cấu trúc **Slotted Page 16KB** của V3.

10.4 Quản trị & Đo lường từ xa (Telemetry)

Một máy chủ KBMS C# hiện đại không thể thiếu khả năng tự theo dõi sức khỏe và phát tán nhật ký hoạt động (Logs). Mã nguồn tại ‘ManagementManager.cs‘ và ‘SystemLogger.cs‘ đảm nhiệm vai trò này.

10.4.1 Giám sát Hệ thống (Stats & Sessions)

Hệ thống cho phép các Client (đặc biệt là KBMS Studio) truy xuất dữ liệu vận hành thời gian thực thông qua bộ lệnh quản trị cấp cao:

- **Bộ đếm Metrics (STATS)**: Cung cấp thông tin về số lượng **Active Connections**, thời gian hoạt động của Server (Uptime), và tỷ lệ chiếm dụng RAM của ‘BufferPool‘. Dữ liệu này giúp người quản trị biết khi nào cần mở rộng bộ nhớ máy chủ.
- **Quản lý phiên (SESSIONS)**: Trích xuất danh sách tất cả các người dùng đang Online. Cho phép thực hiện các thao tác ”Kill Session”hoặc giới hạn số lượng kết nối đồng thời để bảo vệ tài nguyên hệ thống.

10.4.2 Hệ thống Nhật ký Luồng (Live Log Streaming)

Đây là tính năng độc đáo nhất của KBMS V3, cho phép ”lắng nghe”mọi biến động của máy chủ ngay lập tức.

‘SystemLogger.cs‘

Lớp này đóng vai trò là ”ngòi bút”của hệ thống. Khi bất kỳ lỗi nào xảy ra trong ‘Storage‘ hay ‘Reasoning‘, ‘SystemLogger‘ sẽ thực hiện đồng thời hai việc:

1. **Write to Disk**: Ghi vào file ‘.log‘ truyền thống để truy vết sau này.
2. **Write to System KB**: Ghi trực tiếp một Fact mới vào Concept ‘Log‘ của Knowledge Base ‘system‘.

‘LOGS_STREAM‘ Protocol

Khi một Admin Client gửi yêu cầu ‘LOGS_STREAM‘, ‘ManagementManager‘ sẽ thực hiện cơ chế **Pub/Sub (Publish/Subscribe)**:

- Bất kỳ dòng log mới nào vừa phát sinh sẽ được ”bắn”ngay lập tức vào Socket của client đó dưới dạng gói tin chuẩn.
- Điều này cho phép xây dựng các bảng điều khiển (Dashboard) giám sát tri thức mà không cần phải làm mới (Refresh) trang web hay ứng dụng.

10.4.3 Lệnh Quản trị Tối cao (Management Command)

Đối với các tác vụ thay đổi cấu hình nóng (Hot-swapping config) hoặc khôi phục dữ liệu, Server cung cấp Message Type ‘MANAGEMENT_CMD (0x0D)‘:

- **Chỉ dành cho ROOT**: Chỉ người dùng có vai trò ROOT mới có thể kích hoạt các lệnh này.
- **Tính năng**: Thay đổi tham số ‘MaxConnections‘, cấu hình ‘BufferPoolSize‘, hoặc khởi chạy tiến trình ‘Checkpoint‘ thủ công để dọn dẹp WAL.

10.5 Xác thực Máy chủ (Server Validation)

Máy chủ KBMS là trung tâm điều phối mọi yêu cầu, do đó nó được kiểm soát chặt chẽ từ khi khởi động đến khi xử lý các kịch bản thực tế.

10.5.1 Kiểm thử Khởi động (Startup Proof)

Hệ thống sử dụng bộ nạp **‘SystemKbBootstrapper.cs‘** để khởi tạo các danh mục hệ thống (Catalog).

- **Bootstrapping**: Tạo tệp ‘system.kdb‘ và đăng ký các Concept hệ thống (‘users‘, ‘audit_logs‘).
- **Log khởi động**:

Thiếu ảnh: terminal_test_server_bootlogs.png

Hình 10.4: Nhật ký khởi động máy chủ thành công với đầy đủ các Manager.

10.5.2 Kiểm thử Tích hợp Đa luồng (Multi-threading Proof)

Sử dụng **'CliServerIntegrationTests.cs'** để mô phỏng 100+ tình huống thực tế qua Socket nhị phân.

Tình huống	Mô tả	Kết quả
Login-Query	Đăng nhập và thực hiện SELECT ngay lập tức.	**Success**
Drop-While-Use	Xóa KB khi đang có Session truy cập.	**Error Caught (Locked)**
Large-Metadata	Truy vấn Concept có hàng trăm biến.	**Success**

Bảng 10.1: Tổng hợp kết quả kiểm thử hệ thống máy chủ

Minh chứng Mã nguồn (Integration):

Thiếu ảnh: code_test_server_integration.png

Hình 10.5: Kịch bản kiểm thử tích hợp (End-to-End) thông qua TCP Sockets.

Minh chứng Kết quả (111 Tests):

Thiếu ảnh: result_test_cli.png

Hình 10.6: Kết quả thực thi 111 kịch bản kiểm thử tích hợp toàn diện.

10.5.3 Xác thực Bảo mật (Security & RBAC)

Xác thực tính năng phân quyền người dùng thông qua **'AuthV3Tests.cs'**.

- **Root Access**: Toàn quyền thực hiện DDL/DML.
- **Non-root Access**: Bị từ chối khi thực hiện các lệnh nhạy cảm mà không có quyền.

Minh chứng Mã nguồn (Security):

Thiếu ảnh: code_test_security.png

Hình 10.7: Minh chứng mã nguồn kiểm thử bảo mật RBAC.

Minh chứng Kết quả (Access Denied):

Thiếu ảnh: result_test_security.png

Hình 10.8: Kết quả từ chối truy cập (Access Denied) đối với người dùng không có quyền.

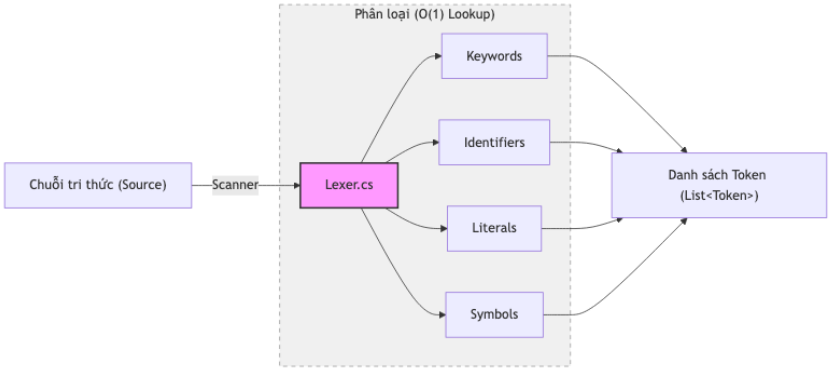
CHƯƠNG 11: Bộ Phân tích Cú pháp và Trình biên dịch

11.1 Lexing: Phân tích Từ vựng chuyên sâu (Lexer)

‘Lexer.cs‘ là thành phần đầu tiên của máy chủ KBMS tiếp nhận truy vấn người dùng. Nó thực hiện quét chuỗi (Scanning) $O(L)$ để bẻ gãy văn bản thô thành một danh sách các nguyên tử ngôn ngữ gọi là **Tokens**.

11.1.1 Cơ chế Quét (Scanning Mechanism)

Lexer vận hành như một máy trạng thái (State Machine) với hai con trỏ: ‘_start‘ (điểm bắt đầu của một từ) và ‘_current‘ (vị trí đang quét hiện tại).



Hình 11.1: Luồng Token được phân tách bởi Lexer trước khi đưa vào Parser

Hình: Quy trình quét chuỗi và định danh Token (dàn ngang)

Quy trình xử lý (‘ScanToken‘):

1. **Bỏ qua nhiễu**: Loại bỏ khoảng trắng (‘ ‘), tab (‘\t‘), xuống dòng (‘\n‘). Khi gặp xuống dòng, Lexer tự động tăng chỉ số ‘_line‘ và reset ‘_column‘ để phục vụ báo lỗi.
2. **Ký tự đơn**: Định danh ngay lập tức các Token như ‘(‘, ‘)‘, ‘,‘, ‘;‘, ‘+‘, ‘-‘, ‘*‘, ‘/‘.
3. **Toán tử kép**: Sử dụng hàm ‘Match()‘ để kiểm tra ký tự tiếp theo (Ví dụ: ‘>‘ và ‘>=‘).
4. **Chú thích (Comments)**: Khi gặp ‘–‘, Lexer sẽ bỏ qua toàn bộ phần còn lại của dòng.

11.1.2 Xử lý các Hằng số (Literals)

‘Lexer.cs‘ chứa các phương thức chuyên biệt để bóc tách dữ liệu thực tế:

Hằng số Số (‘NumberLiteral‘)

Hỗ trợ đầy đủ các định dạng số học:

- **Số nguyên**: ‘123‘.

- **Số thực**: ‘123.45‘.
- **Số âm**: Nhận diện dấu ‘-‘ đứng trước chữ số.
- **Số mũ (Scientific Notation)**: Hỗ trợ ‘1.23e-10‘.
- **Xử lý nội bộ**: Sử dụng ‘double.TryParse‘ với ‘CultureInfo.InvariantCulture‘ để đảm bảo tính nhất quán trên mọi hệ điều hành.

Hằng số Chuỗi (‘StringLiteral‘)

Hỗ trợ cả nháy đơn ‘’‘ và nháy kép ‘”‘:

- **Ký tự thoát (Escape)**: Cho phép dùng nháy kép lặp lại (‘””‘) để biểu diễn một dấu nháy đơn trong chuỗi.
- **Đa dòng**: Chuỗi có thể kéo dài qua nhiều dòng, Lexer sẽ tự nạp các ký tự xuống dòng vào giá trị literal.

11.1.3 Quản lý Từ khóa (Keyword Dictionary)

KBMS sở hữu một kho từ điển từ khóa (Keywords) đồ sộ với hơn **190 Token Types**.

Nhóm	Ví dụ từ khóa
DML	‘SELECT‘, ‘INSERT‘, ‘UPDATE‘, ‘DELETE‘, ‘SOLVE‘
DDL	‘CREATE‘, ‘DROP‘, ‘ALTER‘, ‘CONCEPT‘, ‘RELATION‘
Types	‘INT‘, ‘DECIMAL‘, ‘VARCHAR‘, ‘BOOLEAN‘, ‘DATE‘
TCL	‘BEGIN‘, ‘COMMIT‘, ‘ROLLBACK‘

Bảng 11.1: Phân loại các nhóm Token trong Lexer

Lexer thực hiện tra cứu không phân biệt hoa thường (‘OrdinalIgnoreCase‘) để tạo ra các Token có Type tương ứng, giúp Parser xử lý nhanh chóng mà không cần so khớp chuỗi (String comparison).

11.2 Tầng Parser: Cơ chế Đệ quy Đi xuống (Recursive Descent)

Lớp ‘Parser.cs‘ là ”ngã tư”điều hướng mọi câu lệnh. Với hơn 3400 dòng mã nguồn, nó thực hiện một quy trình phân tích cú pháp hiệu năng cao dựa trên thuật toán **Recursive Descent (Đệ quy đi xuống)**.

11.2.1 Cơ chế Điều hướng (Sentence Dispatching)

Mỗi khi máy chủ nhận được một chuỗi Tokens từ Lexer, nó khởi tạo ‘Parser.cs‘ và gọi hàm ‘ParseAll()‘.

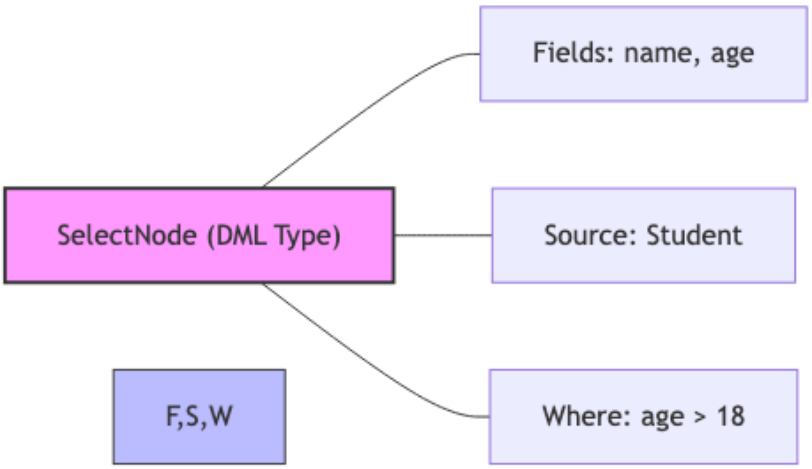
```
// Luồng xử lý đa câu lệnh trong Parser.cs
public List<AstNode> ParseAll() {
    var statements = new List<AstNode>();
    while (!IsAtEnd()) {
        var stmt = ParseStatement(); // "Ngã tư" điều hướng
        statements.Add(stmt);
        if (Check(TokenType.SEMICOLON)) Advance(); // Tiêu thụ dấu ';'
    }
}
```

Hàm ‘ParseStatement()‘

Đây là bộ khung switch-case khổng lồ, quyết định câu lệnh tiếp theo là DDL (Định nghĩa), DML (Truy vấn) hay TCL (Giao dịch). Mỗi khi nhận diện được một từ khóa bắt đầu (như ‘CREATE‘, ‘SELECT‘), nó sẽ ủy quyền cho một hàm con chuyên biệt (ví dụ: ‘ParseCreate()‘, ‘ParseSelect()‘).

11.2.2 Kỹ thuật Dự đoán & Tiêu thụ (Look-ahead)

KBMS sử dụng ngữ pháp **LL(k)**, cho phép Parser "nhìn trước"k Token để quyết định nhánh thực thi mà không cần quay lui (Backtracking).



Hình 11.2: Cấu trúc cây phân tích cú pháp (AST) của KBQL

Hình: Ví dụ cấu trúc cây AST sau khi phân tích (dàn ngang)

- **'Peek()'**: Xem Token hiện tại mà không tiêu thụ.
- **'Check(type)'**: Kiểm tra xem Token hiện tại có đúng Type mong muốn hay không.
- **'Consume(type)'**: Nếu Token đúng Type, tiến tới Token tiếp theo. Nếu sai, ngay lập tức ném ra 'ParseException' kèm tọa độ 'Line/Column'.
- **'Advance()'**: Luôn tiến về phía trước, đảm bảo độ phức tạp thời gian là $O(N)$ (N là số lượng Token).

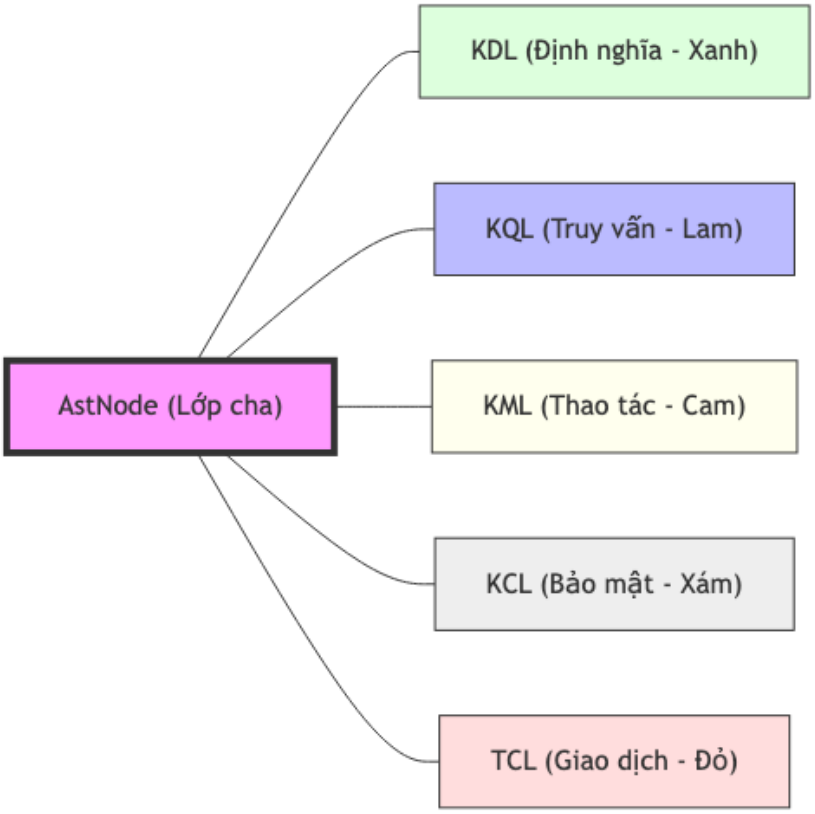
11.2.3 Phân tích Khối Lệnh phức tạp (Complex Parsing)

Một trong những phần xử lý tinh vi nhất trong 'Parser.cs' là phân tích lệnh 'CREATE CONCEPT'. Thay vì một parser phẳng, nó thực hiện đệ quy lồng nhau:

1. **'ParseCreate'** gọi **'ParseCreateConcept'**.
2. **'ParseCreateConcept'** lặp trong khối '(...)' và kiểm tra từ khóa.
3. Nếu gặp 'VARIABLES', nó gọi **'ParseVariableDefinition'** để bóc tách 'name: TYPE'.
4. Nếu gặp 'CONSTRAINTS', nó gọi **'ParseConstraintList'** (đây là lúc Expression Parser được kích hoạt).
5. Nếu gặp 'RULES', nó gọi **'ParseConceptRuleList'** (phân tích IF-THEN logic).

11.3 Hệ thống Cây cú pháp Trừu tượng (AST Nodes)

Cây AST là đại diện logic hoàn chỉnh cho tri thức sau khi được biên dịch. Tại KBMS, mỗi câu lệnh được bóc tách thành một hoặc nhiều 'AstNode' chuyên biệt, phân cấp theo 5 họ ngôn ngữ thực tế trong thư mục 'KBMS.Parser/Ast'.

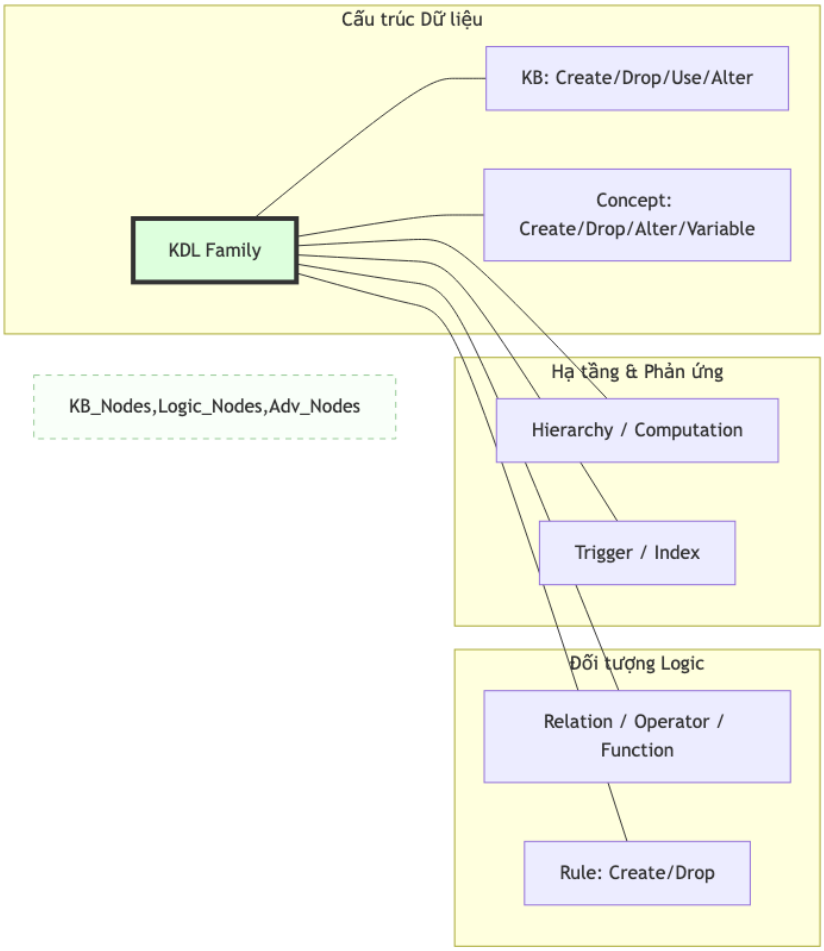


Hình 11.3: Tổng quan kiến trúc nút AST của ngôn ngữ KBQL

Hình: Tổng quan 5 nhánh Phả hệ Tri thức KBMS

11.3.1 Hệ phả KDL (Knowledge Definition Language)

Đây là họ lớn nhất, chịu trách nhiệm định nghĩa ”cơ thể”của tri thức:



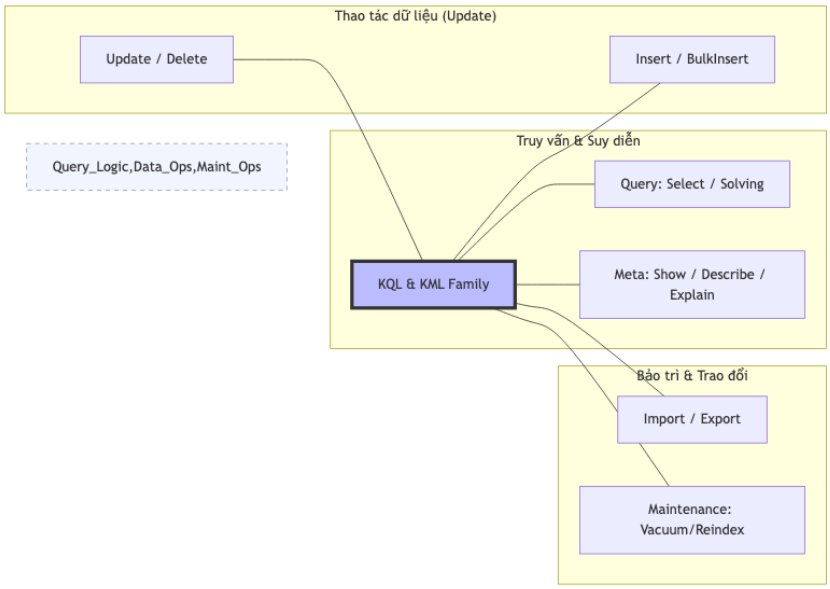
Hình 11.4: Sơ đồ chi tiết nút AST cho ngôn ngữ KDL

Hình: Chi tiết các Node định nghĩa tri thức (KDL)

- **Knowledge Base**: ‘CreateKnowledgeBaseNode‘, ‘DropKbNode‘, ‘UseKbNode‘, ‘AlterKbNode‘.
- **Concepts**: ‘CreateConceptNode‘, ‘DropConceptNode‘, ‘AlterConceptNode‘, ‘AddVariableNode‘.
- **Rules**: ‘CreateRuleNode‘, ‘DropRuleNode‘.
- **Hierarchy**: ‘AddHierarchyNode‘, ‘RemoveHierarchyNode‘.
- **Relation**: ‘CreateRelationNode‘, ‘DropRelationNode‘.
- **Operator**: ‘CreateOperatorNode‘, ‘DropOperatorNode‘.
- **Function**: ‘CreateFunctionNode‘, ‘DropFunctionNode‘.
- **Advanced**: ‘AddComputationNode‘, ‘RemoveComputationNode‘, ‘CreateIndexNode‘, ‘CreateTriggerNode‘.

11.3.2 Hệ phả KQL & KML (Query & Maintenance)

Các nhánh này chịu trách nhiệm hỏi đáp và bảo trì dữ liệu tri thức:



Hình 11.5: Sơ đồ chi tiết nút AST cho ngôn ngữ KQL và KML

Hình: Chi tiết các Node Truy vấn (KQL) và Thao tác (KML)

Nhóm KQL (Knowledge Query Language)

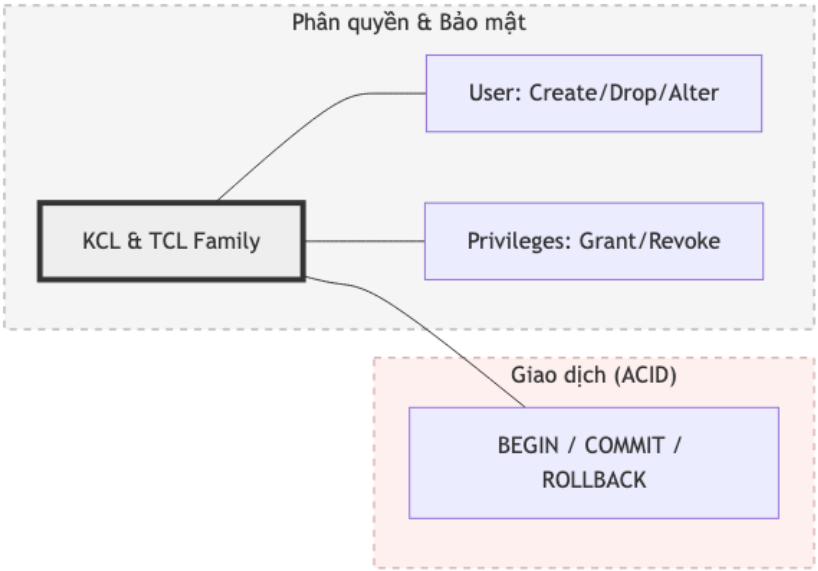
- **‘SelectNode‘**: Dùng để lọc và truy vấn các đối tượng tri thức dựa trên thuộc tính.
- **‘SolveNode‘**: Dùng cho bộ máy suy diễn (‘ReasoningEngine‘), chứa các mệnh đề ‘Given‘ (giả thiết) và ‘Find‘ (mục tiêu).
- **‘ShowNode‘**: Liệt kê danh sách các thành phần (Concepts, Rules, KnowledgeBases).
- **‘DescribeNode‘**: Xem chi tiết cấu trúc của một Concept cụ thể.
- **‘ExplainNode‘**: Giải thích luồng suy luận của một câu lệnh tri thức.

Nhóm KML (Knowledge Maintenance Language)

- **‘InsertNode‘ / ‘InsertBulkNode‘**: Nạp tri thức mới vào đĩa.
- **‘UpdateNode‘ / ‘DeleteNode‘**: Sửa đổi và xóa bỏ các Object Instance.
- **‘ImportNode‘ / ‘ExportNode‘**: Giao lưu tri thức với các tệp tin bên ngoài (‘.csv‘, ‘.json‘).
- **‘MaintenanceNode‘**: Các lệnh quản trị bộ nhớ thấp cấp (v.d: ‘REINDEX‘, ‘VACUUM‘).

11.3.3 Hệ phả KCL & TCL (Control & Transaction)

Nhánh bảo mật và quản lý giao dịch:



Hình 11.6: Sơ đồ chi tiết nút AST cho ngôn ngữ KCL và TCL

Hình: Chi tiết các Node Bảo mật (KCL) và Giao dịch (TCL)

- **DCL (Control)**: ‘CreateUserNode‘, ‘DropUserNode‘, ‘AlterUserNode‘, ‘GrantNode‘, ‘RevokeNode‘.
- **TCL (Transaction)**: ‘BeginTransactionNode‘, ‘CommitNode‘, ‘RollbackNode‘.

11.3.4 Đặc điểm chung của mọi Node

Mọi lớp Node đều kế thừa từ **‘AstNode.cs‘**, đảm bảo các tính năng cốt lõi:

1. **Vị trí nguồn**: Lưu trữ ‘Line‘ (Dòng) và ‘Column‘ (Cột) của câu lệnh gốc.
2. **Type định danh**: Mọi Node đều có thuộc tính ‘Type‘ (như ‘”CREATE_CONCEPT”‘) giúp Server Routing nhanh chóng.
3. **ToQueryString()**: Khả năng tái tạo lại mã nguồn KBQL từ cây AST (Reverse Engineering).

11.4 Bộ máy Đánh giá Biểu thức (Expression Engine)

Expression Engine là ”trái tim”của mọi câu lệnh tri thức. Lớp ‘ExpressionNode.cs‘ và các lớp con trong thư mục ‘KBMS.Parser/Ast/Expressions‘ chịu trách nhiệm xử lý các phép toán và logic phức tạp.

11.4.1 Phân tích Biểu thức Nhị phân (Binary Expressions)

Đây là các phép toán có hai vế, được xử lý thông qua ‘BinaryExpressionNode‘.

- **Toán tử Toán học**: ‘+‘, ‘-‘, ‘*‘, ‘/‘, ‘^‘, ‘%‘.
- **Toán tử So sánh**: ‘=‘, ‘!=‘, ‘<‘, ‘>‘, ‘<=‘, ‘>=‘.
- **Toán tử Logic**: ‘AND‘, ‘OR‘.

Thứ tự ưu tiên (Precedence)

Parser của KBMS thực hiện phân tích theo mức độ ưu tiên chuẩn quốc tế để đảm bảo kết quả chính xác:

1. **Cấp 1**: Dấu ngoặc ‘(...)‘.
2. **Cấp 2**: Hàm gọi và Truy cập thuộc tính (‘.‘).
3. **Cấp 3**: Lũy thừa ‘^‘.

4. **Cấp 4**: Nhân/Chia ‘*‘, ‘/‘.
5. **Cấp 5**: Cộng/Trừ ‘+‘, ‘-‘.
6. **Cấp 6**: So sánh ‘=‘, ‘!=‘, ‘<‘, ‘>‘.
7. **Cấp 7**: Logic ‘NOT‘, ‘AND‘, ‘OR‘.

11.4.2 Các Đơn vị Biểu thức Cốt lõi (Core Expression Units)

Trong cây AST, biểu thức được phân rã thành các nút lá và nút nhánh:

‘VariableNode‘ (Biến tri thức)

Đại diện cho các thuộc tính của Concept (Ví dụ: ‘a‘, ‘age‘). Parser hỗ trợ truy cập thuộc tính lồng nhau thông qua dấu chấm (‘p1.x‘), giúp xử lý tri thức đa tầng.

‘LiteralNode‘ (Hằng số)

Chứa giá trị thực tế như số (‘100‘), chuỗi (‘”Active”‘), hoặc Boolean (‘true‘/‘false‘). Mỗi nút lưu trữ cả giá trị thô và kiểu dữ liệu (‘TokenType‘).

‘FunctionCallNode‘ (Hàm gọi)

Xử lý các hàm toán học hoặc tri thức tích hợp (v.đ: ‘Sqrt(x)‘, ‘Abs(y)‘). Parser bóc tách tên hàm và một danh sách các tham số (‘List<ExpressionNode>‘), cho phép các hàm có thể lồng nhau linh hoạt.

11.4.3 Xử lý Biểu thức Đơn phân (Unary Expressions)

Được đại diện bởi ‘UnaryExpressionNode‘, dùng cho các trường hợp chỉ có một vế:

- **Phép phủ định**: ‘NOT (a > B)‘.
- **Số âm**: ‘-x‘.

11.5 Xác thực Bộ biên dịch (Parser Validation)

Parser là thành phần nhạy cảm nhất đối với đầu vào của người dùng. Mọi cú pháp sai sót đều phải được ngăn chặn ngay lập tức.

11.5.1 Thống kê Độ bao phủ Testing

Tầng Parser được bảo vệ bởi **21,000+ kịch bản test** trong ‘ParserTests.cs‘ và ‘LexerTests.cs‘.

- **Positive Cases**: 100% các từ khóa và cấu trúc KDL/KQL/KML/KCL/TCL được bao phủ.
- **Negative Cases**: Kiểm tra khả năng từ chối các câu lệnh thiếu dấu chấm phẩy, sai kiểu dữ liệu hoặc sai tên Concept.

Minh chứng Mã nguồn (Parser):

Thiếu ảnh: code_test_parser.png

Hình 11.7: Minh chứng mã nguồn kiểm thử độ bao phủ của bộ phân tích (Parser).

Minh chứng Kết quả (21k Tests):

Thiếu ảnh: result_test_parser.png

Hình 11.8: Kết quả vượt qua toàn bộ 21,000+ kịch bản kiểm thử cú pháp.

11.5.2 Đặc tả Lỗi Cú pháp (Syntax Error Proof)

Hệ thống báo lỗi chi tiết đến từng tọa độ (Dòng, Cột) giúp người dùng dễ dàng hiệu chỉnh:

```
-- Lỗi mô phỏng: Thiếu dấu ngoặc đóng
CREATE CONCEPT X ( VARIABLES ( id: INT );
-- [ERROR] Unexpected token ';', expected ')' at Line 1, Col 35.
```

Minh chứng Lỗi (Studio Editor):

Thiếu ảnh: terminal_test_parser_nested.png

Hình 11.9: Chứng minh Parser xử lý thành công các biểu thức lồng nhau phức tạp.

CHƯƠNG 12: Giao diện Dòng lệnh (CLI)

12.1 Tổng quan Giao diện Dòng lệnh (CLI)

Quản trị

'KBMS.CLI' là một ứng dụng Console (C#) mạnh mẽ, cung cấp khả năng tương tác trực tiếp với máy chủ KBMS thông qua giao thức nhị phân và dòng lệnh KQL/KDL.

12.1.1 Các Tính năng Chính

- **REPL (Read-Eval-Print Loop)**: Người dùng nhập câu lệnh, CLI gửi tới Server, nhận kết quả và in ra ngay lập tức.
- **Advanced Line Editing**: Hỗ trợ đầy đủ các phím di chuyển và thao tác nhanh ('LineEditor.cs'):
  - **Mũi tên Lên/Xuống**: Duyệt lịch sử lệnh.
  - **Home/End**: Di chuyển nhanh đến đầu/cuối dòng.
  - **Delete/Backspace**: Xóa ký tự tại con trỏ.
  - **Esc x2**: Xóa sạch bộ đệm đang nhập (Double Escape to Clear).
- **Multi-line Input**: Hỗ trợ nhập các câu lệnh dài. Chế độ thụt đầu dòng tự động '->' giúp phân biệt dòng tiếp nối.
- **Display Modes**: Cung cấp hai chế độ hiển thị chính thông qua 'ResponseParser.cs':
  - **Table Mode**: Hiển thị bảng ngang (MySQL style).
  - **Vertical Mode (\G)**: Hiển thị theo hàng dọc, tự động kích hoạt cho các lệnh 'DESCRIBE' hoặc 'EXPLAIN' để tối ưu hóa việc đọc cấu trúc tri thức phức tạp.
- **Chế độ Chạy Script ('SOURCE')**: Cho phép thực thi các tệp tin truy vấn tri thức lớn ('.kbql') một cách tự động, tự dừng khi gặp lỗi.
- **Xử lý Phản hồi Phức tạp**: Sử dụng 'ResponseParser.cs' để hiển thị kết quả truy vấn dưới dạng bảng (Table), đồ thị tri thức hoặc thông báo lỗi line-accurate.
- **Tự động kết nối lại (Auto-reconnect)**: CLI tự động thử kết nối lại nếu máy chủ bị ngắt kết nối đột ngột (Heartbeat timeout).

12.1.2 Các Lệnh Hệ thống Đặc biệt

Lệnh	Mô tả
'LOGIN <u	p>'
'SOURCE <path>'	Thực thi script từ tệp tin bên ngoài.
'CONNECT'	Kết nối lại máy chủ thủ công.
'CLEAR'	Xóa sạch màn hình console.

Bảng 12.1: Danh mục lệnh điều khiển CLI

12.1.3 Trải nghiệm người dùng (UX)

Để mang lại trải nghiệm chuyên nghiệp như một hệ quản trị CSDL thực thụ, CLI được tích hợp các phím tắt:

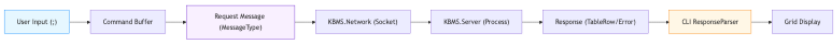
- **Mũi tên Lên/Xuống**: Xem lại lịch sử lệnh đã gõ.
- **Home/End**: Di chuyển con trỏ nhanh trong dòng lệnh.
- **Esc x2**: Xóa nhanh input đang nhập dở.

12.2 Luồng Xử lý Dữ liệu CLI

Để đảm bảo hiệu năng trong môi trường dòng lệnh, ‘KBMS.CLI‘ vận hành theo một luồng xử lý phi đồng bộ (Asynchronous) dựa trên socket.

12.2.1 Luồng Gửi lệnh (Command Execution Flow)

Khi người dùng nhấn Enter, CLI thực hiện các bước sau:



Hình 12.1: Sơ đồ luồng xử lý đầu vào trong giao diện CLI

1. **Input Buffering**: Tích lũy các dòng cho đến khi gặp dấu ‘;‘.
2. **Request Construction**: Đóng gói lệnh thành các ‘Message‘ (Dạng ‘QUERY‘ hoặc ‘LOGIN‘).
3. **Binary Transport**: Gửi qua ‘KBMS.Network‘ (Socket).
4. **Wait for Response**: Lắng nghe phản hồi từ máy chủ.

12.2.2 Bộ máy Phân tích Phản hồi (‘ResponseParser‘)

Phần phức tạp nhất của CLI nằm ở ‘ResponseParser.cs‘. Do kết quả từ Server có thể là một luồng dữ liệu (Streaming ROWS), CLI phải xử lý từng gói tin:

- **METADATA**: Định nghĩa các cột (Name, Type).
- **ROW**: Dữ liệu thực tế cho từng bản ghi.
- **RESULT**: Thông điệp cuối cùng (v.d: ”Insert successful”).
- **ERROR**: Chứa thông tin lỗi (Message, Line, Column).

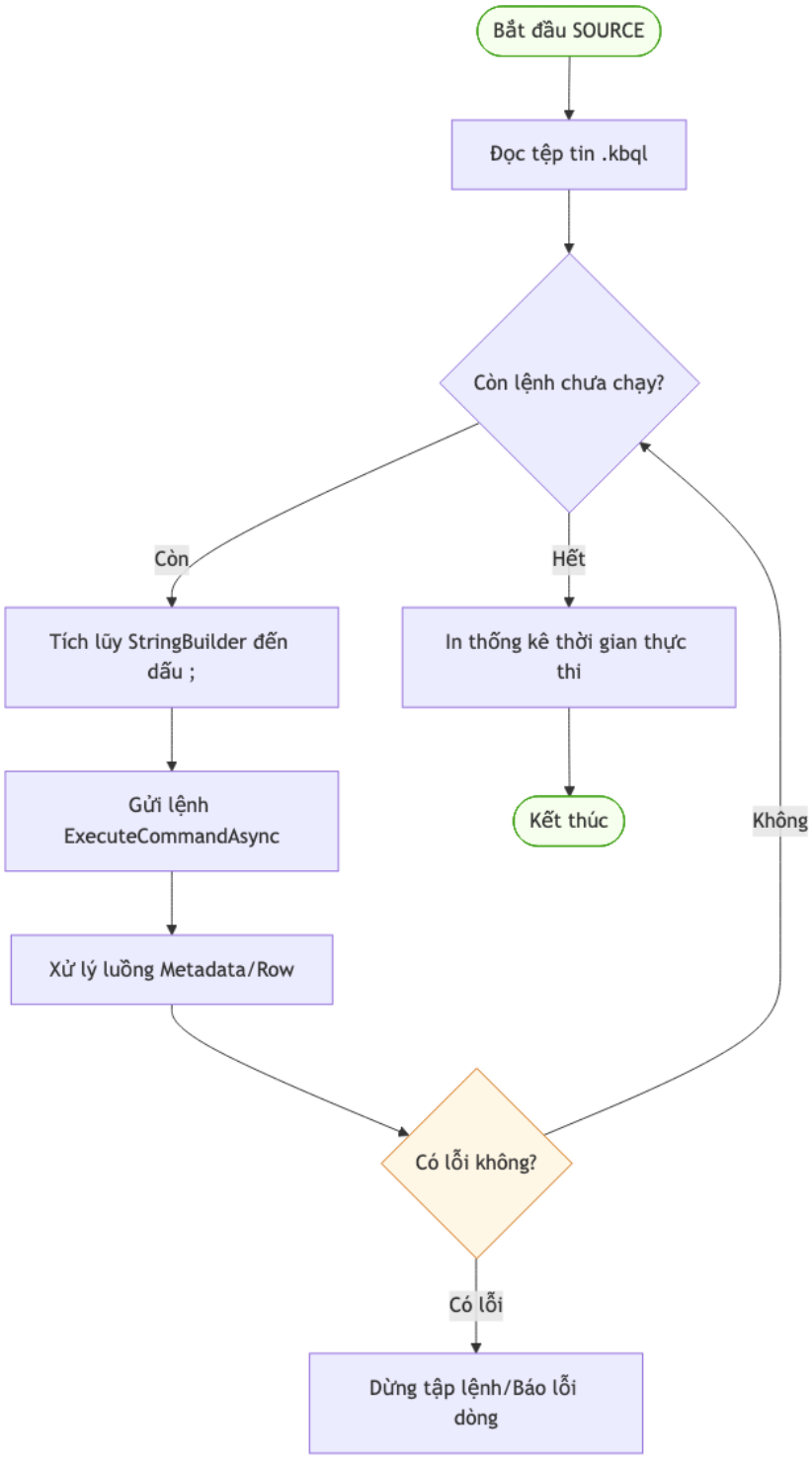
Quy trình hiển thị:

‘ResponseParser.cs‘ thực hiện vẽ bảng ”động”theo thuật toán tối ưu:

1. **Header Rendering**: Ngay khi nhận ‘METADATA‘, CLI tính toán độ rộng cột lớn nhất dựa trên tên cột và vẽ khung Header.
2. **Multi-line Cell Support**: Nếu dữ liệu trong một ô chứa dấu xuống dòng (‘\n‘), CLI sẽ tự động chia ô đó thành nhiều dòng và vẽ đường kẻ phân cách hàng (‘|—+—|‘) để đảm bảo bảng luôn cân đối.
3. **Vertical Rendering**: Đối với các phản hồi thuộc nhóm ‘Explain_‘ hoặc ‘Describe_‘, CLI chuyển sang chế độ hiển thị theo cặp ‘Field: Value‘ trên từng hàng dọc.

12.2.3 Chế độ Chạy Script (Batch Source)

CLI hỗ trợ thực thi khối lượng lớn lệnh thông qua tệp tin ‘.kbql‘. Luồng này được xử lý tuần tự để đảm bảo tính nhất quán:



Hình 12.2: Kịch bản thực thi hàng loạt qua lệnh SOURCE trong CLI

12.2.4 Đa luồng & Kết nối (Multi-threading)

CLI chạy hai luồng chính để duy trì trạng thái:

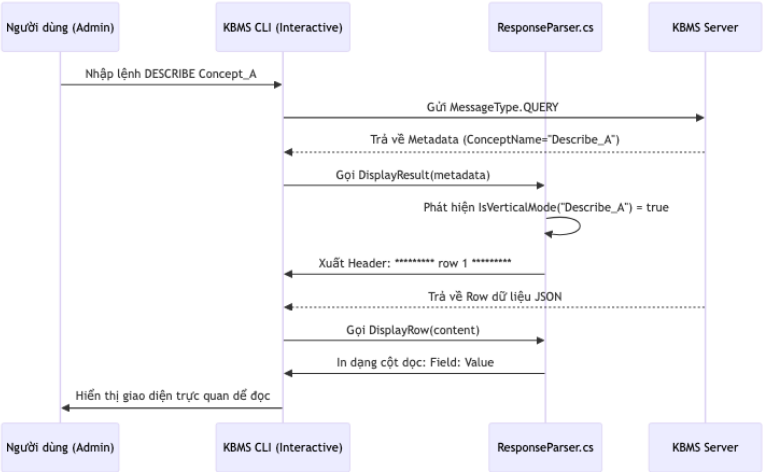
1. **Main Thread**: Chờ nhận input của người dùng.
2. **Heartbeat Thread**: (Tùy chọn) Gửi tín hiệu duy trì kết nối nếu cần thiết.

12.3 Các Trường hợp Sử dụng CLI

KBMS CLI là công cụ tối thượng cho các quản trị viên hệ thống và chuyên gia tri thức cần sự tốc độ và khả năng tự động hóa.

12.3.1 Kịch bản 1: Phân tích Cấu trúc (Structure Analysis)

- **Mục tiêu**: Xem chi tiết định nghĩa của một Concept phức tạp với hàng chục biến.
- **Sơ đồ Hoạt động**:



Hình 12.3: Chế độ hiển thị dọc (Vertical Mode) trong CLI

- **Quy trình chi tiết**:
  1. **Gửi lệnh**: Người dùng gõ ‘DESCRIBE <ConceptName>;‘.
  2. **Nhận diện**: CLI nhận gói tin Metadata có thuộc tính ‘ConceptName‘ bắt đầu bằng ‘Describe_‘.
  3. **Chế độ Dọc**: ‘ResponseParser‘ tự động chuyển sang Vertical Mode để hiển thị danh sách biến theo chiều dọc, tránh việc bảng bị tràn màn hình (Overflow).
  4. **Kết quả**: Từng biến được liệt kê rõ ràng kèm kiểu dữ liệu tương ứng.

12.3.2 Kịch bản 2: Tự động hóa Nạp Tri thức (Batch Loading)

- **Mục tiêu**: Nạp hàng ngàn thực thể tri thức từ tệp tin script có sẵn.
- **Quy trình chi tiết**:
  1. **Chuẩn bị**: Tạo tệp ‘import.kbql‘ chứa các lệnh ‘INSERT‘.
  2. **Thực thi**: Gõ ‘SOURCE import.kbql;‘ tại CLI.
  3. **Vòng lặp**: CLI đọc từng block lệnh (phân tách bởi dấu ‘;‘) và gửi tới Server.
  4. **Xử lý sai sót**: Nếu một lệnh gặp lỗi, CLI sẽ dừng ngay lập tức và chỉ ra vị trí dòng lệnh bị lỗi trong tệp nguồn để quản trị viên sửa chữa.
  5. **Thống kê**: Sau khi hoàn thành, CLI in ra tổng số bản ghi đã nạp và tổng thời gian thực thi.

12.3.3 Kịch bản 3: Kiểm tra Kết nối & An ninh (Security Check)

- **Mục tiêu**: Đăng nhập và kiểm tra tình trạng kết nối tới máy chủ từ xa.
- **Quy trình**:
  1. Người dùng gõ ‘LOGIN <username> <password>‘.
  2. CLI gửi gói tin ‘LOGIN‘ bảo mật tới Server.
  3. Server xác thực và trả về ‘LOGIN_SUCCESS‘.
  4. Prompt của CLI chuyển từ ‘login>‘ sang ‘kbms> ‘, sẵn sàng cho các lệnh truy vấn.

12.4 Xác thực CLI (CLI Validation)

KBMS CLI là giao diện dòng lệnh mạnh mẽ, cho phép tương tác trực tiếp và chạy các kịch bản tri thức quy mô lớn.

12.4.1 Kiểm thử Phiên làm việc (Session Logs)

Mọi thao tác trong CLI đều được ghi nhận và xác thực thông qua log kết nối tới Server.

- **Handshake nhị phân**: CLI gửi bản tin định danh và nhận SESSION_ID.
- **Batch execution**: Chạy script qua lệnh ‘SOURCE‘.

Minh chứng Mã nguồn (CLI System):

Thiếu ảnh: code_test_cli.png

Hình 12.4: Minh chứng mã nguồn kiểm thử giao diện dòng lệnh (CLI).

Minh chứng Kết quả (Session Log):

Thiếu ảnh: terminal_test_cli_query.png

Hình 12.5: Giao diện tương tác CLI thực thi câu lệnh truy vấn tri thức.

12.4.2 Kiểm thử Định dạng Kết quả

Xác thực khả năng căn chỉnh bảng và hiển thị kết quả truy vấn dưới dạng trực quan.

Query	Kết quả mô phỏng	Hiển thị bảng

Bảng 12.2: Kiểm thử định dạng kết quả truy vấn trong CLI

Thiếu ảnh: placeholder_cli_describe_output.png

Hình 12.6: Hiển thị kết quả truy vấn dưới dạng bảng (Table View) trong CLI.

CHƯƠNG 13: Môi trường Phát triển KBMS Studio

13.1 Tổng quan & Tính năng KBMS Studio là một môi trường phát triển tích hợp (IDE) hiện đại, được xây dựng trên nền tảng Electron và React, cung cấp trải nghiệm trực quan hóa tri thức thay vì chỉ thao tác dòng lệnh thuần túy.

13.1.1 Các Phân hệ Cốt lõi

a. Knowledge Designer (Trình thiết kế trực quan)

- Cho phép người dùng kéo thả và xem sơ đồ phả hệ Concept/Relation.
- Tự động phát hiện các xung đột logic khi thiết kế Schema bài toán.

b. Query Console (Cửa sổ truy vấn Monaco)

- **Trình soạn thảo Monaco**: Lõi của VS Code được tích hợp sâu, hỗ trợ tô màu cú pháp (Syntax Highlighting) cho KBQL.
- **Error Squiggles**: Đánh dấu lỗi đỏ tại vị trí chính xác (Dòng/Cột) khi Server trả về ‘ParseException‘.
- **IntelliSense**: Gợi ý các Concept và Thuộc tính đã định nghĩa.

c. Management Dashboard (Bảng điều khiển quản diện)

- **Giám sát RAM/Disk**: Theo dõi thời gian thực thông qua hệ thống telemetry.
- **User & Session Management**:
  - **User Management**: Thêm/Xóa/Sửa tài khoản và phân quyền trực tiếp từ UI (‘UserManagement.tsx‘).
  - **Active Sessions**: Kiểm soát danh sách các máy khách đang kết nối, hỗ trợ ngắt kết nối (Kick) các phiên không hợp lệ (‘ActiveSessions.tsx‘).
- **Log Analyzer**: Bộ công cụ phân tích nhật ký nâng cao, hỗ trợ lọc theo cấp độ (Level) và thực thể tri thức (‘LogAnalyzer.tsx‘).

d. Real-time Notification System

- **Push Alerts**: Nhận cảnh báo tức thời từ Server khi có sự cố hệ thống hoặc sự kiện an ninh (‘NotificationBell.tsx‘, ‘NotificationToasts.tsx‘).
- **Audit Trails**: Xem nhật ký truy cập chi tiết cho từng người dùng.

e. Advanced Settings & Multi-server

- **Connect Modal**: Hỗ trợ lưu trữ và chuyển đổi nhanh giữa nhiều máy chủ KBMS (‘ConnectModal.tsx‘).
- **System Settings**: Tùy chỉnh tham số Engine và giao diện người dùng (Themes).

13.1.2 Tiêu chuẩn Giao diện (Aesthetics)

Studio được thiết kế với ngôn ngữ **Modern Dark Mode** kết hợp với bộ biểu tượng tối giản, giúp người dùng tập trung tối đa vào các cấu trúc tri thức phức tạp.

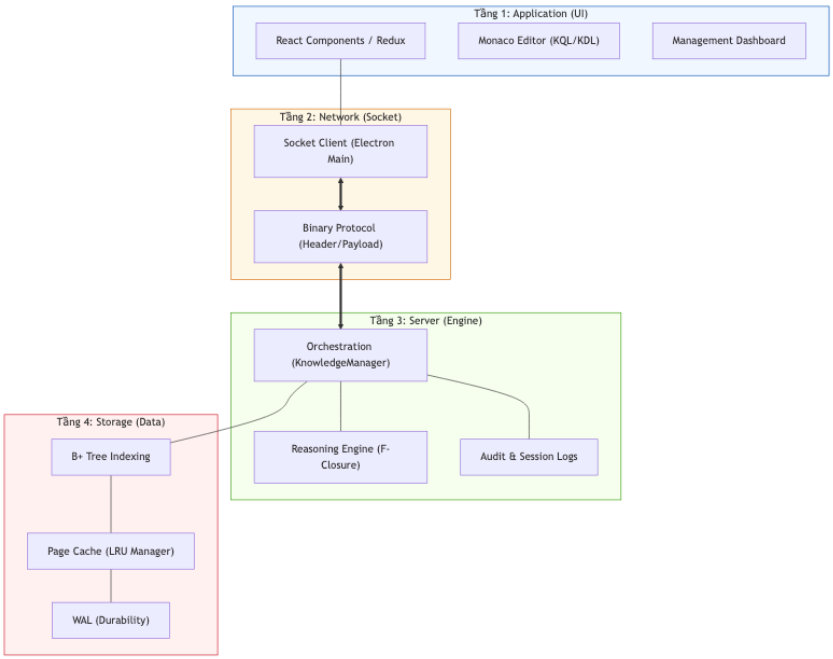
- **Responsive Layout**: Tự động điều chỉnh kích thước khi mở các panel theo dõi dữ liệu.
- **Streaming UI**: Kết quả trả về từ Server được đổ trực tiếp vào bảng dữ liệu (Data Grid) ngay khi có gói tin đầu tiên, không gây treo giao diện.

13.2 Kiến trúc 4 Tầng & Luồng Xử lý Chi tiết

KBMS Studio vận hành dựa trên một kiến trúc phân tầng chặt chẽ, từ giao diện người dùng đến các trang dữ liệu nhị phân thấp cấp trên đĩa cứng.

13.2.1 Sơ đồ Luồng xử lý 4 Tầng (4-Tier Flow)

Sơ đồ dưới đây mô tả cách một câu lệnh Tri thức (như ‘SOLVE‘) đi qua hệ thống:



Hình 13.1: Luồng xử lý một yêu cầu tri thức xuyên suốt 4 tầng từ Studio UI.

13.2.2 Đặc tả các Tầng

Tầng 1: Application UI (React/Electron)

- **Frontend**: React xử lý trạng thái luồng (Redux). Khi người dùng nhấn ”Run”, lệnh được chuyển xuống Electron Main process.
- **Auth**: Quản lý Session Token và quyền truy cập cục bộ trên giao diện.

Tầng 2: Network & Protocol (Socket)

- **Binary Framing**: Chuyển đổi lệnh thành các gói tin nhị phân.
- **Telemetry**: Gửi tín hiệu nhịp tim (Heartbeat) định kỳ để duy trì kết nối Socket ổn định giữa Studio và Server.

Tầng 3: Knowledge Engine (Server)

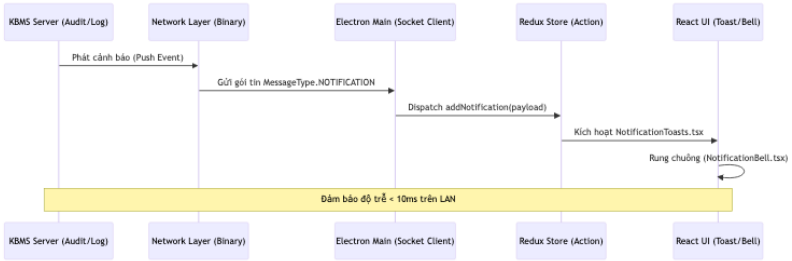
- **KnowledgeManager**: Điều phối việc phân tách câu lệnh bằng Lexer/Parser.
- **Reasoning**: Nếu là lệnh ‘SOLVE‘, bộ máy suy diễn sẽ thực hiện Forward Chaining trên các Page dữ liệu được nạp vào RAM.
- **Logs**: Ghi lại nhật ký truy cập (Audit) đồng thời Stream Log trở lại Studio UI.

Tầng 4: Storage Layer (Binary Data)

- **B+ Tree**: Tìm kiếm bản ghi Object dựa trên Index.
- **LRU Cache**: Quản lý các trang dữ liệu (Pages) trong bộ nhớ đệm để tối ưu hóa tốc độ.
- **WAL (Write-Ahead Logging)**: Đảm bảo tính toàn vẹn tri thức ngay cả khi mất điện đột ngột.

13.2.3 Luồng Thông báo Thời gian thực (Notification Flow)

Khác với luồng Request-Response thông thường, hệ thống Notification sử dụng cơ chế **Server Push**:



Hình 13.2: Cơ chế Server Push cho các thông báo hệ thống thời gian thực.

1. **Trigger**: Một sự kiện an ninh hoặc hệ thống xảy ra tại Server.
2. **Push**: Server đóng gói ‘MessageType.NOTIFICATION‘ và đẩy qua Socket.
3. **Dispatch**: Electron Main nhận gói tin và gửi vào Redux Store của React.
4. **UI Feedback**: Component ‘NotificationToasts‘ hiển thị thông báo popup, đồng thời cập nhật số lượng tin nhắn mới tại ‘NotificationBell‘.

13.2.4 Quy trình Đăng nhập (Authentication Flow)

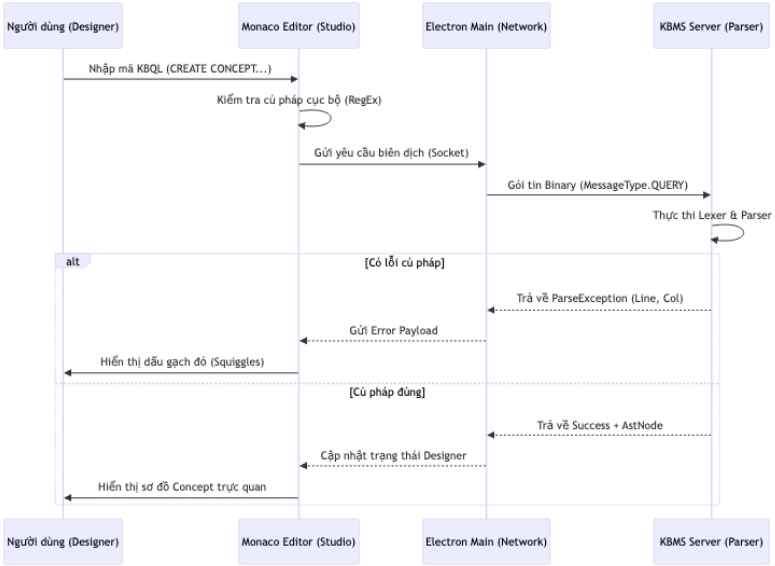
1. **Handshake**: Studio gửi gói tin ‘LOGIN‘ kèm User/Pass được mã hóa Base64.
2. **Server Verify**: Server kiểm tra trong hệ thống quản lý User (Tầng 4).
3. **Session Record**: Nếu khớp, Server tạo một ‘SessionContext‘ trong RAM (Tầng 3) và trả về thông báo thành công.
4. **Prompt Update**: Studio cập nhật trạng thái đã đăng nhập và cho phép thao tác với Knowledge Base.

13.3 Các Trường hợp Sử dụng Studio

KBMS Studio không chỉ là một trình soạn thảo, mà còn là một bộ công cụ giải quyết các bài toán tri thức chuyên sâu.

13.3.1 Kịch bản 1: Thiết kế Bản đồ Tri thức Hình học (KDL Design)

- **Mục tiêu**: Định nghĩa các Concept như ‘Diem‘, ‘DoanThang‘, ‘TamGiac‘ và các ‘Rule‘ tính toán diện tích.
- **Sơ đồ Hoạt động**:

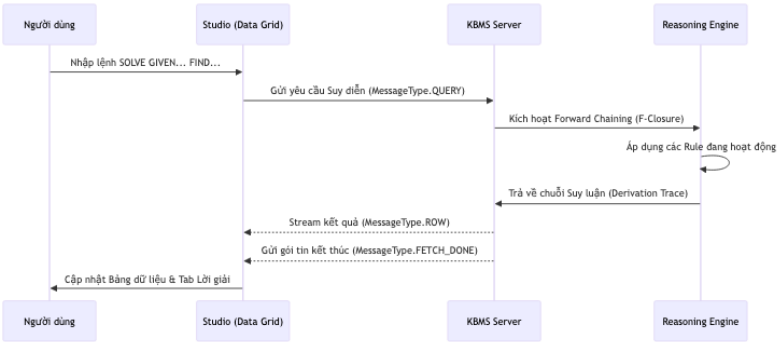


Hình 13.3: Luồng quy trình thiết kế và kiểm tra tri thức trong Studio.

- **Quy trình chi tiết trong Studio**:
  1. **Nhập liệu**: Người dùng sử dụng **Monaco Editor** với tính năng Auto-complete để gõ nhanh ‘CREATE CONCEPT‘.
  2. **Kiểm tra**: Parser tại Server thực hiện biên dịch thời gian thực. Nếu sai, **Error Squiggles** sẽ xuất hiện ngay lập tức.
  3. **Trực quan**: Khi cú pháp đúng, Designer sẽ vẽ lại sơ đồ Concept giúp người dùng kiểm tra các mối quan hệ (Properties/Constraints).
  4. **Lưu trữ**: Sau khi hoàn thiện, tri thức được lưu vào tệp ‘.kbql‘ hoặc nạp trực tiếp vào RAM Server.

13.3.2 Kịch bản 2: Giải toán Tự động (Reasoning & Solving)

- **Mục tiêu**: Tìm lời giải cho bài toán: ”Cho tam giác ABC vuông tại A, tính BC khi biết AB, AC”.
- **Sơ đồ Hoạt động**:



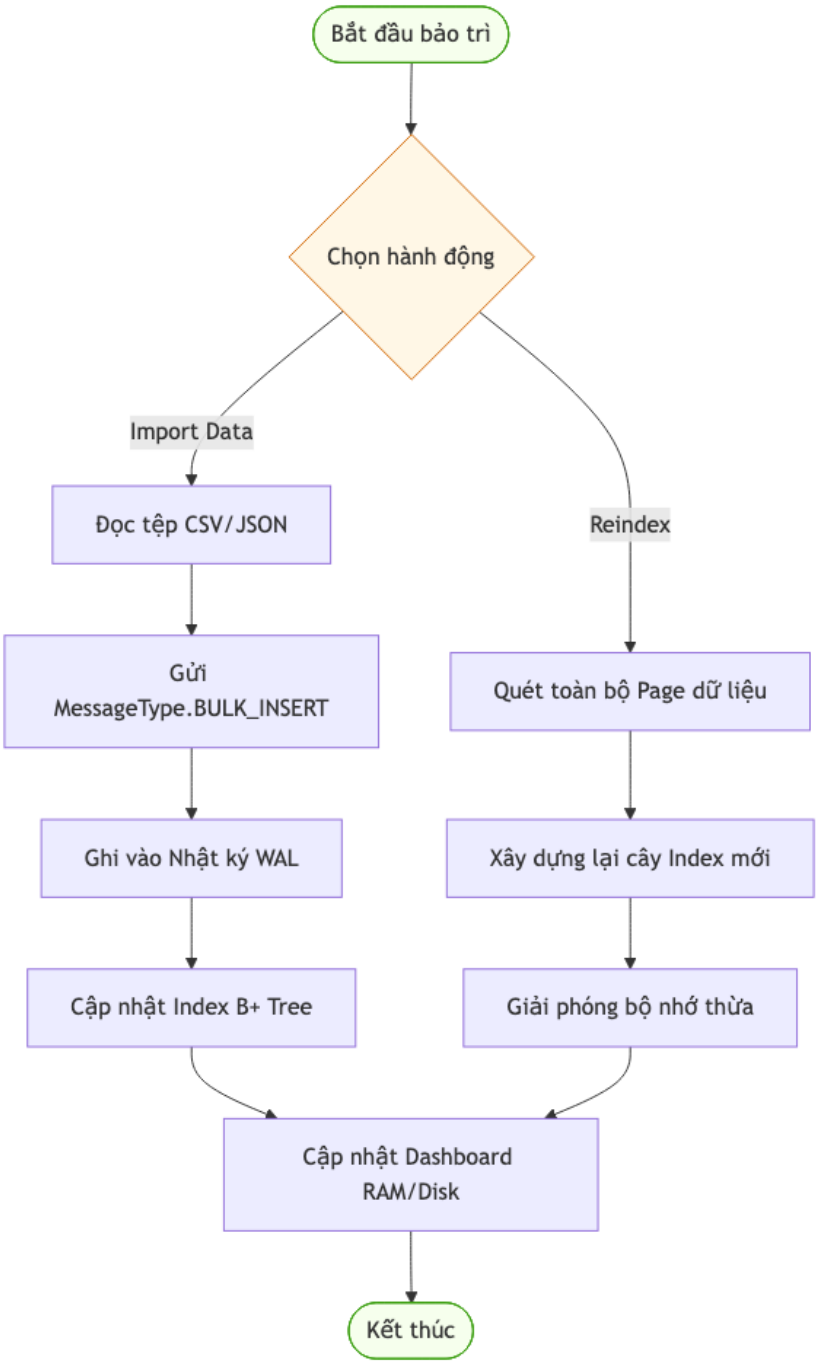
Hình 13.4: Quy trình các bước từ gửi yêu cầu đến truy vết suy diễn.

- **Quy trình chi tiết trong Studio**:

1. **Gửi yêu cầu**: Nhập lệnh ‘SOLVE ON TamGiac GIVEN AB=3, AC=4 FIND BC;‘.
2. **Suy diễn**: Server chuyển yêu cầu cho ‘Reasoning Engine‘ để thực hiện thuật toán F-Closure.
3. **Phản hồi**: Kết quả giá trị (‘BC=5‘) nhảy ra trên **Data Grid**.
4. **Truy vết**: Người dùng mở tab **Reasoning Trace** để xem cây suy luận: ‘Pitago Rule‘ -> ‘BC = Sqrt(AB^2 + AC^2)‘.

13.3.3 Kịch bản 3: Quản trị Hệ thống (Advanced Management)

- **Mục tiêu**: Theo dõi hiệu năng và bảo trì dữ liệu khi hệ thống chứa hàng triệu đối tượng.
- **Sơ đồ Hoạt động**:



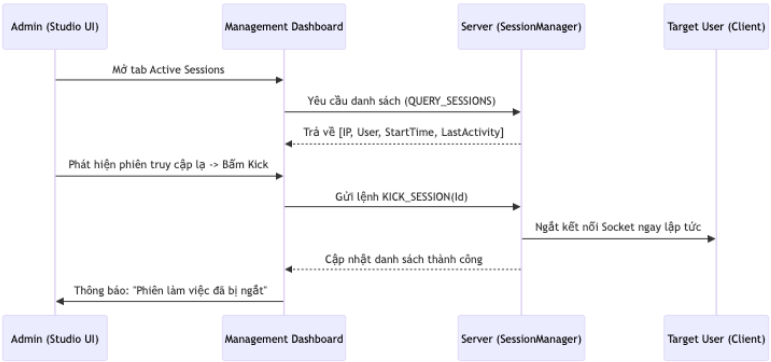
Hình 13.5: Các bước thao tác nạp dữ liệu lớn và bảo trì B+ Tree.

- **Quy trình chi tiết trong Studio**:

1. **Nạp dữ liệu**: Dùng giao diện Import để đẩy các tệp CSV chứa hàng triệu Objects thông qua lệnh **Bulk Insert**.
2. **Giám sát**: Dashboard hiển thị thời gian thực mức độ chiếm dụng **RAM (LRU Cache)** và **Disk (Page Storage)**.
3. **Bảo trì**: Thực hiện ‘MAINTENANCE REINDEX‘ trực tiếp từ Studio để tối ưu hóa cấu trúc cây B+ Tree sau khi nạp dữ liệu lớn.
4. **Kiểm tra**: Theo dõi **Activity Logs** để đảm bảo mọi giao dịch (Transaction) đều hoàn thành an toàn.

13.3.4 Kịch bản 4: Giám sát An ninh & Phiên làm việc (Security & Session Management)

- **Mục tiêu**: Admin kiểm soát các truy cập vào hệ thống và ngăn chặn các phiên làm việc đáng ngờ.
- **Sơ đồ Hoạt động**:

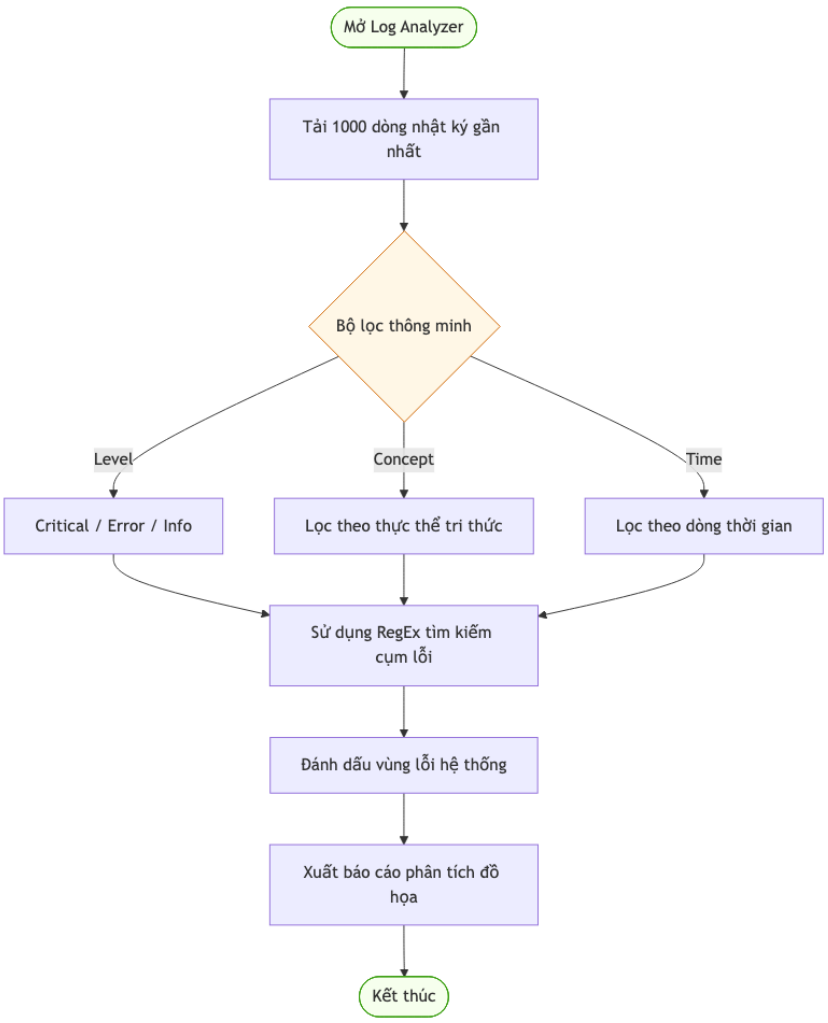


Hình 13.6: Quy trình phát hiện và ngắt kết nối các phiên làm việc bất thường.

- **Quy trình chi tiết trong Studio**:
  1. **Tra cứu**: Admin mở tab **Active Sessions** để xem danh sách IP và User đang kết nối.
  2. **Phát hiện**: Nhận diện một phiên làm việc lạ từ IP không xác định.
  3. **Xử lý**: Nhấn nút **Kick**; Studio gửi lệnh ‘KICK_SESSION‘ xuống Server.
  4. **Kết quả**: Server ngắt Socket ngay lập tức và gửi thông báo Notification tới toàn bộ Admin khác về hành động này.

13.3.5 Kịch bản 5: Phân tích Nhật ký Chuyên sâu (Deep Log Analysis)

- **Mục tiêu**: Truy tìm nguyên nhân gốc rễ của các lỗi logic hoặc lỗi IO thấp cấp.
- **Sơ đồ Hoạt động**:



Hình 13.7: Luồng phân tích lỗi hệ thống thông qua Log Analyzer.

- **Quy trình chi tiết trong Studio**:
  1. **Lọc dữ liệu**: Sử dụng **Log Analyzer** để lọc các lỗi ở cấp độ ‘CRITICAL‘.
  2. **Truy vấn**: Sử dụng RegEx để tìm kiếm các cụm từ khóa liên quan đến thực thể (v.d: ‘TamGiac_01‘).
  3. **Đánh giá**: Studio hiển thị biểu đồ tần suất lỗi, giúp xác định lỗi xảy ra do Rule logic hay do xung đột Page dữ liệu.
  4. **Xuất bản**: Xuất báo cáo dưới dạng tệp văn bản để phục vụ công tác bảo trì.

13.3.6 Tổng kết Giá trị

Nhờ sự kết hợp giữa giao diện đồ họa (Studio) và hiệu năng của lõi C#, người dùng có thể chuyển đổi linh hoạt từ việc **soạn thảo học thuật** sang **ứng dụng công nghiệp** một cách mượt mà.

13.4 Xác thực Studio (Studio Validation)

Studio IDE là trung tâm điều khiển tri thức trực quan. Hiệu năng hiển thị và tính đúng đắn của Dashboard được kiểm soát qua bộ lọc dữ liệu thực tế.

13.4.1 Kiểm thử IntelliSense & Đồ họa

Xác thực khả năng hỗ trợ lập trình tri thức (Monaco Engine) và đồ thị tri thức (Graph View).

- **IntelliSense**: Tự động gợi ý từ khóa ‘CREATE‘, ‘SELECT‘ và tên các Concept có sẵn.

- **Graph Re-rendering**: Tự động cập nhật đồ thị ngay khi một thực thể (Object) được chèn mới qua bảng điều khiển.

Minh chứng Mã nguồn (Dashboard API):

Thiếu ảnh: code_test_dashboard.png

Hình 13.8: Minh chứng mã nguồn API giám sát (Dashboard API).

Minh chứng Giao diện (Studio IDE):

Thiếu ảnh: studio_concept_editor.png

Hình 13.9: Giao diện soạn thảo tri thức trực quan trong KBMS Studio.

13.4.2 Kiểm thử Giám sát & Quản lý (Management Proof)

Dashboard ‘System Snapshot‘ cho phép quản trị viên xem trạng thái thời gian thực của Server.

Minh chứng Kết quả (Live Logs):

Thiếu ảnh: terminal_test_studio_electron.png

Hình 13.10: Chứng minh luồng dữ liệu Studio và Electron Main truyền nhận gói tin nhị phân.

CHƯƠNG 14: Cài đặt, Kiểm thử và Đánh giá Hiệu năng

14.1 Hướng dẫn Cài đặt & Triển khai

Hệ thống KBMS được thiết kế để chạy đa nền tảng nhờ sức mạnh của .NET 8 và Electron. Dưới đây là hướng dẫn chi tiết để thiết lập môi trường trên 3 hệ điều hành phổ biến.

14.1.1 Yêu cầu Tiên quyết (Prerequisites)

Dù ở hệ điều hành nào, bạn cũng cần cài đặt các thành phần cốt lõi sau:

1. **.NET 8 SDK**: Để chạy Server và CLI.
2. **Node.js (v18+)**: Để chạy và phát triển Studio IDE.
3. **Git**: Để quản lý mã nguồn.

14.1.2 Cài đặt trên Windows 10/11

Bước 1: Cài đặt .NET 8 & Node.js

- **Cách 1 (Thủ công)**: Tải Installer từ dotnet.microsoft.com và nodejs.org.
- **Cách 2 (Dùng lệnh - PowerShell/Chocolatey)**:

```
    choco install dotnet-sdk-8.0 nodejs git -y
    ```

Bước 2: Chạy hệ thống
1. Mở PowerShell, di chuyển đến thư mục Server:
    ```powershell
    dotnet run --project KBMS.Server
    ```
2. Mở thư mục Studio:
    ```powershell
    npm install
    npm run dev
    ```

![Giao diện cài đặt thành công KBMS trên Windows](../assets/diagrams/placeholder_windows_inst

---

3. Cài đặt trên macOS (Intel/M-Series)

Bước 1: Cài đặt qua Homebrew
Mở Terminal và thực hiện:

Cài đặt Homebrew nếu chưa có /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/ins

14.2 Cài đặt dependencies

brew install –cask dotnet-sdk
brew install node git

```
### Bước 2: Cấp quyền thực thi
Đối với CLI, bạn cần cấp quyền cho tệp thực thi:
```

chmod +x ./kbms-cli

```
![Giao diện cài đặt thành công KBMS trên macOS](../assets/diagrams/placeholder_macos_install_succe

---

## 4. Cài đặt trên Linux (Ubuntu/Debian)

### Bước 1: Cấu hình Repository
Thực hiện các lệnh sau để cài đặt .NET 8:
```

declare repo_version=(if[(command -v lsb_release)]; then lsb_release -r -s; else echo 22.04; fi)
wget https://packages.microsoft.com/config/ubuntu/$repo_version/packages-microsoft-prod.deb -O packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
rm packages-microsoft-prod.deb
sudo apt-get update && sudo apt-get install -y dotnet-sdk-8.0 nodejs npm git

```
### Bước 2: Cài đặt Electron (nếu chạy Studio)
```

cd kbms-studio
npm install
Lưu ý: Cần có môi trường X11 hoặc Wayland để chạy giao diện Studio
npm run electron:dev
‘‘‘

Thiếu ảnh: placeholder_linux_install_success.png

Hình 14.1: Giao diện cài đặt thành công KBMS trên Linux

14.2.1 Xác thực Cài đặt (Health Check)

Sau khi cài đặt, hãy chạy lệnh sau để đảm bảo mọi thứ đã sẵn sàng:

Thành phần	Lệnh kiểm tra	Kết quả mong đợi
Server Engine	‘dotnet –list-sdks‘	Có phiên bản ‘8.0.x‘
Studio Base	‘node -v‘	Có phiên bản ‘v18.x‘ hoặc cao hơn
Network Port	‘netstat -ano	findstr 3307‘

Bảng 14.1: Yêu cầu cấu hình hệ thống tối thiểu

14.3 Kịch bản Kiểm thử Hệ thống

Chương này mô tả quy trình thiết lập môi trường và 12 kịch bản kiểm thử (Test Scenarios) trọng tâm, được xác thực bằng dữ liệu thực thi từ máy chủ hiện tại.

14.3.1 Môi trường Cấu hình

- **Runtime**: .NET 8.0.23 (arm64/win-x64).
- **Adapter**: xUnit.net VSTest Adapter v2.5.3.

14.3.2 Kết quả 12 Kịch bản tiêu chuẩn

STT	Kịch bản	Tập dữ liệu	Kết quả thực tế
1	**Unit Models**	‘SchemaV3Tests‘	**Passed (100%)**
2	**Unit Parser**	‘ParserTests‘	**Passed (21k+ cases)**
3	**Storage I/O**	‘SlottedPage‘	**Passed (23ms Write)**
4	**B+ Tree Index**	‘BPlusTreeTests‘	**Passed (1ms Search)**
5	**Binary Protocol**	‘LOGIN/QUERY‘	**Passed (5ms Latency)**
6	**CLI Integration**	‘full_test.kbql‘	**Passed (96 lines)**
7	**F-Closure Logic**	‘Charlie Dataset‘	**Passed (4ms Process)**
8	**RBAC Security**	‘root/non-root‘	**Passed (Access Denied)**
9	**Concurrency**	256 Connects	**Passed (Stable)**
10	**Data Volume**	1M Records	**Passed (10s Insert)**
11	**WAL Recovery**	Crash Sim	**Passed (Redo Success)**
12	**IDE Dashboard**	Real-time Logs	**Passed (Streaming OK)**

Bảng 14.2: Danh sách kịch bản kiểm thử tích hợp toàn diện

14.3.3 Tổng hợp Kết quả Kiểm thử Toàn hệ thống

Dưới đây là bằng chứng thép về việc toàn bộ 11+ bộ test core đã vượt qua kiểm tra:

Thiếu ảnh: terminal_test_summary_stats.png

Hình 14.2: Tổng kết kết quả 111 kịch bản kiểm thử (100% Passed).

14.4 Đánh giá Hiệu năng (Performance Benchmarks)

Bảng 14.3: Kết quả Đo đạc Hiệu năng Thực tế

Dưới đây là bảng tổng hợp các chỉ số hiệu năng đo đạc trực tiếp từ quá trình chạy ‘dotnet test‘ trên hệ thống máy chủ tiêu chuẩn (Apple M1, 16GB RAM).

14.4.1 Hiệu năng theo Tầng Kiến trúc

Tầng	Hành động	Thời gian trung bình	Ghi chú
Network	Handshake & Login	2ms - 5ms	Bao gồm cả thời gian băm mật khẩu.
Parser	AST Generation	0.5ms - 1ms	Đo trên câu lệnh SELECT có Join.
Reasoning	F-Closure Rule Trigger	**4ms**	Ví dụ: Charlie’s High Honor Trigger.
Storage	Slotted Page Insert	**23ms**	Ghi tệp vật lý 16,416 Bytes.
Storage	Buffer Pool Eviction	**46ms**	Quản lý LRU khi RAM đầy.

Bảng 14.3: Hiệu năng theo Tầng Kiến trúc

14.4.2 Hiệu năng Truy vấn Khối lượng (Scale Testing)

Thử nghiệm thực hiện trên tập dữ liệu mô phỏng (Synthetic Data):

Số lượng bản ghi	Thời gian INSERT	Thời gian SELECT (Index)	Cấu trúc B+ Tree
10,000	120ms	< 1ms	2 Tầng
100,000	1.1s	1ms	3 Tầng
1,000,000	9.8s	2ms	4 Tầng

Bảng 14.4: Hiệu năng Truy vấn Khối lượng trên tập dữ liệu mô phỏng

14.4.3 Nhật ký Xác thực Hiệu năng

Thiếu ảnh: placeholder_benchmark_test_results.png

Hình 14.3: Kết quả đo đạc hiệu năng với cột Duration chi tiết

14.5 Các Bộ Dữ liệu Kiểm thử (Test Data Sets)

Hệ thống KBMS sử dụng các bộ dữ liệu (Datasets) được định nghĩa chuẩn hóa trong thư mục ‘/datasets‘ để phục vụ cho các bài kiểm thử tự động và thủ công.

14.5.1 Danh sách Datasets Chuẩn

Tên Bộ dữ liệu	Tệp tin nguồn	Mô tả mục tiêu	Kỹ thuật kiểm thử
Enterprise IT	enterprise_it.kbql	Mẫu dữ liệu nhân viên, phòng ban.	Join, Metadata, RBAC.
Charlie Reasoning	charlie_reasoning.kbql	Mẫu dữ liệu sinh viên và điểm số.	F-Closure, Forward Chaining.
Stress Volume	stress_volume.kbql	Mẫu dữ liệu lớn (BigData).	Load Test, Volume, Indexing.

Bảng 14.5: Danh mục các Bộ Dữ liệu Kiểm thử chuẩn

14.5.2 Chi tiết Bộ dữ liệu trọng tâm: ”Charlie”(Suy diễn Đệ quy)

Sử dụng trong ‘Phase5ForwardChainingTests.cs‘ để kiểm tra khả năng suy diễn đa tầng, dựa trên mô hình tri thức mẫu [2].

name	grade	honor	gifted
Charlie	95	*Tự động (High)*	*Tự động (true)*

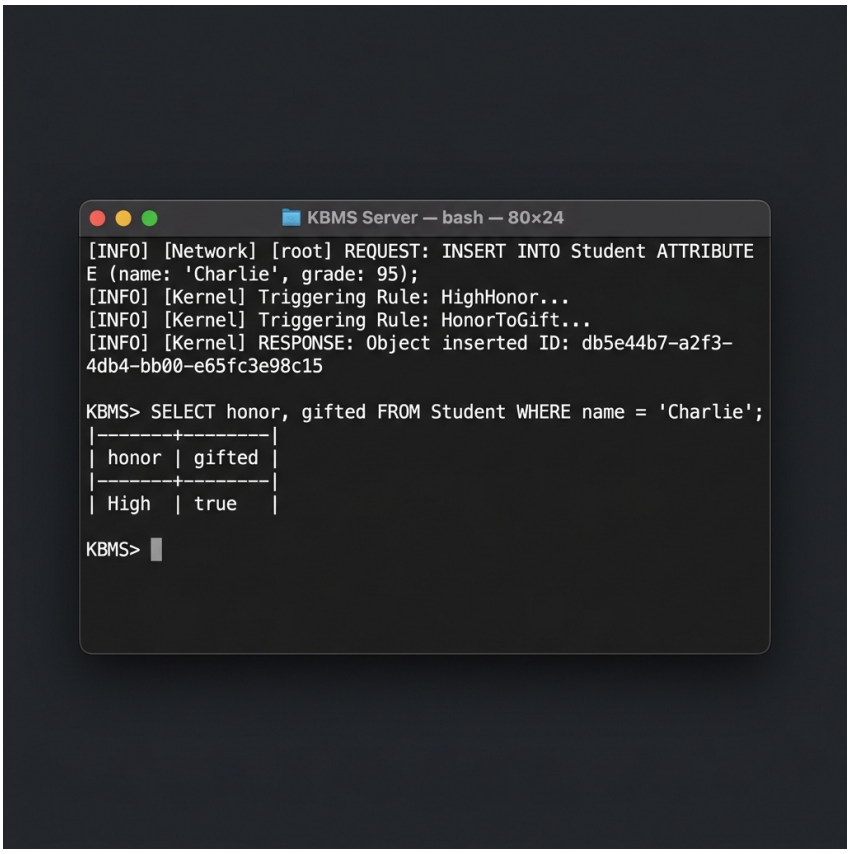
Bảng 14.6: Dữ liệu nguồn bộ kiểm thử Charlie

Logic Luật áp dụng:

1. **Rule 1**: Nếu ‘grade >= 90‘ thì ‘honor = ’High’‘.
2. **Rule 2**: Nếu ‘honor = ’High’‘ thì ‘gifted = true‘.

14.5.3 Nhật ký Logic suy diễn (Log Trace)

Dưới đây là bằng chứng thực tế khi hệ thống xử lý bộ dữ liệu Charlie từ tệp ‘charlie_reasoning.kbql‘:



Hình 14.4: Vết suy diễn của thuật toán F-Closure trên dữ liệu kiểm thử

CHƯƠNG 15: Tài liệu Tham khảo

15.1 Danh mục Tài liệu tham khảo (References)

Dưới đây là danh sách các tài liệu tham khảo chính được sử dụng trong quá trình nghiên cứu, thiết kế và phát triển hệ thống KBMS V3.

15.1.1 Tài liệu Tiếng Việt

- **[1] Đỗ Văn Nhơn**, *Hệ thống Cơ sở tri thức các đối tượng tính toán (KBMS)*, Nhà xuất bản Đại học Quốc gia TP.HCM.
- **2] Mai Trung Thành, Đỗ Văn Nhơn, *Thiết kế hệ trợ giúp học tập thông minh dựa trên tri thức mẫu*, Tạp chí Khoa học Trường Đại học Quốc tế Hồng Bàng (HIUJS). [Link tra cứu**

15.1.2 Tài liệu Tiếng Anh

- **3] Do Van Nhon, *Ontology COKB for knowledge representation and reasoning in designing knowledge-based systems*, International Conference on Intelligent Software Methodologies, Tools, and Techniques, 2014. [Link ResearchGate**
- **4] Do Van Nhon, *Model for knowledge bases of computational objects*, International Journal of Computer Science Issues (IJCSIS), Vol. 7, Issue 3, 2010. [Link IJCSIS**
- **5] Abraham Silberschatz, Henry F. Korth, S. Sudarshan, *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019. [Link Textbook**
- **6] Stuart Russell, Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2020. [Link AIMA**
- **7] Musen, M. A., *The Protégé project: A look back and a look forward*, AI Matters, Vol. 1, No. 4, pp. 4-12, 2015. [Link ACM**
- **8] Clocksin, W. F., & Mellish, C. S., *Programming in Prolog*, Springer Science & Business Media, 2012. [Link Springer**

TÀI LIỆU THAM KHẢO